

I. LIMBAJE FORMALE ȘI AUTOMATE

Curs 1

Durata:

2 ore

Curs 1 Gramatici și automate finite

Descriere Generală: Se studiază noțiunile de gramatică, automat finit și legătura dintre ele din punct de vedere al limbajului

Obiective:

- însușirea noțiunilor: gramatică formală, limbaj formal, automat finit
- cunoașterea clasificării Chomsky, a metodei de construire a gramaticii care generează un limbaj ce conține cuvântul vid, a verificării recursivității
- cunoașterea funcționării unui automat finit și a minimizării acestuia
- echivalarea limbajelor regulate cu limbajele acceptate de automate finite

Cuprins

- I.1. Clasificarea gramaticilor după Chomsky
- I.2. Automate finite
- I.3. Relația dintre automate finite și limbaje regulate

Conținut Curs

I.1. Clasificarea gramaticilor după Chomsky

Definiția 1.1. Fie V o mulțime finită și nevidă numită alfabet. Pentru $\forall k \in \mathbb{N}$ notăm $V^k = \bigcup_{k \in \mathbb{N}} V^k$. Orice cuvânt $(w_1, \dots, w_k) \in V^k$ îl notăm cu $w_1 \cdots w_k$, iar numărul k este lungimea cuvântului. Mulțimea V^0 are un singur element, numit cuvântul vid și notat cu ε . Orice submulțime $L \subset V^*$ se numește limbaj formal sau pe scurt limbaj.

Pentru descrierea limbajelor sunt folosite următoarele metode:

- enumerarea elementelor, când limbajul este finit

- folosirea noțiunii de gramatică, atunci când limbajul este infinit.

Definiția 1.2. O gramatică este un sistem $G = (N, \Sigma, P, S)$ unde:

- N este un alfabet ale cărui elemente se numesc neterminale și se notează cu litere mari.
- Σ este un alfabet ale cărui elemente se numesc terminale și se notează cu litere mici.
- $P \subseteq (N \cup \Sigma)^* N (N \cup \Sigma)^* \times (N \cup \Sigma)^*$ este mulțimea regulilor de producție.
- $S \in N$ este simbolul inițial.

Definiția
gramaticii

Convenim ca orice element $(u, v) \in P$ să-l notăm $u \rightarrow v$, specificând astfel că se înlocuiește cuvântul u cu cuvântul v . Pentru orice producție $u \rightarrow v$ există cel puțin o variabilă în cuvântul u , deoarece $(u, v) \in (N \cup \Sigma)^* N (N \cup \Sigma)^* \times (N \cup \Sigma)^*$.

Definiția 1.3. Pe mulțimea cuvintelor se definește relația binară " $\Rightarrow$ " astfel: $w \Rightarrow \omega$ (se citește w derivează direct pe ω) dacă

- i) dacă $\exists \alpha, u, \beta$ astfel încât $w = \alpha u \beta$ unde $\alpha, u, \beta \in (N \cup \Sigma)^*$,
- ii) există o descompunere $\omega = \alpha v \beta$, $v \in (N \cup \Sigma)^*$ și $u \rightarrow v \in P$.

Relația de
derivare

Această relație nu este neapărat reflexivă și tranzitivă.

Definiția 1.4. Închiderea reflexivă și tranzitivă a relației $\Rightarrow$ o notăm cu $\Rightarrow^*$ și o definim astfel: $w_1 \Rightarrow^* w_2$ dacă $w_1 = w_2$ sau există $k \geq 1$ și $Z_0, \dots, Z_k \in (N \cup \Sigma)^*$ astfel încât $w_1 = Z_0 \Rightarrow Z_1 \Rightarrow \dots \Rightarrow Z_k = w_2$. Închiderea tranzitivă o notăm cu $\Rightarrow^+$ și se definește eliminând din $\Rightarrow^*$ relația $w_1 = w_2$.

Derivarea în k pași o notăm cu $\Rightarrow^k$.

Definiția 1.5. Se numește limbaj generat de o gramatică mulțimea: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$. Două gramatici care generează același limbaj se numesc echivalente.

Limbaj
generat de o
gramatică

Impunând anumite restricții producțiilor unei gramatici, Chomsky a introdus trei tipuri de gramatici:

1. Gramatici dependente de context: orice producție $(u \rightarrow v) \in P$ satisface condiția $|u| \leq |v|$ (gramatici de tipul 1), unde $|x|$ este lungimea lui x .
2. Gramatici independente de context (GIC): orice producție $(u \rightarrow v) \in P$ satisface condiția $|u| = 1, u \in N, v \neq \varepsilon$ (gramatici de tipul 2).
3. Gramatici regulate: orice producție $(u \rightarrow v) \in P$ satisface condiția $u \in N, v \in \Sigma^* \cup \Sigma^* N, v \neq \varepsilon$ (gramatici de tipul 3); dacă $v \in \Sigma \cup \Sigma N$, gramatica se numește în formă redusă.
4. Gramatici care nu au nici un fel de restricții impuse asupra producțiilor (gramatici de tipul 0).

Clasificarea
Chomsky

Dacă se notează cu $L_i, i \in \{0, 1, 2, 3\}$ familia limbajelor generate de gramatica de tip i atunci $L_3 \subset L_2 \subset L_1 \subset L_0$.

Teorema 1.1. Pentru orice gramatică de tip $i \in \{1, 2, 3\}$, există o gramatică de același tip astfel încât simbolul inițial S să nu apară în membrul drept al nici unei producții.

Demonstrație. Fie $G = (N, \Sigma, P, S)$ o gramatică de tipul $i \in \{1, 2, 3\}$ și $S_1 \notin N \cup \Sigma$. Considerăm gramatica $G_1 = (N_1, \Sigma_1, P_1, S_1)$ unde $N_1 = N \cup \{S_1\}$, $\Sigma_1 = \Sigma$, $P_1 = P \cup \{S_1 \rightarrow w / (S \rightarrow w) \in P\}$. Evident, cele două gramatici sunt de același tip. Fie $w \in L(G)$; atunci, în G , există derivarea $S \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_k = w$. Din $(S \rightarrow w_1) \in P$ rezultă că există $(S_1 \rightarrow w_1) \in P_1$. Cum $P \subset P_1$, înseamnă că toate derivările $w_i \Rightarrow w_{i+1}$ din G au loc și în G_1 .

Deci, în G_1 există derivarea $S_1 \xRightarrow{*} w$, adică $w \in L(G_1)$. Invers, fie $w \in L(G_1)$. Înseamnă că în G_1 are loc derivarea $S_1 \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_k = w$. Din $(S_1 \rightarrow w_1) \in P_1$ rezultă că $(S \rightarrow w_1) \in P$, de unde obținem că w_1 nu conține pe S_1 ; deci, în gramatica G există derivarea $S \Rightarrow w_1$. Prin inducție după i se arată că niciun w nu conține pe S_1 , deci toate derivările $w_i \Rightarrow w_{i+1}$ din G_1 au loc și în G . Deducem că în G are loc derivarea $S \Rightarrow w_1 \Rightarrow \dots \Rightarrow w_k = w$; deci $w \in L(G)$. $\square$

Teorema 1.2. Dacă L este un limbaj de tipul $i \in \{1, 2, 3\}$ atunci există o

gramatică $G = (N, \Sigma, P, S)$ care generează limbajul $L \cup \{\varepsilon\}$ astfel încât

1) simbolul inițial S nu apare în membrul drept al niciunei producții din G

2) în mulțimea P există producția $S \rightarrow \varepsilon$

3) gramatica $G_1 = (N, \Sigma, P_1, S)$, unde $P_1 = P \setminus \{S \rightarrow \varepsilon\}$, este de tipul $i \in \{1, 2, 3\}$ și generează limbajul L .

Demonstrație. Din teorema anterioară rezultă existența gramaticii $G_1 = (N, \Sigma, P_1, S)$ de tipul $i \in \{1, 2, 3\}$, care generează limbajul L iar simbolul inițial S nu apare în membrul drept al niciunei producții din G_1 . Adăugând mulțimii P_1 producția $S \rightarrow \varepsilon$ rezultă că $L(G) = L \cup \{\varepsilon\}$. $\square$

Spunem că limbajul L peste alfabetul Σ este recursiv dacă există un algoritm care pentru orice secvență $w \in \Sigma^*$ determină dacă $w \in L$.

Teorema 1.3. Limbajele de tip 1 sunt recursive.

Demonstrație. Se utilizează

Algoritmul REC

Intrare: gramatica $G = (N, \Sigma, P, S)$ de tipul 1 și secvența $w \in \Sigma^*$ de lungime l

Ieșire: mesajul DA dacă $w \in L(G)$ și mesajul NU în caz contrar.

Metoda:

P1. Dacă $l > 0$, mergi la P3.

P2. Dacă $(S \rightarrow \varepsilon) \in P$ atunci tipărește DA
altfel tipărește NU

STOP.

P3. $M_0 = \{S\}$, $i := 0$

P4. $M_{i+1} = M_i \cup \{\beta \in (N \cup \Sigma)^* \mid |\beta| \leq l \text{ și } \exists \alpha \in M_i \text{ cu } \alpha \Rightarrow \beta\}$

P5. Dacă $M_{i+1} \neq M_i$ atunci $i := i + 1$ și mergi la P4.

P6. Dacă $w \in M_i$ atunci tipărește DA
altfel tipărește NU

STOP. $\square$

Testarea
recursivității

I.2. Automate finite

Un automat finit are două componente principale: banda de intrare și unitatea centrală. Banda de intrare este infinită la dreapta și este împărțită în

celule, în fiecare putând fi înscris un simbol din alfabetul de intrare Σ . Unitatea centrală se poate afla într-un număr finit de stări. În funcție de starea curentă și de simbolul citit de pe banda de intrare ea își poate schimba starea.

Definiția 2.1. Se numește automat finit un sistem $A = (Q, \Sigma, \delta, q_0, F)$ unde :

- Q este mulțimea stărilor automatului
- Σ este alfabetul de intrare
- $\delta : Q \times \Sigma \rightarrow P(Q)$ este funcția de tranziție
- q_0 este starea inițială
- F este mulțimea stărilor finale.

Definiția
automatului
finit.

Funcționarea automatului se descrie cu ajutorul configurațiilor.

Definiția 2.2. O configurație a automatului este o pereche (q, α) unde q este starea curentă iar α este secvența de pe banda de intrare, rămasă de citit. Configurația inițială este (q_0, w) iar configurația finală (q, ε) , unde $q \in F$.

Configurație

Funcționarea automatului se realizează prin pași. Un pas reprezintă trecerea de la o configurație la alta: $(q, a\alpha) \vdash (p, \alpha)$ dacă $p \in \delta(q, a)$. Notăm cu $\vdash^+$, $\vdash^*$, $\vdash^i$ închiderea tranzitivă, închiderea reflexivă și tranzitivă și respectiv trecerea în i pași.

Limbaj
acceptat de un
automat finit

Definiția 2.3. Limbajul acceptat de un automat finit A este mulțimea

$$L(A) = \{w \in \Sigma^* \mid (q_0, w) \vdash^* (p, \varepsilon), p \in F\}.$$

Descrierea și funcționarea unui automat finit $A = (Q, \Sigma, \delta, q_0, F)$ pot fi urmărite mai ușor dacă i se asociază un graf G , astfel:

- mulțimea vârfurilor lui G este mulțimea Q a stărilor
- dacă $p, q \in Q$, $a \in \Sigma$ și $q \in \delta(p, a)$ atunci se trasează un arc de la vârful p la vârful q , care va fi etichetat cu a
- fiecare stare va fi scrisă într-un cerc
- starea inițială va fi precedată de $\rightarrow$ iar stările finale vor fi scrise într-un cerc dublu.

Un exemplu de astfel de reprezentare este graful din figura 2.1

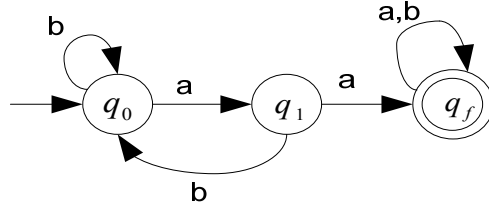


Figura 2.1

Definiția 2.4. Automatul finit $A = (Q, \Sigma, \delta, q_0, F)$ se numește

- **determinist**, dacă $|\delta(q, a)| \leq 1, \quad \forall q \in Q, \forall a \in \Sigma$
- **complet determinist**, dacă $|\delta(q, a)| = 1, \quad \forall q \in Q, \forall a \in \Sigma$
- **nedeterminist**, în caz contrar.

Este clar că este mult mai convenabil să se lucreze cu automate finite complet deterministe. De aceea următoarea teoremă este foarte importantă din punct de vedere practic.

Teorema 2.1. Pentru orice automat finit nedeterminist $A = (Q, \Sigma, \delta, q_0, F)$ există un automat finit complet determinist A' astfel încât $L(A) = L(A')$.

Demonstrație. Construim automatul A' astfel $A' = (Q', \Sigma, \delta', q'_0, F')$:

- $Q' = P(Q)$
- $q'_0 = \{q_0\}$
- $F' = \{S \subseteq Q / S \cap F \neq \emptyset\}$
- $\delta': Q' \times \Sigma \rightarrow Q', \delta'(X, a) = \left\{ q \in Q / q \in \bigcup_{p \in X} \delta(p, a) \right\}$

Este evident că automatul A' este complet determinist.

Prin inducție după i se arată că

$$(X, w) \stackrel{i}{\vdash} (Y, \varepsilon) \Leftrightarrow Y = \{q \in Q / \exists p \in X, (p, w) \stackrel{i}{\vdash} (q, \varepsilon)\}$$

unde $i = |w|$. Folosind această echivalență, pentru $x \in \Sigma^*, |x| = i$, rezultă

$$\begin{aligned} x \in L(A') &\Leftrightarrow (\{q_0\}, x) \stackrel{i}{\vdash} (Y, \varepsilon) \text{ și } Y \in F' \Leftrightarrow \\ &\Leftrightarrow \exists q \in Y \cap F \text{ și } (q_0, x) \stackrel{i}{\vdash} (q, \varepsilon) \Leftrightarrow x \in L(A), \end{aligned}$$

deci $L(A) = L(A')$. $\square$

Construirea
automatului
finit complet
determinist

Este de dorit ca un automat finit să aibă o structură cât mai simplă; o primă posibilitate constă în eliminarea stărilor inaccesibile.

Definiția 2.5. Fie $A = (Q, \Sigma, \delta, q_0, F)$ un automat finit și $q \in Q$. Spunem că starea q este accesibilă dacă există $x \in \Sigma^*$ astfel încât $(q_0, x) \xrightarrow{*} (q, \varepsilon)$. Altfel, starea q se numește inaccesibilă.

Stare
accesibilă

Determinarea stărilor accesibile se face cu următorul algoritm

Algoritmul ACC

Intrare: automatul finit $A = (Q, \Sigma, \delta, q_0, F)$

Ieșire: mulțimea Q_a a stărilor accesibile ale automatului A

Metoda:

P1. $Q_0 = \{q_0\}$, $i := 0$

P2. $Q_{i+1} = Q_i \cup \{p \in Q / \exists q \in Q_i \text{ și } \exists a \in \Sigma \text{ cu } p \in \delta(q, a)\}$

P3. Dacă $Q_{i+1} \neq Q_i$ atunci $i := i + 1$ și mergi la P2.

P4. $Q_a = Q_i$, STOP.

Teorema 2.2. Fiind dat automatul $A = (Q, \Sigma, \delta, q_0, F)$, există un automat $A_a = (Q_a, \Sigma, \delta_a, q_0, F_a)$ care are toate stările accesibile și $L(A) = L(A_a)$.

Demonstrație. Componentele automatului A_a se obțin astfel:

- Q_a se calculează cu algoritmul ACC
- $\delta_a : Q_a \times \Sigma \rightarrow P(Q_a)$ este restricția funcției δ
- $F_a = F \cap Q_a$

Eliminarea stărilor inaccesibile se poate face pentru orice automat finit (determinist sau nu). În continuare vom presupune ca automatele finite cu care lucrăm sunt complet deterministe și au numai stări accesibile. $\square$

Definiția 2.6. Fie $A = (Q, \Sigma, \delta, q_0, F)$ un automat finit, $q_1, q_2 \in Q$ și $x \in \Sigma^*$. Spunem că secvența x distinge stările q_1 și q_2 dacă $(q_1, x) \xrightarrow{*} (p_1, \varepsilon)$, $(q_2, x) \xrightarrow{*} (p_2, \varepsilon)$ și exact una din stările p_1 și p_2 este stare finală.

Definiția 2.7. Spunem că stările q_1 și q_2 sunt k -echivalente și notăm $q_1 \equiv^k q_2$ dacă nu există nicio secvență x , cu $|x| \leq k$, care să distingă q_1 și q_2 . Stările q_1 și q_2 se numesc echivalente și se notează $q_1 \equiv q_2$ dacă sunt k -echivalente pentru orice număr natural k .

Stări
echivalente

Este evident că $\equiv$ și $\equiv^k$ sunt relații de echivalență. Secvența vidă ε distinge stările q_1 și q_2 dacă și numai dacă exact una dintre ele este stare finală. Deci, două stări sunt 0-echivalente dacă și numai dacă sunt ambele finale sau niciuna nu este finală. De asemenea, din $q_1 \equiv^{k+1} q_2$ rezultă $q_1 \equiv^k q_2$.

Teorema 2.3. Fie q_1 și q_2 două stări ale automatului finit $A = (Q, \Sigma, \delta, q_0, F)$ și k un număr natural. Atunci $q_1 \equiv^{k+1} q_2 \Leftrightarrow \forall a \in \Sigma, \delta(q_1, a) \equiv^k \delta(q_2, a)$.

Demonstrație. Fie $q_1 \equiv^{k+1} q_2$; presupunem că $\exists a \in \Sigma$ astfel încât starea $p_1 = \delta(q_1, a)$ nu este k -echivalentă cu starea $p_2 = \delta(q_2, a)$. Înseamnă că există $x \in \Sigma^*$ cu $|x| \leq k$ care distinge p_1 și p_2 , adică $(p_1, x) \vdash^* (p'_1, \varepsilon)$, $(p_2, x) \not\vdash^* (p'_2, \varepsilon)$ și $p'_1 \in F$, $p'_2 \in Q \setminus F$ (sau invers). Pentru secvența ax , cu $|ax| = |x| + 1 \leq k + 1$, avem

$$\begin{aligned} (q_1, ax) &\vdash (p_1, x) \vdash^* (p'_1, \varepsilon) \\ (q_2, ax) &\vdash (p_2, x) \not\vdash^* (p'_2, \varepsilon). \end{aligned}$$

Rezultă că stările q_1 și q_2 nu sunt în relația $\equiv^{k+1}$, care este o contradicție; deci presupunerea făcută este falsă. Implicația reciprocă se demonstrează în mod analog. $\square$

Definiția 2.8. Automatul finit A este redus dacă toate stările sunt accesibile și nu are stări distincte echivalente.

Teorema 2.4. Pentru orice automat finit complet determinist $A = (Q, \Sigma, \delta, q_0, F)$ există un automat finit redus A' astfel încât $L(A) = L(A')$.

Demonstrație. Presupunem că automatul A are toate stările accesibile. Automatul redus se construiește cu algoritmul următor.

Algoritmul RED

Intrare: automatul finit complet determinist $A = (Q, \Sigma, \delta, q_0, F)$ cu $Q = Q_a$.

Ieșire: automatul finit redus A' astfel încât $L(A) = L(A')$

Metoda:

P1. $Q' = Q_{/\equiv} = \{[q] / q \in Q\}$

P2. $F' = \{[q] / q \in F\}$

P3. $\delta': Q' \times \Sigma \rightarrow Q'$, $\delta'([q], a) = [\delta(q, a)]$, $\forall [q] \in Q', \forall a \in \Sigma$

P4. $A' = (Q', \Sigma, \delta', [q_0], F')$

Construirea
automatului
reduc

Pornind de la $Q_{/\equiv} = \{F, Q - F\}$ și utilizând teorema anterioară se construiesc

relațiile $\equiv^1, \equiv^2, \dots, \equiv^k, \equiv^{k+1}$ și se ia $\equiv^k$; în acest moment se termină pasul P1.

Folosind definiția lui δ' rezultă ușor că automatul A' este redus. Să arătăm că cele două automate sunt echivalente. Fie $x \in \Sigma^*$. Dacă $x \in L(A)$ atunci

$$(q_0, x) \xrightarrow{\delta^*} (q, \varepsilon) \text{ în } A \text{ și } q \in F.$$

Rezultă

$$([q_0], x) \xrightarrow{\delta'^*} ([q], \varepsilon) \text{ în } A' \text{ și } [q] \in F';$$

deci $x \in L(A')$.

Dacă $x \notin L(A)$ atunci

$$(q_0, x) \not\xrightarrow{\delta^*} (q, \varepsilon) \text{ în } A \text{ și } q \notin F.$$

Rezultă

$$([q_0], x) \not\xrightarrow{\delta'^*} ([q], \varepsilon) \text{ în } A' \text{ și } [q] \notin F',$$

adică $x \notin L(A')$; deci, $x \in L(A')$ implică $x \in L(A)$. $\square$

I.3. Relația dintre automate finite și limbaje regulate

Vom arăta că familia limbajelor regulate coincide cu cea a limbajelor acceptate de automate finite.

Teorema 3.1. Pentru orice automat finit A există o gramatică regulată G astfel încât $L(A) = L(G)$.

Demonstrație. Fie automatul $A = (Q, \Sigma, \delta, q_0, F)$; considerăm gramatica

$G = (Q, \Sigma, P, q_0)$, unde producțiile P sunt de forma

- 1) dacă $p, q \in Q, a \in \Sigma$ și $q \in \delta(p, a)$ atunci $(p \rightarrow aq) \in P$
- 2) dacă $q \in F$ atunci $(q \rightarrow \varepsilon) \in P$

Este evident că gramatica este regulată, în forma redusă. Fie $w = a_1 a_2 \cdots a_n \in L(A)$; atunci

$$(q_0, w) = (q_0, a_1 a_2 \cdots a_n) \vdash (q_1, a_2 \cdots a_n) \vdash \cdots \vdash (q_{n-1}, a_n) \vdash (q_n, \varepsilon)$$

cu $q_n \in F$. Din $q_i \in \delta(q_{i-1}, a_i)$ rezultă că $q_{i-1} \rightarrow a_i q_i$ pentru $i \in \{1, 2, \dots, n\}$ iar din $q_n \in F$ rezultă că $(q_n \rightarrow \varepsilon) \in P$. Deci, în G , au loc derivările

$$q_0 \Rightarrow a_1 q_1 \Rightarrow a_1 a_2 q_2 \Rightarrow \cdots a_1 \cdots a_n q_n \Rightarrow a_1 \cdots a_n = w$$

Deci $w \in L(G)$ și $L(A) \subseteq L(G)$. Reluând demonstrația în sens invers, rezultă că $L(G) \subseteq L(A)$, adică $L(A) = L(G)$. $\square$

Teorema 3.2. *Orice limbaj regulat este acceptat de un automat finit determinist.*

Demonstrație. Fie $G = (N, \Sigma, P, S)$ o gramatică regulată în formă redusă.

Fie $X \notin N$ și definim automatul nedeterminist $A = (N \cup \{X\}, \Sigma, \delta, \{S\}, F)$

$$\delta(Y, x) = \begin{cases} \{A \in N \mid \exists (Y \rightarrow xA) \in P\} \cup \{X\} & \text{dacă } Y \neq X, (Y \rightarrow x) \in P \\ \{A \in N \mid \exists (Y \rightarrow xA) \in P\} & \text{dacă } Y \neq X, (Y \rightarrow x) \notin P \\ \emptyset & \text{dacă } Y = X \end{cases}$$

$$F = \begin{cases} \{S, X\} & \text{dacă } (S \rightarrow \varepsilon) \in P \\ \{X\} & \text{în caz contrar} \end{cases}$$

Fie $w \in L(G)$, $w \neq \varepsilon$, $w = x_1 x_2 \cdots x_n$; rezultă că

$$S \Rightarrow x_1 A_1 \Rightarrow x_1 x_2 A_2 \Rightarrow \cdots \Rightarrow x_1 \cdots x_{n-1} A_{n-1} \Rightarrow x_1 \cdots x_n.$$

Considerăm stările $s_1 = S, s_2 = A_1, \dots, s_n = A_{n-1}, s_{n+1} = X$. Avem

$$s_2 = A_1 \in \delta(S, x_1) = \delta(s_1, x_1)$$

.....

$$s_n = A_{n-1} \in \delta(A_{n-2}, x_{n-1}) = \delta(s_{n-1}, x_{n-1})$$

$$s_{n+1} = X \in \delta(A_{n-1}, x_n) = \delta(s_n, x_n).$$

Cum $s_1 \in S$ și $s_{n+1} \in F$, rezultă că $w \in L(A)$.

Fie $w = x_1 \cdots x_n \in L(A)$, $n \geq 1$ (deci $w \neq \varepsilon$). Rezultă că există stările

$$s_1 \in \{S\} = S, s_2 \in \delta(s_1, x_1), \dots, s_{n+1} \in \delta(s_n, x_n) \in F.$$

Există producțiile

*Relația dintre
automate finite
și gramatici
regulate*

$$S = s_1 \rightarrow x_1 s_2 = x_1 A_1$$

.....

$$s_n \in \delta(s_{n-1}, x_{n-1})$$

deci $s_{n-1} \rightarrow x_{n-1} s_n$, adică $A_{n-2} \rightarrow x_{n-1} A_{n-1}$; $s_{n+1} \in F$ deci $s_{n+1} = S$ sau $s_{n+1} = X$.

Dacă $s_{n+1} = S$ atunci $s_n \rightarrow x_n S$; dar S nu apare în membrul drept al producțiilor, deci $s_{n+1} = X$. Din $X \in \delta(s_n, x_n) = \delta(A_{n-1}, x_n)$ rezultă $A_{n-1} \rightarrow x_n$. Am obținut

$$S \Rightarrow x_1 A_1 \Rightarrow x_1 x_2 A_2 \Rightarrow \dots \Rightarrow x_1 \dots x_{n-1} A_{n-1} \Rightarrow x_1 \dots x_n, \text{ deci } w \in L(G).$$

Dacă $w = \varepsilon \in L(G)$ atunci $(S \rightarrow \varepsilon) \in P$, $F = \{S, X\}$, $\{S\} \cap F \neq \emptyset$ deci $\varepsilon \in L(A)$. Invers, dacă $\varepsilon \in L(A)$ atunci $(S \rightarrow \varepsilon) \in P$, $w = \varepsilon \in L(G)$.

Acest automat finit nedeterminist poate fi transformat, conform Teoremei 2.1, într-un automat finit determinist care acceptă același limbaj. $\square$

Teme Curs

Teste Autoevaluare

1. Fie gramatica $G = (\{S\}, \{x, y\}, \{S \rightarrow xy, S \rightarrow xSy\}, S)$. Precizați forma elementelor care aparțin lui $L(G)$ 2 puncte
2. Considerăm gramatica G cu producțiile:
 $S \rightarrow AB$
 $B \rightarrow aB/CB$
 $Aa \rightarrow bC/CaB$
 $CB \rightarrow Ac/bAc$
 $bA \rightarrow ba/AAB$
 - a) De ce tip este gramatica 0.5 puncte
 - b) Folosind algoritmul REC, precizați dacă $w=bac$ aparține sau nu lui $L(G)$ 4 puncte
3. Considerăm automatul $A = (\{q_0, q_1, q_f\}, \{a, b\}, q_0, \{q_f\})$ cu
 $\delta(q_0, a) = \{q_1\}, \delta(q_0, b) = \{q_0\}, \delta(q_1, a) = \{q_f\},$
 $\delta(q_1, b) = \{q_0\}, \delta(q_f, a) = \{q_f\}, \delta(q_f, b) = \{q_f\}.$
Verificați că $w = abaab$ aparține lui $L(A)$1.5 puncte
4. Ce relație există între $q_1 \stackrel{k+1}{\equiv} q_2$ și $q_1 \stackrel{k}{\equiv} q_2$? 0.5 puncte
5. Ce relație există între automatele finite și limbajele regulate?.....0.5 puncte
6. Oficiu..... 1 punct

Răspunsuri

1. $L(G) = \{x^n y^n / n = 1, 2, \dots\}$.

2. a) gramatica este de tipul 1

b) Aplicând *Algoritmul REC*, obținem succesiv:

P1. mergi la P3

P3. $L_0 := \{S\}$, $i := 0$

P4. $L_1 = L_0 \cup \{AB\}$

P5. $i = 1$, mergi la P4

P4. $L_2 = L_1 \cup \{AaB, ACB\}$

P5. $i = 2$, mergi la P4

P4. $L_3 = L_2 \cup \{bCB, AAc\}$

P5. $i = 3$, mergi la P4

P4. $L_4 = L_3 \cup \{bAc\}$

P5. $i = 4$, mergi la P4

P4. $L_5 = L_4 \cup \{bac\}$

P5. $i = 5$, mergi la P4

P4. $L_6 = L_5$

P5. -

P6. **DA**

Stop

Răspuns: $w \in L(G)$.

3. Avem

$$(q_0, abaab) \vdash (q_0, aab) \vdash (q_1, ab) \vdash (q_f, b) \vdash (q_f, \varepsilon)$$

deci $abaab \in L(A)$.

4. $q_1 \stackrel{k+1}{=} q_2$ implică $q_1 \stackrel{k}{=} q_2$

5. Un limbaj este reprezentabil într-un automat finit dacă și numai dacă el este regulat.

Rezumat: s-au definit noțiunile de gramatică, limbaj generat de o gramatică, automat finit, limbaj acceptat de un automat finit, relația dintre automate finite și gramatici, minimizarea automatelor finite.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se vor implementa algoritmi care realizează: verificarea apartenenței unui cuvânt la limbajul generat de o gramatică, determinarea stărilor accesibile în vederea simplificării structurii automatului, funcționarea unui automat finit.

Teme Laborator/Seminar

1. Scrieți un program care să implementeze algoritmul REC.
2. Scrieți un program care să determine stările accesibile ale unui automat finit.
3. Scrieți un program care să simuleze funcționarea unui automat finit.

Rezumat: *Implementarea unor algoritmi care să aprofundeze lucrul cu gramatici și automate finite*

Notății

Descriere Generală

Se studiază proprietăți de derivare în gramaticile independente de context, arbori de derivare (la stânga sau la dreapta) și mai mulți algoritmi de simplificare a acestor gramatici.

Obiective

- cunoașterea unor proprietăți cu privire la derivare
- înțelegerea diferenței dintre derivarea la stânga și cea la dreapta și a semnificației arborelui de derivare
- cunoașterea principalilor algoritmi de simplificare a formei gramaticilor independente de context

Cuprins

- I.4. Generalități referitoare la gramaticile independente de context
- I.5. Simplificări ale gramaticilor independente de context
 - I.5.1 Eliminarea ε -producțiilor
 - I.5.2. Eliminarea redenumirilor
 - I.5.3. Eliminarea simbolurilor inutile

Conținut Curs

I.4. Generalități referitoare la gramaticile independente de context

Gramaticile independente de context sunt folosite în descrierea structurii sintactice a limbajelor de programare, gramaticile regulate neputând să acopere gama tuturor construcțiilor sintactice admise de diverse limbaje de programare. Vom pune în evidență câteva proprietăți importante ale acestor gramatici.

Teorema 4.1. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context.
Dacă

$$\alpha_1 \alpha_2 \cdots \alpha_n \xRightarrow{*} \beta$$

atunci

$$\beta = \beta_1 \beta_2 \cdots \beta_n \text{ și } \alpha_i \xRightarrow{*} \beta_i, i \in \{1, 2, \dots, n\}$$

iar producțiile folosite în cele două derivații sunt aceleași.

Demonstrație. Se procedează prin inducție după lungimea k a derivației. Pentru $k=1$ înseamnă că există $\alpha_i = \alpha' A \alpha''$ și producția $A \rightarrow \alpha$; deci se ia $\beta_i = \alpha' \alpha \alpha''$ și $\beta_j = \alpha_j$ pentru $j \neq i$. Presupunem afirmația adevărată pentru derivații de lungime k și considerăm una de lungime $k+1$:

$$\alpha_1 \alpha_2 \cdots \alpha_n \xRightarrow{k+1} \beta.$$

Ultima derivație se scrie

$$\alpha_1 \alpha_2 \cdots \alpha_n \xRightarrow{k} \gamma \Rightarrow \beta.$$

Conform ipotezei de inducție

$$\gamma = \gamma_1 \cdots \gamma_n \text{ și } \alpha_i \xRightarrow{*} \gamma_i, i \in \{1, 2, \dots, n\};$$

în plus, derivațiile din

$$\alpha_1 \alpha_2 \cdots \alpha_n \xRightarrow{k} \gamma$$

sunt aceleași cu cele din

$$\alpha_i \xRightarrow{*} \gamma_i, i \in \{1, 2, \dots, n\}.$$

Din

$$\gamma_1 \gamma_2 \cdots \gamma_n \Rightarrow \beta,$$

conform cazului $k=1$, rezultă

$$\beta = \beta_1 \beta_2 \cdots \beta_n \text{ și } \gamma_i \xRightarrow{*} \beta_i, i \in \{1, 2, \dots, n\}$$

iar producțiile folosite în aceste derivații sunt aceleași cu cele din $\gamma_1 \gamma_2 \cdots \gamma_n \Rightarrow \beta$.

În final rezultă concluzia din teoremă. $\square$

Definiția 4.1. Un arbore orientat și ordonat este un graf orientat cu proprietățile:

- 1) Există un vârf r , numit rădăcină, care nu are predecesori
- 2) Fiecare vârf diferit de rădăcină are exact un predecesor
- 3) Există un drum de la r la fiecare vârf diferit de rădăcină
- 4) Pe mulțimea succesorilor fiecărui vârf este definită o relație de ordine totală.

Proprietate de
derivare

Arbore
generator (de
derivare)

Definiția 4.2. Un arbore generator în gramatica independentă de context $G = (N, \Sigma, P, S)$ este un arbore T orientat și ordonat, cu proprietățile:

- 1) etichetele nodurilor aparțin mulțimii $N \cup \Sigma \cup \{\varepsilon\}$
- 2) eticheta oricărui nod interior este un neterminat al gramaticii
- 3) dacă n este nod interior cu eticheta A iar descendenții lui, în ordine de la stânga la dreapta, sunt $n_1, n_2, \dots, n_k$ și au etichetele, respectiv, $A_1, A_2, \dots, A_k$ atunci $(A \rightarrow A_1 A_2 \dots A_k) \in P$
- 4) dacă un nod are eticheta ε atunci el este unicul descendent al părintelui său.

Relația dintre
derivare și
arbore

Dacă în definiția anterioară rădăcina este etichetată cu A atunci T se va numi **A -arbore**. Un arbore generator se mai numește arbore de derivare. O importanță deosebită au S -arborii care au nodurile terminale etichetate cu elemente din mulțimea $\Sigma \cup \{\varepsilon\}$.

Teorema 4.2. În gramatica independentă de context $G = (N, \Sigma, P, S)$ există derivarea $A \xRightarrow{*} \alpha$ dacă și numai dacă există un A -arbore care produce pe α .

Demonstrație. Se utilizează inducția după lungimea derivației și respectiv după numărul nodurilor interioare și se ține seama de definiția arborelui generator. $\square$

Se lucrează cu derivații la stânga sau la dreapta. Spunem că α derivează la stânga (dreapta) în β dacă de fiecare dată neterminatul care derivează este cel mai din stânga (dreapta).

Unei derivații i se asociază un arbore generator; totuși, este posibil ca o derivație să aibă mai mulți arbori generatori.

Definiția 4.3. O gramatică independentă de context G este neambiguă dacă orice secvență $w \in L(G)$ admite o singură derivație la stânga (deci un singur arbore generator); în caz contrar gramatica este ambiguă.

Gramaticile independente de context utilizate pentru a defini sintaxa limbajelor

de programare trebuie să fie neambigue.

I.5. Simplificări ale gramaticilor independente de context

Se urmărește modificarea formei producțiilor gramaticii astfel încât să se elimine elementele redundante sau inutile sau să se obțină forme care avantajează algoritmi de analiză sintactică.

I.5.1 Eliminarea ε -producțiilor

O ε -producție este o producție de forma $A \rightarrow \varepsilon$. Prezența ε -producțiilor poate duce la derivații de lungime mare sau poate împiedica folosirea unor algoritmi de analiză sintactică. Dacă limbajul generat de gramatică conține cuvântul vid, va fi păstrată doar ε -producția $S \rightarrow \varepsilon$.

Definiția 5.1. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context.

Simbolul $A \in N$ este anulabil dacă există o derivație de forma $A \xRightarrow{*} \varepsilon$.

Simbolurile anulabile se determină cu

Algoritmul ANL

Intrare: gramatica $G = (N, \Sigma, P, S)$ independentă de context

Ieșire: mulțimea N^{anl} a simbolurilor anulabile

Metoda:

P1. $M_0 = \{A \in N \mid \exists (A \rightarrow \varepsilon) \in P\}$, $i := 0$

P2. $M_{i+1} = M_i \cup \{A \in N \mid \exists \alpha \in M_i^* \text{ astfel încât } (A \rightarrow \alpha) \in P\}$

P3. Dacă $M_{i+1} \neq M_i$ atunci $i := i + 1$ și mergi la P2.

P4. $N^{anl} = M_i$, STOP.

Teorema 5.1. Pentru orice gramatică independentă de context $G = (N, \Sigma, P, S)$ există o gramatică independentă de context G' astfel încât $L(G') = L(G) - \{\varepsilon\}$.

Demonstrație. Folosind algoritmul ANL se determină mulțimea N^{anl} a simbolurilor anulabile. Fie

$$(A \rightarrow \alpha) \in P, \alpha \neq \varepsilon, \alpha = \alpha_1 A_1 \alpha_2 \cdots \alpha_k A_k \alpha_{k+1}.$$

Se înlocuiește fiecare producție $A \rightarrow \alpha_1 A_1 \alpha_2 \cdots \alpha_k A_k \alpha_{k+1}$ cu producții de forma

$$A \rightarrow \alpha_1 X_1 \alpha_2 \cdots \alpha_k X_k \alpha_{k+1}$$

unde

$$X_i \in \{A_i, \varepsilon\} \text{ dacă } A_i \in N^{anl}$$

și

$$X_i = A_i \text{ în caz contrar.}$$

Apoi, se elimină din P toate ε -producțiile. Notând cu P' mulțimea astfel obținută, gramatica G' este $G' = (N^{anl}, \Sigma, P', S)$. $\square$

Ca o consecință, rezultă următoarea teoremă, care a mai fost discutată anterior.

Teorema 5.2. *Pentru orice gramatică independentă de context G există o gramatică independentă de context G'' echivalentă cu G și care nu conține ε -producții, exceptând eventual pe $S' \rightarrow \varepsilon$ (S' fiind simbolul inițial al gramaticii G''), caz în care S' nu apare în membrul drept al niciunei producții din G'' .*

Demonstrație. Fie G' gramatica construită în teorema anterioară. Dacă $\varepsilon \notin L(G)$ atunci se ia $G'' = G'$. Dacă $\varepsilon \in L(G)$ atunci se ia $G'' = (N^{anl} \cup \{S'\}, \Sigma, S', P' \cup \{S' \rightarrow S / \varepsilon\})$. $\square$

1.5.2. Eliminarea redenumirilor

O redenumire (sau producție singulară) a unei gramatici independente de context este o producție de forma $A \rightarrow B$.

Teorema 5.3. *Pentru orice gramatică independentă de context fără ε -producții există o altă gramatică de același tip și fără redenumiri, echivalentă cu ea.*

Demonstrație. Fie $G = (N, \Sigma, P, S)$ gramatica inițială, astfel încât S nu apare în membrul doi al nici unei producții. Pentru fiecare $A \in N$ definim mulțimea $V(A) = \left\{ B \in N / A \xRightarrow{+}_G B \right\}$. Deoarece G nu are ε -producții, toți termenii derivării $A \Rightarrow B_1 \Rightarrow \cdots \Rightarrow B_k = B$ au lungimea egală cu 1. Presupunând că simbolurile $B_1, \dots, B_k$ sunt distincte, rezultă că derivația $A \xRightarrow{*} B$ este de

*Eliminarea
simbolurilor
anulabile*

lungime mai mică decât $|N|$; deci, mulțimile $V(A)$ se pot calcula printr-un proces iterativ în cel mult $|N|-1$ pași. Construim mulțimea de producții P' în felul următor:

- 1) eliminăm din P toate redenumirile
- 2) fie $(B \rightarrow \alpha) \in P$ o producție nesingulară; adăugăm la P' toate producțiile de forma $A \rightarrow \alpha$ pentru $B \in V(A)$.

În final luăm $G' = (N, \Sigma, P', S)$, care este independentă de context și fără redenumiri. Este evident că $L(G') \subseteq L(G)$, iar incluziunea inversă se demonstrează ușor. $\square$

Acum se poate demonstra echivalența gramaticilor regulate cu cele în formă redusă.

Teorema 5.4. Pentru orice gramatică de tipul 3, $G = (N, \Sigma, P, S)$, există o gramatică de același tip $G_1 = (N_1, \Sigma_1, P_1, S_1)$ echivalentă cu G și având proprietatea că orice producție $u \rightarrow v$ satisface condiția $v \in \Sigma_1 \cup \Sigma_1 N_1$; o astfel de gramatică se numește în formă redusă.

Demonstrație. Conform teoremei anterioare există o gramatică $G' = (N, \Sigma, P', S)$ de același tip cu G , echivalentă cu ea și fără redenumiri. Producțiile gramaticii G' fiind de forma $u \rightarrow v$ cu $v \in \Sigma^* \cup \Sigma^* N$, pentru fiecare producție de forma

$$X \rightarrow a_1 \dots a_n Y, \text{ cu } a_1, \dots, a_n \in \Sigma, X, Y \in N \text{ și } n \geq 2,$$

introducem variabilele $A_1, \dots, A_{n-1}$, distincte și verificând condiția $A_i \notin N \cup \Sigma$ pentru orice $i \in \{1, 2, \dots, n-1\}$. Adăugăm mulțimii N aceste variabile, iar în P' în locul producției

$$X \rightarrow a_1 \dots a_n Y$$

introducem producțiile

$$X \rightarrow a_1 A_1, A_1 \rightarrow a_2 A_2, \dots, A_{n-1} \rightarrow a_n Y.$$

Procedând astfel pentru toate producțiile și apoi și pentru cele de forma

$$X \rightarrow a_1 \dots a_n \text{ cu } a_1, \dots, a_n \in \Sigma, X \in N \text{ și } n \geq 2,$$

în acest caz având $A_{n-1} \rightarrow a_n$ în loc de $A_{n-1} \rightarrow a_n Y$, obținem o mulțime de neterminale N_1 și o mulțime de producții P_1 . Gramatica $G_1 = (N_1, \Sigma_1, P_1, S_1 = S)$ este de același tip cu G' , deci și cu G . $\square$

Algoritmul de
eliminare a
redenumirilor

Echivalarea
gramaticilor
regulate cu
cele în formă
redușă

1.5.3. Eliminarea simbolurilor inutile

Definiția 5.2. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context. Simbolul $X \in N \cup \Sigma$ se numește utilizabil dacă există o derivație de forma $S \Rightarrow^* \alpha X \beta \Rightarrow^* w \in \Sigma^*$; altfel X este inutilizabil.

Calitatea de simbol utilizabil poate fi verificată în două etape, conform definițiilor următoare.

Definiția 5.3. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context. Simbolul $A \in N$ se numește productiv dacă există o derivație de forma $A \Rightarrow^* w \in \Sigma^*$; altfel simbolul se numește neproductiv.

Definiția 5.4. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context. Simbolul $X \in N \cup \Sigma$ se numește accesibil dacă există o derivație de forma $S \Rightarrow^* \alpha X \beta$; altfel simbolul se numește inaccesibil.

Teorema 5.5. Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context. Există o gramatică G' independentă de context și echivalentă cu G , care are toate simbolurile neterminale (exceptând eventual S) productive.

Demonstrație. Simbolurile productive se determină cu

Algoritmul PROD

Intrare: gramatica $G = (N, \Sigma, P, S)$ independentă de context

Ieșire: mulțimea N^{prod} a simbolurilor productive

Metoda:

P1. $M_0 = \{A \in N / \exists x \in \Sigma^*, \text{ astfel încât } \exists (A \rightarrow x) \in P\}$, $i := 0$

P2. $M_{i+1} = M_i \cup \{A \in N / \exists \alpha \in (M_i \cup \Sigma)^* \text{ astfel încât } (A \rightarrow \alpha) \in P\}$

P3. Dacă $M_{i+1} \neq M_i$ atunci $i := i + 1$ și mergi la P2.

P4. $N^{prod} = M_i$, STOP.

Fie $G' = (N^{prod} \cup S, \Sigma, P', S)$, unde mulțimea P' conține producțiile din P formate numai cu simboluri din $N^{prod} \cup \{S\} \cup \Sigma$.

Algoritm de
determinare a
simbolurilor
productive

Din $P' \subseteq P$ rezultă $L(G') \subseteq L(G)$. Apoi, deoarece orice derivație $S \xRightarrow[G]{*} w \in \Sigma^*$ folosește numai producții din P' , rezultă $L(G) \subseteq L(G')$ și deci $L(G) = L(G')$. $\square$

Consecința 5.1. *Dacă $G = (N, \Sigma, P, S)$ este o gramatică independentă de context atunci $L(G) \neq \emptyset$ dacă și numai dacă S este simbol productiv.*

Deoarece există un algoritm pentru determinarea simbolurilor productive, înseamnă că problema mulțimii vide pentru limbaje independente de context este rezolvabilă algoritmic.

Teorema 5.6. *Pentru orice gramatică independentă de context G există o gramatică independentă de context G'' , cu toate simbolurile accesibile și echivalentă cu G .*

Demonstrație. Mulțimea simbolurilor accesibile se determină cu

Algoritmul ACS

Intrare: gramatica $G = (N, \Sigma, P, S)$ independentă de context

Ieșire: mulțimea N^{acs} a simbolurilor accesibile

Metoda:

P1. $M_0 = \{S\}$, $i := 0$

P2. $M_{i+1} = M_i \cup \{X \in N \cup \Sigma / \exists A \in M_i \cap N \text{ astfel încât } \exists (A \rightarrow \alpha X \beta) \in P\}$

P3. Dacă $M_{i+1} \neq M_i$ atunci $i := i + 1$ și mergi la P2.

P4. $N^{acs} = M_i$, STOP.

Gramatica G'' este $G'' = (N \cap N^{acs}, \Sigma \cap N^{acs}, P'', S)$, unde P'' conține toate producțiile din P formate numai cu simboluri accesibile. $\square$

*Algoritm de
determinare a
simbolurilor
accesibile*

Teorema 5.7. *Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context cu $L(G) \neq \emptyset$. Atunci există o gramatică G_1 independentă de context și fără simboluri inutile echivalentă cu G .*

Demonstrație. Fie $G' = (N', \Sigma, P', S)$ gramatica echivalentă cu G , care nu conține simboluri neproductive și $G'' = (N'', \Sigma'', P'', S)$ gramatica echivalentă cu G' și care are toate simbolurile accesibile. Rezultă că $L(G) = L(G') = L(G'')$ și rămîne de arătat că G'' are toate simbolurile utilizabile. Deoarece orice $A \in N''$

este accesibil pentru G'' , există derivația $S \xRightarrow[G'']{*} \alpha A \beta$. Cum toate simbolurile din secvența $\alpha A \beta$ sunt productive pentru gramatica G' , înseamnă că există derivația $\alpha A \beta \xRightarrow[G']{*} w \in \Sigma^*$. Toate simbolurile din ultimele două derivații sunt accesibile pentru G'' și $S \xRightarrow[G'']{*} \alpha A \beta \xRightarrow[G'']{*} w \in \Sigma^*$; deci simbolul A este utilizabil în G'' . $\square$

Teme Curs

Teste Autoevaluare

1. Fie gramatica cu producțiile

$$S \rightarrow AB$$

$$A \rightarrow aAb / \varepsilon$$

$$B \rightarrow Bc / \varepsilon$$

Construiți un S-arbore de derivare asociat..... 1 punct

2. Considerăm gramatica cu producțiile

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / a$$

și secvența $w = a * (a + a)$.

Construiți o derivare la stânga pentru w 1 punct

3. Fie gramatica $G = (N, \Sigma, P, S)$ cu producțiile $S \rightarrow SaSb / SbSa / \varepsilon$.

Precizați dacă gramatica este sau nu ambiguă. Justificați răspunsul.

.....1 punct

4. Fie gramatica cu producțiile

$$S \rightarrow aSb \mid aA \mid b, A \rightarrow bA, B \rightarrow bS \mid b \mid c$$

Care este mulțimea simbolurilor productive? 2 puncte

5. Care sunt simbolurile accesibile ale gramaticii anterioare?..... 2 puncte

6. Fie gramatica cu producțiile

$$S \rightarrow aAB \mid AC$$

$$A \rightarrow bA \mid CC$$

$$B \rightarrow SB \mid b$$

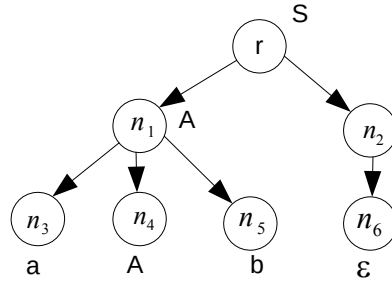
$$C \rightarrow aCb \mid \varepsilon$$

Determinați mulțimea simbolurilor anulabile.....2 puncte

7. Oficiu.....1 punct

Răspunsuri

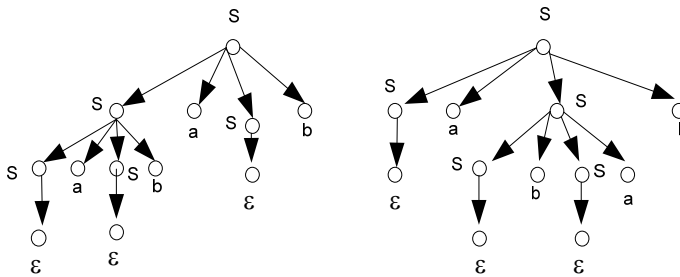
1. Un exemplu de S-arbore este cel din figura următoare



2. O derivație la stânga a lui w este

$$\begin{aligned} E &\Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow a * F \Rightarrow a * (E) \Rightarrow a * (E + T) \Rightarrow a * (T + T) \\ &\Rightarrow a * (F + T) \Rightarrow a * (a + T) \Rightarrow a * (a + F) \Rightarrow a * (a + a) \end{aligned}$$

3. Gramatica este ambiguă. Secvența $w \in abab \in L(G)$ admite doi arbori generator diferiți



4. $M_0 = \{S, B\}$, $M_1 = M_0$, $N^{Prod} = \{S, B\}$
 5. $M_0 = \{S\}$, $M_1 = \{S, a, b\}$, $M_2 = M_1$, $N^{acs} = \{S, a, b\}$
 6. $M_0 = \{C\}$, $M_1 = \{C, A\}$, $M_2 = \{C, A, S\}$, $M_3 = M_2$.

Prin urmare $N^{anl} = \{S, A, C\}$

Rezumat: S-a descris relația dintre derivare și arborele generator asociat și s-au studiat algoritmi de simplificare a gramaticilor independente de context

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se urmărește echivalarea unei gramatici independente de context cu una care a fost simplificată.

Teme Laborator/Seminar

1. Scrieți un program care să echivaleze o gramatică independentă de context cu una fără simboluri productive.
2. Scrieți un program care să echivaleze o gramatică independentă de context cu una care are toate simbolurile accesibile.
3. Scrieți un program care să echivaleze o gramatică independentă de context cu una fără producții vide.
4. Scrieți un program care să echivaleze o gramatică independentă de context cu una care are toate simbolurile utilizabile.

Rezumat: *Aplicațiile urmăresc trecerea de la o gramatică independentă de context la una în formă mai simplă; se poate simplifica în cascadă, la fiecare pas simplificând gramatica de la pasul anterior.*

Notății

Curs 3 *Automate pushdown și gramatici independente de context*

Curs 3
Durata:
2 ore

Descriere Generală

Se definesc noțiunile de automat pushdown, configurație, limbaj acceptat.

Se demonstrează echivalența dintre limbajul acceptat cu memoria vidă și cel acceptat după criteriul stării finale.

Se demonstrează echivalența limbajului acceptat de un automat pushdown cu cel generat de o gramatică independentă de context

Obiective

- cunoașterea structurii și a modului de funcționare a unui automat pushdown
- stabilirea relației dintre automate pushdown și gramatici independente de context

Cuprins

I.6. Funcționarea automatelor și relația cu limbajele independente de context

Conținut Curs

I.6. Funcționarea automatelor și relația cu limbajele independente de context

Automatele pushdown sunt mecanisme de recunoaștere a limbajelor independente de context. Numele este dat de organizarea memoriei auxiliare sub formă de stivă.

Definiția 6.1. Se numește automat pushdown un sistem $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ unde:

- Q este mulțimea stărilor automatului
- Σ este alfabetul de intrare
- Γ este alfabetul intern (pushdown)
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P_f(Q \times \Gamma^*)$ este funcția de tranziție, unde P_f este

mulțimea părților finite.

- $q_0 \in Q$ este starea inițială
- $Z_0 \in \Gamma$ este simbolul intern inițial
- $F \subset Q$ este mulțimea stărilor finale.

Definiția 6.2. Funcționarea automatului este dictată de funcția de tranziție și se definește cu ajutorul configurației (q, w, γ) , $q \in Q, w \in \Sigma^*, \gamma \in \Gamma^*$. Configurația inițială este (q_0, w, Z_0) .

Trecerea de la o configurație la alta se face prin pași: $(q, aw, Z\gamma) \vdash (q_1, w, \alpha\gamma)$ dacă $(q_1, \alpha) \in \delta(q, a, Z)$. Notăm cu $\vdash^+$, $\vdash^*$, $\vdash^i$ închiderea tranzitivă, închiderea reflexivă și tranzitivă și respectiv trecerea în i pași.

Definiția 6.3. Limbajul acceptat de automatul pushdown P după criteriul benzii vide este: $L_\varepsilon(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \varepsilon)\}$.

Limbaj
acceptat

Limbajul acceptat de automatul pushdown P după criteriul strării finale este: $L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \varepsilon, \alpha), p \in F\}$.

Teorema 6.1. Fie L un limbaj independent de context. Atunci există un automat pushdown M astfel încât $L = L_\varepsilon(M)$.

Demonstrație. Fie $G = (N, \Sigma, P, S)$ astfel încât $L = L(G)$. Construim $M = (q, \Sigma, N \cup \Sigma, \delta, q, S, \emptyset)$ unde:

$$t1) \delta(q, \varepsilon, A) = \{(q, \alpha) \mid (A \rightarrow \alpha) \in P\}$$

$$t2) \delta(q, a, a) = \{(q, \varepsilon)\} \quad \forall a \in \Sigma.$$

$$t3) \delta(q, b, Z) = \emptyset \text{ în toate celelalte cazuri.}$$

Tranzițiile t1) produc pe banda pushdown derivații la stînga din gramatica G iar tranzițiile t2) permit trecerea la simbolul următor, pe benzile de intrare și pushdown, în cazul coincidenței dintre simbolurile citite de pe cele două benzi.

Fie $A \in N$ și $w \in \Sigma^*$; trebuie stabilită echivalența

$$(1) \quad A \Rightarrow^* w \Leftrightarrow (q, w, A) \vdash^* (q, \varepsilon, \varepsilon)$$

Demonstrația de la stînga la dreapta se face prin inducție după lungimea k a

Construirea
unui automat
pushdown cu
memoria vidă
care acceptă
un limbaj
independent de
context

derivației. Pentru $k = 1$, $A \Rightarrow w$ înseamnă $(A \rightarrow w) \in P$ și $w = a_1 a_2 \cdots a_p \in \Sigma^*$.

Folosind t1) rezultă

$$(q, w) \in \delta(q, \varepsilon, A),$$

deci

$$(q, w, A) \vdash (q, w, w) = (q, a_1 \cdots a_p, a_1 \cdots a_p).$$

Folosind p tranziții de tipul t2 rezultă

$$(q, a_1 \cdots a_p, a_1 \cdots a_p) \vdash^* (q, \varepsilon, \varepsilon),$$

deci

$$(q, w, A) \vdash^* (q, \varepsilon, \varepsilon).$$

Presupunem implicația “ $\Rightarrow$ ” adevărată pentru derivații de lungime mai mică

decât k și fie $A \Rightarrow^k w$. Înseamnă că

$$A \Rightarrow \alpha = \alpha_1 A_1 \alpha_2 \cdots \alpha_r A_r \alpha_{r+1} \Rightarrow^k w, \text{ unde } \alpha_i \in \Sigma^*, A_i \in N$$

Rezultă descompunerea

$$w = \alpha_1 w_1 \alpha_2 \cdots \alpha_r w_r \alpha_{r+1} \text{ cu } A_i \Rightarrow^{k_i} w_i, \quad i \in \{1, 2, \dots, r\} \text{ și } k_1 + k_2 + \cdots + k_r = k - 1.$$

Conform ipotezei de inducție,

$$(2) \quad (q, w_i, A_i) \vdash^* (q, \varepsilon, \varepsilon), \quad i \in \{1, 2, \dots, r\}$$

Folosind t1), t2) și (2) rezultă

$$\begin{aligned} (q, w, A) \vdash (q, w, \alpha) &= (q, \alpha_1 w_1 \alpha_2 \cdots w_r \alpha_{r+1}, \alpha_1 A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \\ &\vdash^* (q, w_1 \alpha_2 \cdots w_r \alpha_{r+1}, A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \\ &\vdash^* (q, \alpha_2 \cdots w_r \alpha_{r+1}, \alpha_2 \cdots A_r \alpha_{r+1}) \\ &\vdash^* (q, \alpha_{r+1}, \alpha_{r+1}) \vdash^* (q, \varepsilon, \varepsilon) \end{aligned}$$

Implicația inversă se demonstrează prin inducție după numărul de pași din

$(q, w, A) \vdash^* (q, \varepsilon, \varepsilon)$. Dacă

$$(q, w, A) \vdash (q, \varepsilon, \varepsilon),$$

se aplică t1; deci

$$(A \rightarrow \varepsilon) \in P, \quad w = \varepsilon \text{ și } A \Rightarrow w.$$

Presupunem implicația adevărată pentru un număr de pași mai mic decât k și fie

$$(3) \quad (q, w, A) \vdash^* (q, \varepsilon, \varepsilon) \text{ în } k \text{ pași}$$

La primul pas din (3) se folosește t1. Această trecere se descompune în

$$(q, w, A) \vdash (q, w, \alpha) = (q, w, \alpha_1 A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \vdash^* (q, \varepsilon, \varepsilon)$$

unde ultima trecere are loc în $k - 1$ pași și

$$(A \rightarrow \alpha = \alpha_1 A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \in P.$$

Rezultă $w = \alpha_1 z_1$ și

$$(4) \quad (q, \alpha_1 z_1, \alpha_1 A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \xrightarrow{\star} (q, z_1, A_1 \alpha_2 \cdots A_r \alpha_{r+1}) \xrightarrow{\star} (q, \varepsilon, \varepsilon).$$

Fie w_1 prefixul lui z_1 care se consumă de pe banda de intrare până când conținutul benzii pushdown devine pentru prima dată mai mic decât $|A_1 \alpha_2 \cdots A_r \alpha_{r+1}|$; deci

$$(5) \quad z_1 = w_1 y_1, (q, w_1, A_1) \xrightarrow{\star} (q, \varepsilon, \varepsilon) \text{ în } k_1 \leq k-1 \text{ pași și } A_1 \xRightarrow{*} w_1$$

Din (4) și (5) rezultă

$$(q, y_1, \alpha_2 \cdots A_r \alpha_{r+1}) \xrightarrow{\star} (q, \varepsilon, \varepsilon).$$

Repetând raționamentul se obține

$w = \alpha_1 w_1 \alpha_2 \cdots \alpha_r w_r \alpha_{r+1}$, $(q, w_i, A_i) \xrightarrow{\star} (q, \varepsilon, \varepsilon)$ în $k_i \leq k-1$ pași și conform ipotezei de inducție $A_i \xRightarrow{*} w_i$.

În final,

$$A \Rightarrow \alpha = \alpha_1 A_1 \cdots \alpha_r A_r \alpha_{r+1} \xRightarrow{*} \alpha_1 w_1 \cdots \alpha_r w_r \alpha_{r+1} = w.$$

Luând $A = S$ în (1), rezultă $w \in L(G) \Leftrightarrow L_\varepsilon(M)$. $\square$

Teorema 6.2. Pentru orice automat pushdown M , $L_\varepsilon(M)$ este limbaj independent de context.

Demonstrație. Fie $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$. Construim $G = (N, \Sigma, P, S)$ astfel

$$N = \{[p, X, q] \mid p, q \in Q, x \in \Gamma\} \cup \{S\},$$

$S \notin (N \cup \Sigma)$ fiind simbol nou iar producțiile sunt de forma

1. $S \rightarrow [q_0, Z_0, q]$ pentru orice $q \in Q$
2. Dacă $(q, X_1 \cdots X_r) \in \delta(p, a, Z)$ și $r > 0$ atunci
 $[p, Z, p_r] \rightarrow a[q, X_1, p_1][p_1, X_2, p_2] \cdots [p_{r-1}, X_r, p_r]$
 pentru orice $p_1, \dots, p_r \in Q$
3. Dacă $(q, \varepsilon) \in \delta(p, a, Z)$ atunci $[p, Z, q] \rightarrow a$.

Evident, G este independentă de context. Pentru a stabili egalitatea $L(G) = L_\varepsilon(M)$ se arată că are loc echivalența

$$(1) \quad [p, X, q] \xRightarrow{*} w \Leftrightarrow (p, w, X) \xrightarrow{\star} (q, \varepsilon, \varepsilon), \quad \forall p, q \in Q, X \in \Gamma \text{ și } w \in \Sigma^*$$

Implicația " $\Rightarrow$ " se demonstrează prin inducție după lungimea derivației iar cea inversă, prin inducție după numărul de pași.

*Limbajul
acceptat cu
memoria
pushdown vidă
este
independent de
context*

Fie $w \in L_\varepsilon(M)$; rezultă

$$(q_0, w, Z_0) \xrightarrow{*} (q, \varepsilon, \varepsilon) \text{ cu } q \in Q.$$

Din (1) rezultă $[q_0, Z_0, q] \xRightarrow{*} w$. Conform producțiilor de tipul 1, avem $S \Rightarrow [q_0, Z_0, q] \xRightarrow{*} w$, adică $w \in L(G)$. Fie acum $w \in L(G)$, adică $S \xRightarrow{*} w$. Primul pas în această derivare este de forma $S \Rightarrow [q_0, Z_0, q] \xRightarrow{*} w$. Ținând seama de (1), rezultă $(q_0, w, Z_0) \xrightarrow{*} (q, \varepsilon, \varepsilon)$, adică $w \in L_\varepsilon(M)$. $\square$

Am lucrat cu automate care se opresc după criteriul benzii pushdown vide. Acestea sunt, însă, echivalente, din punct de vedere al limbajului acceptat, cu automate care se opresc după criteriul stării finale. Teoremele următoare demonstrează această echivalență.

Teorema 6.3. *Pentru orice automat pushdown P există un automat pushdown P' astfel încât $L_\varepsilon(P) = L(P')$.*

Demonstrație. Fie $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$; am luat $F = \emptyset$, deoarece stările finale nu intervin în funcționare. Definim $P' = (Q \cup \{q'_0, q_f\}, \Sigma, \Gamma \cup \{X\}, \delta', q'_0, X, \{q_f\})$

unde q'_0 și q_f sunt stări noi iar X este simbol nou. Funcția de tranziție este definită prin

- t1) $\delta'(q'_0, \varepsilon, X) = \{(q_0, Z_0 X)\}$
- t2) $\delta'(q, a, Z) = \delta(q, a, Z) \quad \forall q \in Q, \quad \forall a \in \Sigma \cup \{\varepsilon\}, \quad \forall Z \in \Gamma$
- t3) $\delta'(q, \varepsilon, X) = \{(q_f, \varepsilon)\} \quad \forall q \in Q$
- t4) δ' ia ca valoare $\emptyset$ în celelalte cazuri.

Tranziția t1) aduce automatul P' în configurația inițială a automatului P , având în plus simbolul X la baza stivei. Tranzițiile t2) permit automatului P' să simuleze automatul P . Dacă, în urma simulării, automatul P' rămâne doar cu simbolul X pe banda pushdown (ceea ce corespunde golirii benzii pushdown a lui P) atunci P' trece, prin tranziția t3), în starea finală q_f . Dacă, în plus, a fost citită toată banda de intrare, atunci P' este într-o configurație de acceptare. $\square$

Teorema 6.4. *Pentru orice automat pushdown P există un automat pushdown P' astfel încât $L(P) = L_\varepsilon(P')$.*

Teoreme 6.3 și 6.4 demonstrează

Demonstrație. Fie $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. Construim $P' = (Q \cup \{q'_0, q_\varepsilon\}, \Sigma, \Gamma \cup \{X\}, \delta', q'_0, X, \emptyset)$, unde q_ε și q'_0 două stări distincte, care nu aparțin lui Q , iar $X \notin \Gamma$. Funcția de tranziție δ' este definită astfel

t1) $\delta'(q'_0, \varepsilon, X) = \{(q_0, Z_0 X)\}$
t2) $\delta'(q, a, Z) \supseteq \delta(q, a, Z)$ pentru $\forall q \in Q, \forall a \in \Sigma \cup \{\varepsilon\}$ și $\forall Z \in \Gamma$
t3) $\delta'(q, \varepsilon, Z) \supseteq \{(q_\varepsilon, \varepsilon)\}$ pentru $\forall q \in F$ și $\forall Z \in \Gamma$
t4) $\delta'(q_\varepsilon, \varepsilon, Z) = \{(q_\varepsilon, \varepsilon)\}$ pentru $\forall Z \in \Gamma \cup \{X\}$
t5) δ' ia valoarea $\emptyset$ în rest.

echivalența
dintre limbajul
acceptat după
criteriul stării
finale și cel
acceptat cu
memoria vidă

Tranziția t1) aduce automatul P' în configurația inițială a lui P , cu deosebire că la baza benzii pushdown se află simbolul X . Rolul lui X este de a evita golirea simultană a benzilor de intrare și pushdown fără ca automatul să fie în stare finală. Tranzițiile t2) permit automatului P' să simuleze funcționarea lui P . Dacă P' ajunge într-o stare finală atunci el poate efectua fie tranziții de tipul t2) fie poate intra în faza de golire a benzii pushdown prin tranziții de tipul t3). Tranzițiile t4) permit golirea benzii pushdown. $\square$

Consecința 6.1. Pentru orice limbaj L , următoarele afirmații sunt echivalente

- 1) L este independent de context
- 2) L este acceptat de un automat pushdown după criteriul benzii pushdown vide
- 3) L este acceptat de un automat pushdown după criteriul stării finale.

Sinteza
teoremelor din
curs

Definiția 6.4. Automatul pushdown $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ este determinist dacă

- 1) $|\delta(q, a, Z)| \leq 1, \forall q \in Q, \forall a \in \Sigma \cup \{\varepsilon\}, \forall Z \in \Gamma$
- 2) $\forall q \in Q, \forall Z \in \Gamma, \delta(q, \varepsilon, Z) \neq \emptyset \Rightarrow \forall a \in \Sigma, \delta(q, a, Z) \neq \emptyset$.

Automat
pushdown
determinist

Între automatele pushdown nedeterministe și deterministe nu există o relație similară cu cea din cazul automatelor finite. Justificarea acestei afirmații este următoarea: se verifică ușor că limbajele $L_1 = \{0^n 1^n / n \geq 0\}$ și $L_2 = \{0^n 1^{2n} / n \geq 1\}$ sunt independente de contexte, deci și $L = L_1 \cup L_2$ este independent de context. Înseamnă că există un automat pushdown care acceptă pe L ; totuși, s-a demonstrat că, nu există automate pushdown deterministe care să accepte pe L .

Teme Curs

Teste Autoevaluare

1. Care sunt componentele unui automat pushdown?..... 2 puncte
2. Definiți noțiunea de configurație..... 1 punct
3. Care sunt criteriile după care este acceptat un limbaj?.....2 puncte
4. Ce legatură există între cele două moduri de acceptare..... 1 punct
5. Definiți noțiunea de automat pushdown determinist..... 2 puncte
6. Definiți relația dintre automate pushdown și gramatici independente de context.....1 punct
7. Oficiu..... 1 punct

Răspunsuri

1. - Q este mulțimea stărilor automatului

- Σ este alfabetul de intrare

- Γ este alfabetul intern (pushdown)

- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P_f(Q \times \Gamma^*)$ este funcția de tranziție, unde P_f este mulțimea părților finite.

- $q_0 \in Q$ este starea inițială

- $Z_0 \in \Gamma$ este simbolul intern inițial

- $F \subset Q$ este mulțimea stărilor finale.

2. (q, w, γ) , $q \in Q$, $w \in \Sigma^*$, $\gamma \in \Gamma^*$

3. criteriul benzii vide este:

$$L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \xrightarrow{*} (p, \varepsilon, \varepsilon)\}.$$

criteriul stării finale este:

$$L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \xrightarrow{*} (p, \varepsilon, \alpha), p \in F\}.$$

4. Un limbaj este acceptat de un automat pushdown după criteriul stării finale dacă și numai dacă este acceptat după criteriul benzii vide

5. Automatul pushdown $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ este determinist dacă

$$1) |\delta(q, a, Z)| \leq 1, \quad \forall q \in Q, \forall a \in \Sigma \cup \{\varepsilon\}, \forall Z \in \Gamma$$

$$2) \forall q \in Q, \forall Z \in \Gamma, \quad \delta(q, \varepsilon, Z) \neq \emptyset \Rightarrow \forall a \in \Sigma, \delta(q, a, Z) \neq \emptyset.$$

6. Pentru orice limbaj L , următoarele afirmații sunt echivalente

1) L este independent de context

2) L este acceptat de un automat pushdown după criteriul benzii vide

3) L este acceptat de un automat pushdown după criteriul stării finale

Rezumat: S-a definit noțiunea de automat pushdown, s-a demonstrat echivalența dintre cele două moduri de acceptare, și dintre automate pushdown și limbaje independente de context.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se va însuși, pe baza unui program, modul de funcționare al unui automat pushdown în vederea acceptării unui cuvânt.

Teme Laborator/Seminar

1. Să se scrie un program care să simuleze funcționarea unui automat pushdown determinist; analizați cele două criterii.

Rezumat: <i>Funcționarea automatelor pushdown</i>
--

Notății

Curs 4 Clase speciale de gramatici independente de context **(I)**

Curs 4
Durata:
2 ore

Descriere Generală

Se echivalează o gramatică independentă de context cu alta:

- 1) în forma normală Chomsky
- 2) fără recursie la stânga
- 3) factorizată la stânga

Se stabilește condiția necesară și suficientă ca o gramatică independentă de context să fie LL

Obiective

- cunoașterea metodei de trecere de la o gramatică independentă de context arbitrară la una în forma normală Chomsky
- cunoașterea algoritmului de înlăturare a recursivității la stânga și a celui de factorizare la stânga
- familiarizarea cu noțiunea de gramatică LL(k) și însușirea metodei de verificare a condiției LL

Cuprins

- I.7.1 Gramatici în forma normală Chomsky
- I.7.2. Gramatici nerecursive la stânga
- I.7.3. Gramatici factorizate la stânga
- I.7.4. Gramatici LL(k)

Conținut Curs

I.7.1 Gramatici în forma normală Chomsky

Una din simplificările de bază aduse producțiilor unei gramatici independente de context este enunțată într-o teoremă dată de Chomsky. Orice gramatică ale cărei producții satisfac această teoremă se numește în formă normală Chomsky.

Teorema 7.1. Orice limbaj independent de context care nu conține cuvântul vid poate fi generat de o gramatică în care toate producțiile sunt de forma $X \rightarrow YZ$ sau $X \rightarrow a$, unde X, Y, Z sunt neterminale iar a este terminal.

Demonstrație. Conform teoremei 1.14 este suficient să lucrăm cu gramatici independente de context fără redenumiri. Fie $G = (N, \Sigma, P, S)$ o astfel de gramatică și $X \rightarrow Y_1 \dots Y_m$ o producție a lui G . Dacă $m = 1$, atunci Y_1 este terminal și această producție satisface forma normală Chomsky. Dacă $m \geq 2$, atunci fiecare terminal Y_i , $1 \leq i \leq m$, din producție este înlocuit cu un nou simbol Z_i diferit de toate simbolurile din $N \cup \Sigma$ și de simbolurile introduse anterior. Reținem și producția $Z_i \rightarrow Y_i$, care satisface forma normală Chomsky. Transformând astfel toate regulile obținem mulțimea de producții P_1 și o nouă gramatică $G_1 = (N \cup \{Z_i\}, \Sigma, P_1, S)$ echivalentă cu G . Apoi, pentru fiecare producție de forma $X \rightarrow A_1 \dots A_n$, cu $n > 2$, din gramatica G_1 , se introduc $n - 2$ simboluri $B_1, \dots, B_{n-2}$ și producțiile

$$(1) \quad X \rightarrow A_1 B_1, \quad B_1 \rightarrow A_2 B_2, \quad \dots, \quad B_{n-2} \rightarrow A_{n-1} A_n.$$

Procedăm așa cu fiecare producție care are cel puțin trei simboluri în membrul al doilea și adăugăm la mulțimea neterminalelor simbolurile noi. Înlocuim, apoi, producția $X \rightarrow A_1 \dots A_n$, cu $n > 2$, cu producțiile (1). Obținem o nouă gramatică $G' = (N', \Sigma, P', S)$ independentă de context, aflată în formă normală Chomsky; G' este echivalentă cu G_1 și deci și cu G . $\square$

*Echivalarea
unei gramatici
independente
de context cu
una în formă
Chomsky*

I.7.2. Gramatici nerecursive la stânga

Definiția 7.1. Fie GIC $G = (N, \Sigma, P, S)$. Neterminalul X se numește recursiv la stânga dacă $X \xrightarrow{+} X\alpha$ pentru un anumit $\alpha \in (N \cup \Sigma)^*$. Dacă G conține cel puțin un neterminal recursiv la stânga, gramatica se numește cu recursie stângă.

Recursivitatea directă, ce apare pentru un neterminal A sub forma producțiilor $A \rightarrow \beta_1 / \beta_2 / \dots / \beta_n / A\alpha_1 / \dots / A\alpha_m$, unde $\beta_1, \beta_2, \dots, \beta_n$ nu se pot scrie sub forma $A\gamma$, se elimină înlocuind aceste producții cu

*Echivalarea
unei gramatici
independente
de context cu*

$$A \rightarrow \beta_1 A' / \beta_2 A' / \dots / \beta_n A'$$

$$A' \rightarrow \alpha_1 A' / \alpha_2 A' / \dots / \alpha_m A' / \varepsilon$$

una fără
recursie la
stânga

unde A' este un neterminal nou.

Această procedură nu elimină, însă, recursivitatea stângă generată de derivări în doi sau mai mulți pași. În acest caz trebuie folosit următorul algoritm:

Algoritmul NREC - de eliminare a recursiei la stânga

Intrare : gramatica G , fără cicluri și fără ε - producții

Ieșire : o gramatică echivalentă cu G , dar fără recursie stângă

Metoda :

P1. ordonează neterminalele în ordinea $A_1, A_2, \dots, A_n$

P2. for $i := 1$ to n do

for $j := 1$ to $i-1$ do

- înlocuiește fiecare producție de forma

$A_i \rightarrow A_j \alpha$ cu producțiile

$A_i \rightarrow \beta_1 \alpha / \beta_2 \alpha / \dots / \beta_m \alpha$ dacă $A_j \rightarrow \beta_1 / \dots / \beta_m$ sunt toate

A_j - producțiile

- elimină recursivitatea stângă directă a lui A_i dacă este cazul.

Algoritmul de
eliminare a
recursiei

I.7.3. Gramatici factorizate la stânga

Dacă $A \rightarrow \alpha\beta_1 / \alpha\beta_2$ sunt două A -producții, iar cuvântul de la intrare începe cu α , nu știm dacă trebuie făcută o expandare a lui A cu $\alpha\beta_1$ sau cu $\alpha\beta_2$. Regulile de acest fel vor fi modificate astfel încât alternativele pentru expandare să difere începând cu primul simbol.

Algoritmul FACT - de factorizare la stânga

Intrare : o GIC G

Ieșire : o gramatică echivalentă, factorizată la stânga

Metoda : Pentru fiecare neterminal A se determină cel mai lung prefix $\alpha \neq \varepsilon$ comun la două sau mai multe alternative. Se înlocuiesc A -producțiile $A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \dots / \alpha\beta_n / \gamma$, unde γ reprezintă toate alternativele ce nu încep cu α , cu

$A \rightarrow \alpha A' / \gamma$

$A' \rightarrow \beta_1 / \beta_2 / \dots / \beta_n$

Factorizarea
la stânga

unde A' este un neterminal nou. Se repetă această transformare până când nu mai există, pentru nici-un neterminal, alternative care să aibă un prefix comun.

Exemplul 7.1. Factorizarea stângă a gramaticii cu producțiile

$$S \rightarrow aEbS / aEbScS / d, \quad E \rightarrow e$$

este

$$S \rightarrow aEbSS' / d$$

$$S' \rightarrow dS / \varepsilon$$

$$E \rightarrow e$$

I.7.4. Gramatici LL(k)

Fie $G = (N, \Sigma, P, S)$ o gramatică neambiguă și $w = a_1 a_2 \dots a_n \in L(G)$.

Atunci există o unică derivare la stânga $p_0 p_1 \dots p_{m-1}$ astfel încât $S = \alpha_0, \alpha_i \xRightarrow{p_i} \alpha_{i+1}; 0 \leq i < m$ și $\alpha_m = w$. Dacă $\alpha_i = a_1 \dots a_j A \beta$, dorim ca secvența α_{i+1} să poată fi determinată cunoscând primele j simboluri (partea din cuvântul de intrare citită până în acel moment), următoarele k simboluri $a_{j+1} \dots a_{j+k}$ (pentru un anumit k) și neterminalul A . Dacă aceste trei cantități determină în mod unic producția folosită pentru expandarea lui A , vom spune că gramatica G este $LL(k)$. În continuare vom considera că neterminalul care derivează este cel mai din stânga.

Definiția 7.2. Fie $G = (N, \Sigma, P, S)$ o GIC, k un număr natural și $\alpha \in (N \cup \Sigma)^*$. Definim funcția $PRIM_k^G(\alpha)$ astfel:

$$PRIM_k^G(\alpha) = \{w \in \Sigma^* / |w| < k \text{ și } \alpha \xRightarrow{*} w$$

$$\text{sau } |w| = k \text{ și } \alpha \xRightarrow{*} wx \text{ pentru un anumit } x\}$$

Am notat cu $|x|$ lungimea secvenței x . Dacă nu există pericol de confuzie vom renunța să mai precizăm gramatica G .

Definiția 7.3. Fie $G = (N, \Sigma, P, S)$ o GIC. Spunem că G este $LL(k)$ dacă:

$$(I) \quad S \xRightarrow{*} wA\alpha \implies w\beta\alpha \xRightarrow{*} wx$$

$$(2) \quad S \xRightarrow{*} wA\alpha \Longrightarrow w\gamma\alpha \xRightarrow{*} wy$$

$$(3) \quad PRIM_k(x) = PRIM_k(y)$$

implică $\beta = \gamma$.

O gramatică este de tip LL dacă ea este $LL(k)$ pentru un anumit k .

Teorema 7.2. Fie $G = (N, \Sigma, P, S)$ o GIC. Atunci G este $LL(k)$ dacă și numai dacă este adevărată condiția : $\forall A \in N$ astfel încât există derivarea $S \xRightarrow{*} wA\alpha$ și pentru orice $A \rightarrow \beta$ și $A \rightarrow \gamma$ producții distincte, rezultă $PRIM_k(\beta\alpha) \cap PRIM_k(\gamma\alpha) = \emptyset$

Demonstrație.

„ $\implies$ ” Fie G o gramatică $LL(K)$. Presupunem prin reducere la absurd că

$$\exists A \in N \text{ astfel încât pentru orice derivație } S \xRightarrow{*} wA\alpha$$

$$\exists (A \rightarrow \beta) \in P, \exists (A \rightarrow \gamma) \in P, \beta \neq \gamma \text{ și } PRIM_k(\beta\alpha) \cap PRIM_k(\gamma\alpha) \neq \emptyset.$$

Fie $u \in PRIM_k(\beta\alpha) \cap PRIM_k(\gamma\alpha)$; rezultă $u \in PRIM_k(\beta\alpha)$, deci

$$\beta\alpha \xRightarrow{*} x \in \Sigma^* \text{ și } u = PRIM_k(x)$$

și $u \in PRIM_k(\gamma\alpha)$, deci $\gamma\alpha \xRightarrow{*} y \in \Sigma^*$ și $u = PRIM_k(y)$.

Am obținut

$$1) \quad S \xRightarrow{*} wA\alpha \Longrightarrow w\beta\alpha \xRightarrow{*} wx, \quad x \in \Sigma^*$$

$$2) \quad S \xRightarrow{*} wA\alpha \Longrightarrow w\gamma\alpha \xRightarrow{*} wy, \quad y \in \Sigma^*$$

$$3) \quad PRIM_k(x) = PRIM_k(y)$$

și totuși $\beta \neq \gamma$; deci gramatica nu este $LL(k)$. Ajungem la o contradicție, deci presupunerea făcută este falsă.

„ $\impliedby$ ” Presupunem prin absurd că G nu este $LL(K)$; deci sunt adevărate condițiile 1), 2), 3) din definiția 1.31 și totuși $\beta \neq \gamma$. Condiția 1) implică existența derivației $\beta\alpha \xRightarrow{*} x \in \Sigma^*$; analog, 2) implică $\gamma\alpha \xRightarrow{*} y \in \Sigma^*$. De aici și din 3) obținem

$$PRIM_k(x) = PRIM_k(y) \in PRIM_k(\beta\alpha) \cap PRIM_k(\gamma\alpha),$$

deci $PRIM_k(\beta\alpha) \cap PRIM_k(\gamma\alpha) \neq \emptyset$, contradicție. Rezultă că presupunerea făcută este falsă, deci G este $LL(k)$. $\square$

Definiția 7.4. Fie $G = (N, \Sigma, P, S)$ o GIC; definim funcția $URM_k(\beta)$,

Definiția
gramaticii LL

Condiția
necesară și
suficientă ca o
gramatică să
fie LL

unde k este un întreg pozitiv iar $\beta \in (N \cup \Sigma)^*$ astfel:

$$URM_k(\beta) = \{w | S \xRightarrow{*} \alpha\beta\gamma \text{ și } w \in PRIM_k(\gamma)\}.$$

Vom da un algoritm care să verifice dacă o gramatică este $LL(k)$, deoarece teorema 7.2 este dificil de utilizat.

Definiția 7.5. Fie Σ un alfabet și $L_1, L_2 \subseteq \Sigma^*$. Definim $L_1 \oplus_k L_2 = \{w / \exists x \in L_1 \text{ și } \exists y \in L_2 \text{ astfel încât}$

a) $w = xy$ dacă $|xy| \leq k$ și

b) w este format din primele k simboluri din xy , în caz contrar}.

Ținând seama de definiția de mai sus rezultă ușor că

$$PRIM_k(\alpha\beta) = PRIM_k(\alpha) \oplus_k PRIM_k(\beta) \quad \forall \alpha, \beta \in (N \cup \Sigma)^*.$$

Acum, condiția ca o gramatică să fie $LL(k)$ se reformulează astfel : G este $LL(k)$ dacă și numai dacă $(\forall) A \in N, \exists (S \xRightarrow{*} wA\alpha)$ astfel încât $\forall (A \rightarrow \beta) \in P, \forall (A \rightarrow \gamma) \in P, \beta \neq \gamma$ implică

$$(PRIM_k(\beta) \oplus_k L) \cap (PRIM_k(\gamma) \oplus_k L) = \emptyset, \text{ unde } L = PRIM_k(\alpha).$$

Algoritmul TEST-LL - de testare a condiției $LL(k)$

Intrare : o GIC $G = (N, \Sigma, P, S)$ și un număr întreg pozitiv k

Ieșire : „da“ dacă gramatica este $LL(k)$ și „nu“ în caz contrar

Metoda :

Se consideră că toate neterminalele sunt nemarcate.

Pasul 1. Pentru orice $A \in N$, nemarcat, pentru care există cel puțin două A - producții distincte calculează

$$\sigma(A) = \{L \subseteq \Sigma^{*k} | S \xRightarrow{*} wA\alpha \text{ și } L = PRIM_k(\alpha)\}$$

Pasul 2. Dacă $A \rightarrow \beta$ și $A \rightarrow \gamma$ sunt două A - producții distincte, calculează

$$f(L) = (PRIM_k(\beta) \oplus_k L) \cap (PRIM_k(\gamma) \oplus_k L)$$

pentru orice $L \in \sigma(A)$. Dacă $f(L) \neq \emptyset$, răspunde „nu“ și oprește algoritmul.

Dacă $f(L) = \emptyset$ pentru orice $L \in \sigma(A)$, repetă pasul 2 pentru toate perechile distincte de A - producții și la sfârșit marchează pe A .

Pasul 3. Repetă pașii 1 și 2 pentru toate neterminalele nemarcate din N .

Pasul 4. Răspunde „da“ dacă toate neterminalele au fost marcate; în caz contrar

Algoritmul de
testare a
condiției LL

se merge la pasul 1.

Pentru a putea aplica acest algoritm trebuie calculate funcțiile $PRIM_k$ și σ .

Algoritmul PRIM. - de calculare a funcției $PRIM$

Intrare : o GIC $G = (N, \Sigma, P, S)$ k un număr întreg pozitiv și

$\alpha = X_1 \dots X_n \in (N \cup \Sigma)^*, n \geq 1$

Ieșire : $PRIM_k(\alpha)$

Metoda : ținând seama de descompunerea

$PRIM_k(\alpha) = PRIM_k(X_1) \oplus_k \dots \oplus_k PRIM_k(X_n)$ rezultă că este suficient să calculăm $PRIM_k(x)$ cu $x \in N$; dacă $x \in \Sigma \cup \{\varepsilon\}$ atunci $PRIM_k(x) = \{x\}$.

Calculăm recursiv mulțimile $F_i(x)$ pentru $x \in N \cup \Sigma$ și $i \geq 0$

Pasul 1. $F_i(a) = \{a\}; \forall a \in \Sigma \cup \{\varepsilon\}, i \geq 0$

Pasul 2.

$$F_0(A) = \left\{ x \in \Sigma^{*k} / (A \rightarrow x\alpha) \in P \right. \\ \left. \text{unde fie } |x| = k \text{ fie } |x| < k \text{ si } \alpha = \varepsilon \right\}$$

Calcularea
valorilor
funcției PRIM

Pasul 3. Presupunem că mulțimile $F_0, F_1, \dots, F_{i-1}$ au fost calculate pentru orice $A \in N$. Atunci :

$$F_i(A) = F_{i-1}(A) \cup \left\{ x / x \in F_{i-1}(y_1) \oplus_k \dots \oplus_k F_{i-1}(y_n), \right. \\ \left. (A \rightarrow y_1 \dots y_n) \in P \right\}$$

Pasul 4. Dacă pentru $\forall A \in N$ avem $F_{i-1}(A) = F_i(A)$ luăm

$PRIM_k(A) = F_i(A)$ și oprim algoritmul; altfel se merge la pasul 3.

Teorema 7.3. Algoritmul PRIM calculează corect valorile funcției $PRIM_k$.

Demonstrație : Deoarece $F_i(A) \subset \Sigma^{*k}$ rezultă că mulțimile $F_i(A)$, cu A fixat și $i = 0, 1, \dots$ sunt în număr finit. Cum $F_i(A) \subset F_{i+1}(A)$, există un n astfel încât $F_{n-1}(A) = F_n(A)$. Apoi pentru orice $j > n$ avem $F_n(X) = F_j(X)$ și deci $F_n(A) = \bigcup_{i=0}^{\infty} F_i(A)$. Este suficient să arătăm că $PRIM_k(A) = \bigcup_{i=0}^{\infty} F_i(A)$.

„ $\subset$ ” Fie $x \in PRIM_k(A)$. Atunci, există un r astfel încât $A \xrightarrow{r} y$ și $x = PRIM_k(y)$. Arătăm prin inducție după r că $x \in F_{r-1}(A)$. Pentru $r = 1$ este trivial, deoarece $x \in F_0(A)$. Să fixăm pe r și să presupunem că ipoteza este

Corectitudinea
algoritmului
PRIM

verificată pentru valori mai mici decât r . Atunci $A \Longrightarrow Y_1 \dots Y_n \xrightarrow{\Gamma-1} y$ unde $y = y_1 y_2 \dots y_n$ și $Y_p \xrightarrow{\Gamma_p} y_p$ pentru $1 \leq p \leq n$. Evident $r_p < r$ și conform ipotezei de inducție, $PRIM_k(y_i) \in F_{r-1}(Y_i) \subset F_{r-2}(Y_i)$. Obținem

$$\begin{aligned} x &= PRIM_k(y) = PRIM_k(y_1 \dots y_n) = PRIM_k(y_1) \oplus_k \dots \oplus_k PRIM_k(y_n) \\ &\subset F_{r-2}(Y_1) \oplus_k \dots \oplus_k F_{r-2}(Y_n) \subset F_{r-1}(A). \end{aligned}$$

„ $\supset$ ” Arătăm că $F_r(A) \subset PRIM_k(A)$ prin inducție după r .

Pentru $r = 0$ este evident. Presupunem afirmația adevărată pentru valori mai mici decât r și să o demonstrăm pentru r . Fie $x \in F_r(A)$; dacă $x \in F_{r-1}(A)$ rezultă că $x \in PRIM_k(A)$, conform ipotezei de inducție.

Dacă $(A \rightarrow y_1 \dots y_p) \in P$ și $x \in F_{r-1}(y_1) \oplus_k \dots \oplus_k F_{r-1}(y_p)$, avem $x \in PRIM_k(y_1) \oplus_k \dots \oplus_k PRIM_k(y_p) = PRIM_k(y_1 \dots y_p) \subset PRIM_k(A)$.

Pentru ultima incluziune am folosit proprietatea următoare

$\alpha \xrightarrow{\Gamma} \beta$ implică $PRIM_k(\beta) \subset PRIM_k(\alpha)$, a cărei demonstrație este evidentă. $\square$

Algoritmul URM1 - de calculare a funcției σ

Intrare : o GIC $G = (N, \Sigma, P, S)$ și k un număr întreg nenegativ

Ieșire : $\sigma(A)$ pentru $\forall A \in N$

Metoda : Pentru $\forall A, B \in N$ calculăm

$\sigma(A, B) = \{L \mid L \subset \Sigma^{*k}, \exists (A \xrightarrow{\Gamma} wB\alpha) \text{ și } L = PRIM_k(\alpha)\}$ Pentru aceasta construim mulțimile $\sigma_i(A, B)$ pentru $\forall A, B \in N$ și $i = 0, 1, \dots$ astfel:

Pasul 1: fie $\sigma_0(A, B) = \{L \subseteq \Sigma^k \mid (A \rightarrow wB\alpha) \in P \text{ și } L = PRIM_k(\alpha)\}$

Pasul 2: presupunem că mulțimile $\sigma_{i-1}(A, B)$ au fost calculate pentru orice $A, B \in N$ și definim $\sigma_i(A, B)$ astfel :

(a) dacă $L \subset \sigma_{i-1}(A, B)$ atunci $L \in \sigma_i(A, B)$

(b) dacă există producția $A \rightarrow X_1 \dots X_n$ și pentru un j , $1 \leq j \leq n$, există o mulțime $L' \in \sigma_{i-1}(X_j, B)$, atunci include în $\sigma_i(A, B)$ pe

$$L = L' \oplus_k PRIM_k(X_{j+1} \dots X_n)$$

Pasul 3: dacă pentru orice $A, B \in N$ există un i astfel încât $\sigma_i(A, B) = \sigma_{i-1}(A, B)$ luăm $\sigma(A, B) = \sigma_i(A, B)$; în caz contrar se merge la pasul (2).

Calculul
valorilor
funcției σ

Pasul 4: pentru orice $A \in N$ se ia $\sigma(A) = \sigma(S, A)$.

Teorema 7.4. *Algoritmul URMI calculează corect valorile funcției σ .*

Demonstrație. Se arată că $L \in \sigma(A)$ dacă și numai dacă există $w \in \Sigma^*$ și $\alpha \in (N \cup \Sigma)^*$ astfel încât $S \xRightarrow{*} wA\alpha$ și $L = PRIM_k(\alpha)$. Demonstrația este similară cu cea din teorema precedentă. $\square$

Teme Curs

Teste Autoevaluare

Testul 1

1. Considerăm gramatica $G = (\{S, A, B\}, \{a, b\}, P, S)$ cu producțiile
 $S \rightarrow aAB, S \rightarrow BA, A \rightarrow BBB, A \rightarrow a, B \rightarrow AS, B \rightarrow b$. Să se construiască gramatica echivalentă aflată în forma normală Chomsky..... 2 puncte
2. Fie gramatica cu producțiile
 $S \rightarrow Aa / b, A \rightarrow Ac / Sd / e$
 - a) Să se arate că este recursivă la stânga..... 1 punct
 - b) Să se construiască o gramatică echivalentă, fără recursie la stânga..... 2 puncte
3. Fie $L_1 = \{\epsilon, abb\}$ și $L_2 = \{b, bab\}$. Calculați
 $L_1 \oplus_2 L_2$1 .punct
4. Considerăm gramatica $G = (\{S, A, B\}, \{a, b\}, P, S)$ cu producțiile
 $S \rightarrow aAaB / bAbB$
 $A \rightarrow a / ab$
 $B \rightarrow aB / a$
Folosind definițiile calculați $PRIM_3(S)$ și $\sigma(A)$ 3 puncte
5. Oficiu 1 punct

TESTUL 2

Se consideră gramatica $G = (\{S, A, B\}, \{a, b\}, P, S)$ cu producțiile

$$S \rightarrow aAaB / bAbB$$

$$A \rightarrow a / ab$$

$$B \rightarrow aB / a$$

- a) Folosind algoritmul PRIM, să se calculeze valorile funcției $PRIM_3$ 3 puncte
- b) Folosind algoritmul σ să se calculeze valorile funcției σ 3 puncte
- c) Să se verifice că gramatica este $LL(3)$ 3 puncte
- Oficiu.....1 punct

Răspunsuri la Testul 1

1. Gramatica $G' = (\{S, A, B, C, D, E\}, \{a, b\}, P', S)$ cu producțiile $P' = \{S \rightarrow CD, D \rightarrow AB, C \rightarrow a, S \rightarrow BA, A \rightarrow BE, E \rightarrow BB, A \rightarrow a, B \rightarrow AS, B \rightarrow b\}$ este în formă normală Chomsky și este echivalentă cu G
2. a) Neterminalul S este recursiv la stânga deoarece $S \Rightarrow Aa \Rightarrow Sda$
 b) Considerând că ordinea neterminalurilor este S, A , gramatica echivalentă are producțiile

$$S \rightarrow Aa / b$$

$$A \rightarrow bdA' / eA'$$

$$A' \rightarrow cA' / adA' / \varepsilon$$
 unde A' este neterminal nou
3. $L_1 \oplus_2 L_2 = \{b, ba, ab\}$
4. $PRIM_3(S) = \{aaa, aab, bab\}$
 $\sigma(A) = \{\{aa, aaa\}, \{ba, baa\}\}$

Răspunsuri la Testul 2

- $PRIM_3(S) = \{aaa, aab, bab\}$
- a) $PRIM_3(A) = \{a, ab\}$
 $PRIM_3(B) = \{a, aa, aaa\}$
 $\sigma(S) = \sigma(S, S) = \emptyset$
 - b) $\sigma(A) = \sigma(S, A) = \{\{aa, aaa\}, \{ba, baa\}\}$
 $\sigma(B) = \sigma(S, B) = \{\{\varepsilon\}\}$
 - c) $\sigma(S) = \emptyset$, deci $L = \emptyset, f(L) = \emptyset$ și se marchează S

Pentru neterminalul A

$$\sigma(A) = \{\{aa, aaa\}, \{ba, baa\}\}$$

Pentru producțiile $A \rightarrow a, A \rightarrow ab$ și $L = \{aa, aaa\}$ avem

$$f(L) = \{aaa\} \cap \{aba\} = \emptyset$$

pentru producțiile $A \rightarrow a, A \rightarrow ab$ și $L = \{ba, baa\}$ avem

$$f(L) = \{aba\} \cap \{abb\} = \emptyset \text{ și se marchează } S.$$

Pentru neterminalul B :

$$\sigma(B) = \{\{\varepsilon\}\}$$

pentru producțiile $B \rightarrow aB$, și $B \rightarrow a$ și $L = \{\{\varepsilon\}\}$ avem
 $f(L) = \{aa, aaa\} \cap \{a\} = \emptyset$ și se marchează neterminalul B

Rezumat: S-a echivalat o gramatică independentă de context cu altele, utile în anumite tipuri de analiză sintactică. S-a prezentat o metodă de verificare a faptului că o gramatică este sau nu de tipul LL.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Teme Laborator/Seminar

1. Să se scrie un program care să construiască forma normală Chomsky
2. Să se implementeze algoritmul de eliminare a recursiei la stânga
3. Să se implementeze algoritmul de calculare a valorilor funcției PRIM

Rezumat: Se echivalează o gramatică independentă de context cu gramatici care vor simplifica analiza sintactică. Se implementează algoritmul PRIM folosit la testarea condiției LL.

Notății

Curs 5 Clase speciale de gramatici independente de context (II)

Curs 5
Durata:
2 ore

Descriere Generală

Se definește noțiunea de gramatică LR apoi se demonstrează condiția necesară și suficientă ca o gramatică să fie LR și se dă un algoritm de verificare.

Se definesc gramaticile de precedență simplă și slabă, relațiile de precedență Wirth-Weber și se explică modul de construire a lor.

Obiective

- cunoașterea metodei de verificare că o gramatică este LR
- cunoașterea metodei de verificare că o gramatică este de precedență simplă sau slabă

Cuprins

I.7.5. Gramatici LR(k)

I.7.6. Gramatici de precedență

Conținut Curs

I.7.5. Gramatici LR(k)

Gramaticile *LR* au fost introduse în 1965 de către Knuth iar clasa limbajelor generate de ele este clasa limbajelor independente de context deterministe. Analiza *LR* prezintă avantajul generalității, toate limbajele de programare ce acceptă o definiție sintactică *BNF* fiind analizabile *LR*.

Pentru construirea arborelui de derivare se dispune de următoarele informații:

i_1) o poziție inițială în cuvântul de analizat pe care o notăm cu p ,

i_2) întregul context stânga al cuvântului sursă, adică $a_1a_2\dots a_p$,

i_3) următoarele k simboluri ale sursei situate după poziția p , adică $a_{p+1}a_{p+2}\dots a_{p+k}$,

Formularea
cerințelor LR

i_4) la fiecare pas, neterminalul care derivează este cel mai din dreapta .

Gramatica G trebuie să aibă astfel de proprietăți încât informațiile $i_1) - i_4)$ să ne asigure următoarele:

- 1) dacă p indică sau nu limita dreaptă a părții reductibile,
- 2) dacă p indică limita dreaptă a părții reductibile atunci $i_1) - i_4)$ determină și limita din stânga,
- 3) dacă a fost determinată partea reductibilă, $i_1) - i_4)$ determină și producția ce va fi utilizată pentru reducere.

Fie $G = (N, \Sigma, P, S)$ și $w \in L(G)$. Pentru $A \in N$ vrem să determinăm șirul $\alpha_0, \alpha_1, \dots, \alpha_m$ astfel încât: $S = \alpha_0 \xRightarrow{d} \alpha_1 \xRightarrow{d} \dots \xRightarrow{d} \alpha_m = w$. Fie $\alpha_{i-1} = \alpha A x$ și $\alpha_i = \alpha \beta x$; informațiile $i_1) - i_4)$ trebuie să determine în mod unic producția $A \rightarrow \beta$ folosită în derivarea $\alpha_{i-1} \xRightarrow{d} \alpha_i$. În cazul gramaticilor LR toate derivările sunt la dreapta; pentru simplificarea scrierii nu vom mai specifica tipul derivării.

Cele de mai sus pot fi unificate sub forma conceptului formal de gramatică $LR(k)$.

Definiția 7.6. Fie $G = (N, \Sigma, P, S)$ o GIC. Se numește gramatică extinsă a lui G , gramatica $G' = (N \cup \{S'\}, \Sigma, P \cup \{S' \rightarrow S\}, S')$ unde $S' \notin N$. Producția $S' \rightarrow S$ o vom numerota cu 0 iar celelalte cu 1, 2, ..., p .

Definiția 7.7. Fie $G = (N, \Sigma, P, S)$ o GIC și $G' = (N', \Sigma, P', S')$ gramatica sa extinsă. G este $LR(k)$, $k \geq 0$, dacă:

- 1) $S' \xRightarrow{*} \alpha A w \implies \alpha \beta w$
- 2) $S' \xRightarrow{*} \gamma B x \implies \gamma \delta x = \alpha \beta y$
- 3) $PRIM_k(w) = PRIM_k(y)$

implică $\alpha A y = \gamma B x$ (adică $\alpha = \gamma, A = B$ și $x = y$).

Definiția
gramaticii de
tip LR

Definiția 7.8. Cuvântul $\gamma \in (N \cup \Sigma)^*$ este un prefix viabil în gramatica $G = (N, \Sigma, P, S)$ dacă $S \xRightarrow{*} \alpha A w \implies \alpha \beta w$ și γ este un prefix al lui $\alpha \beta$; $\alpha \beta$ se numește parte deschisă.

Definiția 7.9. Fie $G = (N, \Sigma, P, S)$ o GIC. Vom spune că $[A \rightarrow \beta_1 \beta_2, u]$ este linie $LR(k)$, dacă $(A \rightarrow \beta_1 \beta_2) \in P$ și $u \in \Sigma^{*k}$. $A \rightarrow \beta_1 \beta_2$ se numește nucleul iar u șirul de anticipare al liniei.

Definiția 7.10. Linia $LR(k)$ $[A \rightarrow \beta_1 \beta_2, u]$ este validă pentru prefixul viabil $\alpha \beta_1$ dacă există o derivare de forma $S \xRightarrow{*} \alpha A w \Rightarrow \alpha \beta_1 \beta_2 w$ și $u = PRIM_k(w)$.

Exemplul 7.2. Fie gramatica cu producțiile:

$$S \rightarrow C / D$$

$$C \rightarrow aC / b$$

$$D \rightarrow aD / c$$

Linia $[C \rightarrow a.C, \varepsilon]$ este validă pentru aaa deoarece există derivația $S \xRightarrow{*} aaC \Rightarrow aaaC$ cu $\alpha = aa$ și $w = \varepsilon$.

Definiția 7.11. Funcția $EFF : (N \cup \Sigma)^* \rightarrow P_f(\Sigma^*)$ (ε -free first) este o restricție a funcției $PRIM$ și este definită astfel:

$$EFF_k(\alpha) = PRIM_k(\alpha) \text{ dacă } \alpha \notin N(N \cup \Sigma)^* \text{ și}$$

$$EFF_k(\alpha) = \{w / \alpha \xRightarrow{*} wx, w = PRIM_k(wx) \text{ și ultima producție folosită nu este } \varepsilon\text{-producție când } \alpha \in N(N \cup \Sigma)^*\}$$

Exemplul 7.3. Pentru gramatica G cu producțiile

$$S \rightarrow AB$$

$$A \rightarrow Ba / \varepsilon$$

$$B \rightarrow Cb / C$$

$$C \rightarrow c / \varepsilon$$

avem $PRIM_2(S) = \{\varepsilon, a, b, c, ab, ac, ba, ca, cb\}$, $EFF_2(S) = \{ca, cb\}$.

Lema 7.1. Fie $G = (N, \Sigma, P, S')$ o gramatică extinsă care nu este $LR(k)$. Atunci:

$$1) S' \xRightarrow{*} \alpha A w \Rightarrow \alpha \beta w$$

$$2) S' \xRightarrow{*} \gamma B x \Rightarrow \gamma \delta x = \alpha \beta y$$

$$3) PRIM_k(w) = PRIM_k(y)$$

$$4) |\alpha\beta| \leq |\gamma\delta|$$

implică $\alpha Ay \neq \gamma Bx$.

Demonstrație. Din definiția gramaticii $LR(k)$ deducem că pot fi satisfăcute toate condițiile din lema exceptând $|\alpha\beta| \leq |\gamma\delta|$. Presupunând $|\alpha\beta| > |\gamma\delta|$ se ajunge la contradicția $\alpha Ay = \gamma Bx$. $\square$

Teorema 7.5. O gramatică $G = (N, \Sigma, P, S)$ este $LR(k)$ dacă și numai dacă este îndeplinită următoarea condiție, pentru orice $u \in \Sigma^{*k}$: dacă $[A \rightarrow \beta., u]$ este linie $LR(k)$ validă pentru prefixul viabil $\alpha\beta$ atunci nu există nici-o altă linie $[A_1 \rightarrow \beta_1.\beta_2, v]$ validă pentru $\alpha\beta$ cu $u \in EFF_k(\beta_2 v)$.

Demonstrație.

" $\Rightarrow$ " Presupunem prin absurd că $\exists u \in \Sigma^{*k}$ și există prefixul viabil $\alpha\beta$ astfel încât $[A \rightarrow \beta., u]$ și $[A_1 \rightarrow \beta_1.\beta_2, v]$ sunt două linii valide pentru prefixul $\alpha\beta$. Înseamnă că:

$$S' \xRightarrow{*} \alpha Aw \Longrightarrow \alpha\beta w \text{ cu } u = PRIM_k(w)$$

$$S' \xRightarrow{*} \alpha_1 A_1 x \Longrightarrow \alpha_1 \beta_1 \beta_2 x \text{ cu } v = PRIM_k(x)$$

și $\alpha\beta = \alpha_1 \beta_1$. În plus, $\beta_2 x \xRightarrow{*} uy$, $u \in EFF_k(\beta_2 v)$.

Cazul I. Dacă $\beta_2 = \varepsilon$ atunci $u = v$ și

$$S' \xRightarrow{*} \alpha Aw \Longrightarrow \alpha\beta w$$

$$S' \xRightarrow{*} \alpha_1 A_1 x \Longrightarrow \alpha_1 \beta_1 x$$

$$PRIM_k(w) = PRIM_k(x) = u = v.$$

Deoarece cele două linii $LR(k)$ sunt distincte, înseamnă că fie $A \neq A_1$, fie $\beta \neq \beta_1$. În plus, $\alpha Ax \neq \alpha_1 A_1 x$, deci G nu este $LR(k)$.

Cazul II. Dacă $\beta_2 = z \in \Sigma^+$ atunci

$$S' \xRightarrow{*} \alpha Aw \Longrightarrow \alpha\beta w, \quad u = PRIM_k(w)$$

$$S' \xRightarrow{*} \alpha_1 A_1 x \Longrightarrow \alpha_1 \beta_1 zx, \quad v = PRIM_k(x)$$

$$PRIM_k(zx) = PRIM_k(w)$$

În acest caz G nu poate fi $LR(k)$ deoarece $\alpha Azx \neq \alpha_1 A_1 x$.

Cazul III. Presupunem că β_2 conține cel puțin un simbol neterminal. Atunci

Condiția
necesară și
suficientă ca o
gramatică să
fie LR

$\beta_2 \xRightarrow{*} u_1 B u_3 \implies u_1 u_2 u_3$ unde $u_1 u_2 \neq \varepsilon$, deoarece conform definiției, $u \in EFF_k(\beta_2 v)$ și prin urmare un simbol neterminal de început nu poate fi înlocuit prin cuvântul vid. Astfel, avem derivațiile:

$$S' \xRightarrow{*} \alpha A w \implies \alpha \beta w$$

$$S' \xRightarrow{*} \alpha_1 A_1 x \implies \alpha_1 \beta_1 \beta_2 x \xRightarrow{*} \alpha_1 \beta_1 u_1 B u_3 x \implies \alpha_1 \beta_1 u_1 u_2 u_3 x = \alpha \beta u_1 u_2 u_3 x$$

cu $u_1 u_2 u_3 x = u y$. Deci $PRIM_k(w) = PRIM_k(u_1 u_2 u_3 x) = u$.

Deoarece gramatica este $LR(k)$ rezultă că $\alpha A u_1 u_2 u_3 x = \alpha_1 \beta_1 u_1 B u_3 x$, adică $A u_1 u_2 = \beta_1 u_1 B$, egalitate imposibilă deoarece $u_1 u_2 \in \Sigma^+$.

" $\Leftarrow$ " Presupunem că G nu este $LR(k)$. Atunci avem

$$S' \xRightarrow{*} \alpha A w \implies \alpha \beta w$$

$$S' \xRightarrow{*} \gamma B x \implies \gamma \delta x = \alpha \beta y$$

$$PRIM_k(w) = PRIM_k(y) = u$$

dar $\alpha A y \neq \gamma B x$. Putem alege derivările astfel încât $\alpha \beta$ să aibă lungimea cât mai mică posibil. Ținând seama de lema 7.1 presupunem că $|\alpha \beta| \leq |\gamma \delta|$. Notăm cu $\alpha_1 A_1 y_1$ ultima formă derivațională la dreapta în derivarea $S' \xRightarrow{*} \gamma B x$ astfel încât lungimea părții sale deschise nu depășește $|\alpha \beta| + 1$, adică $|\alpha_1 A_1| \leq |\alpha \beta| + 1$.

Derivarea de mai sus se poate scrie

$$S' \xRightarrow{*} \alpha_1 A_1 y_1 \implies \alpha_1 \beta_1 \beta_2 y_1 \xRightarrow{*} \alpha_1 \beta_1 y$$

cu $\alpha_1 \beta_1 = \alpha \beta$. Din alegerea lui $\alpha_1 A_1 y_1$ avem $|\alpha_1| \leq |\alpha \beta| \leq |\gamma \delta|$. În plus, în derivarea $\beta_2 y_1 \xRightarrow{*} y$ nu se folosește ca ultimă producție o ε -producție, căci dacă $B \rightarrow \varepsilon$ ar fi ultima producție folosită atunci $\alpha_1 A_1 y_1$ nu ar fi ultima formă derivațională la dreapta în derivarea $S' \xRightarrow{*} \gamma B x$ a cărei parte deschisă nu depășește în lungime $|\alpha \beta| + 1$. Astfel, $u = PRIM_k(y) \in EFF_k(\beta_2 y_1)$. Rezultă că $[A_1 \rightarrow \beta_1 \beta_2, v]$ este linie $LR(k)$ validă pentru $\alpha \beta$, unde $v = PRIM_k(y_1)$.

Din $S' \xRightarrow{*} \alpha A w \implies \alpha \beta w$ deducem că $[A \rightarrow \beta, u]$ este linie validă pentru $\alpha \beta$. Să arătăm acum că liniile $[A \rightarrow \beta, u]$ și $[A_1 \rightarrow \beta_1 \beta_2, v]$ sunt distincte. Pentru aceasta presupunem contrariul; deci $A_1 \rightarrow \beta_1 \beta_2$ coincide cu $A \rightarrow \beta$. Atunci, ultima derivare de mai sus este de forma

$$S' \xRightarrow{*} \alpha_1 A y \implies \alpha_1 \beta y$$

cu $\alpha_1 \beta = \alpha \beta$. Atunci $\alpha_1 = \alpha$ și $\alpha A y = \gamma B x$, contrar ipotezei că G nu este

$LR(k).$ □

Fie G o GIC și γ un prefix viabil. Notăm cu $V_k^G(\gamma)$ mulțimea liniilor $LR(k)$ valide pentru prefixul viabil γ . Vom renunța să mai specificăm pe k și G dacă acestea se subînțeleg.

Algoritmul LINII-LR - de calculare a mulțimilor V_k

Intrare: $G = (N, \Sigma, P, S)$ o GIC, $\gamma = x_1x_2...x_n \in (N \cup \Sigma)^*$ și $n \geq 0$

Ieșire: $V_k^G(\gamma)$

Metoda: Se calculează succesiv

$$V_k^G(\varepsilon), V_k^G(x_1), V_k^G(x_1x_2), \dots, V_k^G(x_1x_2...x_n)$$

Pasul 1. Se construiește mulțimea $V_k^G(\varepsilon)$

(a) Dacă $(S \rightarrow \alpha) \in P$, se include în $V_k^G(\varepsilon)$ linia $[S \rightarrow \cdot \alpha, \varepsilon]$

(b) Dacă $[A \rightarrow \cdot B\beta_2, u] \in V_k^G(\varepsilon)$ și $(B \rightarrow \beta) \in P$, atunci pentru

$\forall x \in PRIM_k(\beta_2u)$ se include în $V_k^G(\varepsilon)$ linia $[B \rightarrow \cdot \beta, x]$, dacă nu a fost deja inclusă.

(c) Se repetă pasul 1.b până când nici-o linie nu mai poate fi adăugată la $V_k^G(\varepsilon)$.

Pasul 2. Presupunem că a fost calculată mulțimea $V_k^G(x_1x_2...x_{i-1})$ și calculăm

$V_k^G(x_1x_2...x_i)$ astfel:

(a) Dacă $[A \rightarrow \beta_1x_i\beta_2, v] \in V_k^G(x_1x_2...x_{i-1})$ atunci se include în $V_k^G(x_1...x_i)$ linia $[A \rightarrow \beta_1x_i\cdot\beta_2, v]$.

(b) Dacă $[A \rightarrow \beta_1\cdot B\beta_2, u] \in V_k^G(x_1...x_i)$ și $(B \rightarrow \beta) \in P$ atunci se include în $V_k^G(x_1x_2...x_i)$ linia $[B \rightarrow \cdot \beta, x]$ pentru $\forall x \in PRIM_k(\beta_2u)$, dacă aceasta nu a fost deja inclusă.

(c) Se repetă pasul (2.b) până când nu se mai poate adăuga nici-o linie la mulțimea $V_k^G(x_1x_2...x_i)$.

Definiția 7.12. Fie gramatica $G = (N, \Sigma, P, S)$ și $k \geq 0$

$A = V_k^G(\gamma)$, unde $\gamma \in (N \cup \Sigma)^*$. Definim funcția GOTO astfel:

$GOTO(A, x) = A'$, unde $A' = V_k^G(\gamma x)$ și $x \in N \cup \Sigma$.

Calcularea
liniilor LR

Următorul algoritm dă o metodă sistematică de calculare a mulțimii de linii $LR(k)$.

Algoritmul COLECȚIE

Intrare: GIC $G = (N, \Sigma, P, S)$ și $k \geq 0$, număr de intrare

Ieșire: $S = \{A / A \in V_k^G(\gamma) \text{ și } \gamma \text{ este prefix viabil al lui } G\}$

Metoda: Inițial $S = \emptyset$

Pasul 1. Se include în S mulțimea $V_k^G(\varepsilon)$ nemarcată.

Pasul 2. Dacă $A \in S$ este nemarcată, atunci se marchează A după ce se calculează pentru fiecare $x \in N \cup \Sigma$ mulțimea $A' = \text{GOTO}(A, x)$. Dacă $A' \neq \emptyset$ și nu există deja în S atunci se adugă A' la S , nemarcată.

Pasul 3. Se repetă pasul (2) până când toate elementele lui S sunt marcate.

Mulțimea S se numește *colecția canonică* de mulțimi de linii $LR(k)$, pentru gramatica G .

Definiția 7.13. Fie $G = (N, \Sigma, P, S)$ o GIC și k un număr întreg nenegativ. O mulțime A de linii $LR(k)$ se numește *consistentă* dacă nu conține două linii de forma $[A \rightarrow \beta_1 \mu]$ și $[B \rightarrow \beta_1 \beta_2 \nu]$ cu $\mu \in \text{EFF}_k(\beta_2 \nu)$.

Din această definiție și din teorema 7.5 rezultă următorul algoritm de testare a condiției $LR(k)$.

Algoritmul TESTARE-LR

Intrare: $G = (N, \Sigma, P, S)$ o GIC și $k \geq 0$ un număr întreg

Ieșire : "DA" dacă G este $LR(k)$ și "NU" în caz contrar

Metoda

Pasul 1. Utilizând algoritmul COLECȚIE se calculează colecția canonică de mulțimi de linii $LR(k)$.

Pasul 2. Se examinează fiecare mulțime de linii $LR(k)$ din S și se determină dacă este consistentă.

Pasul 3. Dacă toate mulțimile din S sunt consistente se răspunde "DA"; în caz contrar se răspunde "NU".

Testare
condiție LR

I.7.6. Gramatici de precedență

Așa cum am văzut anterior, dacă gramatica G este $LR(k)$, determinarea capătului din dreapta al părții reductibile se face cu ajutorul unei funcții asupra următoarelor k simboluri din porțiunea neexplorată a cuvântului de intrare. Adică, dacă este dată derivarea $S \xRightarrow{*} \alpha Aw \Rightarrow \alpha \beta w$, atunci determinarea lui β se face cu ajutorul unei funcții asupra cuvântului $PRIM_k(\beta)$. Conceptul de gramatică de precedență este legat de existența unor relații (numite de precedență) între simbolurile lui $N \cup \Sigma$, care permit izolarea părții reductibile. Dacă $S \xRightarrow[d]{*} \alpha Aw \xRightarrow[d]{*} \alpha \beta w = \alpha \beta' y z w'$ cu $\beta = \beta' y$ și $w = z w'$, atunci izolarea capătului din dreapta a lui β rezultă din:

- 1) toate simbolurile consecutive ale lui β satisfac o anumită relație
- 2) ultimul simbol al lui β și primul simbol al lui w , adică perechea (y, z) , satisfac o altă relație.

Deci, izolarea părții reductibile depinde numai de relația în care se găsesc două simboluri succesive. Relațiile, trei la număr, sunt notate de obicei cu $<, \doteq, >$. În cazul precedenței simple, într-o derivare dreaptă de forma:

$$S \xRightarrow[d]{*} \alpha X_{k+1} A a x \xRightarrow[d]{*} \alpha X_{k+1} X_k \dots X_1 a x$$

porțiunea $X_k \dots X_2 X_1$ este delimitată prin cele trei relații astfel:

- la dreapta: $X_1 > a_1$
- la stânga: $X_{k+1} < X_k$
- în interior: $X_{i+1} \doteq X_i$ pentru $i = 1, \dots, k-1$.

De asemenea, între simbolurile consecutive din αX_{k+1} , avem relația $<$ sau $\doteq$. Facem observația că toate derivările sunt la dreapta.

Definiția 7.14. Fiind dată o GIC $G = (N, \Sigma, P, S)$ se numesc relații de precedență Wirth-Weber relațiile $<, \doteq, >$ definite pentru $X, Y \in N \cup \Sigma$ și $a \in \Sigma$ astfel:

- (1) $X < Y$ dacă există producția $C \rightarrow \beta_1 X B \beta_2$ și $B \xRightarrow{\pm} Y \alpha$
- (2) $X \doteq Y$ dacă există producția $C \rightarrow \beta_1 X Y \beta_2$

(3) $X > a$ dacă există producția $C \rightarrow \beta_1 A \beta_2$ și $A \xRightarrow{+} \alpha X$, $B \xRightarrow{*} a \beta$

La acestea se adaugă relațiile ce implică marcajul \$, de început și sfârșit al cuvântului de intrare:

(4) $\$ < X$ dacă $S \xRightarrow{+} X \alpha$

(5) $X > \$$ dacă $S \xRightarrow{+} \alpha X$.

Definiția 7.15. O GIC $G = (N, \Sigma, P, S)$ este proprie dacă:

(1) nu are simboluri inutile

(2) nu există cicluri, adică derivări de forma $A \xRightarrow{+} A$

(3) nu are ε -producții în afara, eventual, a producției $S \rightarrow \varepsilon$, caz în care S nu apare în membrul drept al nici unei producții.

Definiția 7.16. O GIC proprie, fără ε -producții, în care între oricare două simboluri există cel mult o relație de precedență Wirth-Weber se numește gramatică de precedență. Dacă, în plus, gramatica este și unic invertibilă, ea se numește de precedență simplă.

Lema 7.2. Fie $G = (N, \Sigma, P, S)$ o gramatică proprie fără ε -producții.

(1) Dacă $X < Y$ sau $X \doteq Y$ și $(Y \rightarrow A \alpha) \in P$ atunci $X < A$.

(2) Dacă $X < a$ sau $X \doteq Y$ sau $X > a$ și $(X \rightarrow \alpha Y) \in P$ atunci $Y > a$.

Demonstrație. Demonstrăm (1). Dacă $X < Y$ atunci $(C \rightarrow \beta_1 X B \beta_2) \in P$ și $B \xRightarrow{+} Y \gamma$; deci $B \xRightarrow{+} Y \gamma \Rightarrow A \alpha \gamma$. Din $(C \rightarrow \beta_1 X B \beta_2) \in P$ și $B \xRightarrow{+} A \alpha \gamma$ rezultă $X < A$. Dacă $X \doteq Y$ atunci $(C \rightarrow \beta_1 X Y \beta_2) \in P$; ținând seama și de faptul că $(Y \rightarrow A \alpha) \in P$ rezultă $X < A$. Punctul (2) se demonstrează similar. $\square$

Teorema 7.6. Fie $G = (N, \Sigma, P, S)$ o gramatică proprie fără ε -producții și derivarea:

$$\$ S \$ \xRightarrow{*} X_p X_{p-1} \dots X_{k+1} A a_1 \dots a_q \Rightarrow X_p X_{p-1} \dots X_{k+1} X_k \dots X_1 a_1 \dots a_q$$

Atunci:

(1) pentru $k < i < p$ avem $X_{i+1} < X_i$ sau $X_{i+1} \doteq X_i$

$$(2) X_{k+1} < X_k$$

$$(3) \text{ pentru } 1 \leq i \leq k-1, X_{i+1} \doteq X_i$$

$$(4) X_1 > a_1.$$

Demonstrație. Folosim inducția după numărul n al pașilor în care are loc derivarea indirectă. Pentru $n=0$ avem $SS \xRightarrow{0} SS \Rightarrow X_k \cdots X_1$ și conform definiției 7.14 avem $X_k < X_k, X_i \doteq X_{i+1}$ pentru $1 \leq i \leq k-1$ și $X_1 > \varepsilon$.

$X_k \cdots X_1$ nu poate fi cuvânt vid deoarece gramatica este presupusă fără ε -producții. Presupunem că afirmația din teoremă este adevărată pentru n și fie derivația:

$$\begin{aligned} SS \xRightarrow{n} X_p \cdots X_{k+1} A a_1 \cdots a_q &\Rightarrow X_p \cdots X_{k+1} X_k \cdots X_1 a_1 \cdots a_q \\ &\Rightarrow X_p \cdots X_{j+1} Y_r \cdots Y_1 X_{j-1} \cdots X_1 a_1 \cdots a_q \end{aligned}$$

unde la ultimul pas, X_j ($1 \leq j \leq p$) a fost înlocuit cu $Y_r \cdots Y_1$, deci $X_{j-1}, \dots, X_1$ sunt terminale (dacă $j=1$ atunci $X_{j-1} \cdots X_1 = \varepsilon$). Conform ipotezei inductive,

$X_{j+1} < X_j$ sau $X_{j+1} \doteq X_j$ și din lema 1.2. rezultă $X_{j+1} < Y_r$. Apoi, $Y_r \doteq Y_{r-1} \doteq \cdots \doteq Y_1$, deoarece $Y_r \cdots Y_1$ este membrul drept al unei reguli. Între X_j și X_{j-1} (sau a_1) există una dintre cele trei relații posibile; conform punctului (2) al lemei 1.2 avem $Y_1 > X_{j-1}$ sau $Y_1 > a$ dacă $j=1$. De asemenea, $X_{i+1} < X_i$ sau $X_{i+1} \doteq X_i$ pentru $j < i < p$, conform ipotezei de inducție. $\square$

Corolarul 7.1. Dacă G este gramatică de precedență, atunci concluziile teoremei 7.6 devin:

$$(1) \text{ pentru } k < i < p, \text{ fie } X_{i+1} < X_i, \text{ fie } X_{i+1} \doteq X_i;$$

$$(2) X_{k+1} < X_k;$$

$$(3) \text{ pentru } 1 \leq i \leq k-1, X_{i+1} \doteq X_i;$$

$$(4) X_1 > a_1;$$

(5) între oricare două simboluri din șirul $X_p \cdots X_1 a_1$ nu mai există nici-o altă relație în afara celor precizate în (1)-(4).

Plecând de la definiția 7.14, vom da un algoritm de calculare a relațiilor

Corectitudinea
relațiilor de
precedență

Wirth-Weber. Îl vom explica pe exemplul următor.

Exemplul 7.4. Fie gramatica cu producțiile:

$$S \rightarrow bAAc, \quad A \rightarrow Sa \mid a$$

Ne propunem să determinăm relațiile Wirth-Weber. Fie $\mathbf{M}_{\equiv}$, $\mathbf{M}_{>}$, $\mathbf{M}_{<}$ mulțimea perechilor ce se află respectiv în relația $\equiv$, $>$, $<$. Mulțimea $\mathbf{M}_{\equiv}$ se determină căutând în partea dreaptă a producțiilor, toate perechile de simboluri consecutive; obținem $\mathbf{M}_{\equiv} = \{(b,A), (A,A), (A,c), (S,a)\}$. Pentru a calcula mulțimea $\mathbf{M}_{<}$ se pornește cu ea vidă și cu o mulțime M inițializată cu $\mathbf{M}_{\equiv}$. Se caută în M perechi de forma (X,B) cu $B \in N$; în exemplul nostru ele sunt (b,A) și (A,A) . Pentru fiecare B din aceste perechi căutăm în membrul drept al B – producțiilor primul simbol; fie el Y . Se adaugă (X,Y) la mulțimea $\mathbf{M}_{<}$ și dacă $Y \in N$ se adaugă și la M . În exemplul nostru, din (b,A) obținem (b,a) și (b,S) . Ambele se adaugă la $\mathbf{M}_{<}$ iar (b,S) se adaugă și la M . Calcularea lui $\mathbf{M}_{<}$ se încheie când nu mai există în M nici-o pereche (X,B) netratată. În final se adaugă perechile $(\$,Y)$ unde $S \xrightarrow{+} Y$; în cazul nostru, $(\$,b)$. Obținem $\mathbf{M}_{<} = \{(b,a), (b,S), (b,b), (A,a), (A,S), (A,b), (\$,b)\}$. Pentru a calcula mulțimea $\mathbf{M}_{>}$, o considerăm inițial vidă. Fie M o mulțime inițializată cu $\mathbf{M}_{\equiv}$. În M căutăm perechi de forma (A,B) , cu $A \in N$; în exemplul nostru ele sunt $(A,A), (A,c), (S,a)$. Se calculează $PRIM_1(B)$, apoi pentru fiecare A – producție se ia ultimul simbol al părții drepte, fie el X . Se adaugă la $\mathbf{M}_{>}$ toate perechile (X,a) cu $a \in PRIM_1(B)$, iar la mulțimea M perechile (X,a) cu $X \in N$. În exemplul nostru, pentru (A,A) avem $PRIM_1(A) = \{a,b\}$, iar ultimul simbol al părții drepte este a . Deci perechile adăugate la $\mathbf{M}_{>}$ sunt: (a,a) și (a,b) . Mulțimea rezultată este: $\mathbf{M}_{>} = \{(a,a), (a,b), (a,c), (c,a), (c,\$)\}$. Tabelul relațiilor de precedență este prezentat mai jos.

*Construirea
relațiilor de
precedență*

Condiția ca relațiile de precedență definite pentru o GIC să fie disjuncte este destul de tare. Dacă se impune numai condiția de disjuncție care asigură izolarea corectă a capătului din dreapta al părții reductibile, adică

$> \cap (< \cup \doteq) = \emptyset$, se obține o gramatică de precedență slabă.

	S	A	a	b	c	\$
S			$\doteq$			
A	$<$	$\doteq$	$<$	$<$	$\doteq$	
a			$>$	$>$	$>$	
b	$<$	$\doteq$	$<$	$<$		
c			$>$			$>$
\$				$<$		

Definiția 7.17. O gramatică G , proprie și fără ε -producții, se numește de precedență slabă dacă:

(1) $> \cap (< \cup \doteq) = \emptyset$

(2) dacă $A \rightarrow \alpha X \beta$ și $B \rightarrow \beta$ sunt producții ale lui G , atunci nici una din relațiile $X < B$, $X \doteq B$ nu este validă.

Teme Curs

Teste Autoevaluare

1. Considerăm gramatica $G = (\{E\}, \{a, b, 0, 1\}, P, E)$ cu producțiile

$E \rightarrow E1aE0$ și $E \rightarrow b$. Se cere

- a) Să se construiască colecția canonică de linii $LR(1)$ 4 puncte
- b) Să se verifice că gramatica este $LR(1)$ 2 puncte

2. Fie gramatica cu producțiile:

$E \rightarrow E + T / + T / T$

$T \rightarrow T * F / F$

$F \rightarrow (E) / a$

- a) Să se construiască matricea relațiilor de precedență simplă..... 2 puncte
- b) Să se verifice că gramatica este de precedență slabă..... 1 punct

Oficiu..... 1 punct

Răspunsuri

1. a)

$$V_1(\varepsilon) = A_0 = \{[E' \rightarrow .E, \varepsilon], [E \rightarrow .E1aE0, \varepsilon/1], [E \rightarrow .b, \varepsilon/1]\}$$

$$GOTO(A_0, E) = V_1(E) = A_1 = \{[E' \rightarrow E., \varepsilon], [E \rightarrow E.1aE0, \varepsilon/1]\}$$

$$GOTO(A_0, b) = V_1(b) = A_2 = \{[E \rightarrow b., \varepsilon/1]\}$$

$$GOTO(A_1, 1) = V_1(E1) = A_3 = \{[E \rightarrow E1.aE0, \varepsilon/1]\}$$

$$GOTO(A_3, a) = V_1(E1a) = A_4$$

$$= \{[E \rightarrow E1a.E0, \varepsilon/1], [E \rightarrow .E1aE0, 0/1], [E \rightarrow .b, 0/1]\}$$

$$GOTO(A_4, E) = V_1(E1aE) = A_5 = \{[E \rightarrow E1aE.0, \varepsilon/1], [E \rightarrow E.1aE0, 0/1]\}$$

$$GOTO(A_4, b) = V_1(E1ab) = A_6 = \{[E \rightarrow b., 0/1]\}$$

$$GOTO(A_5, 0) = V_1(E1aE0) = A_7 = \{[E \rightarrow E1aE0., \varepsilon/1]\}$$

$$GOTO(A_5, 1) = V_1(E1aE1) = A_8 = \{[E \rightarrow E1.aE0, 0/1]\}$$

$$GOTO(A_8, a) = V_1(E1aE1a) = A_9 =$$

$$\{[E \rightarrow E1a.E0, 0/1], [E \rightarrow .E1aE0, 0/1], [E \rightarrow .b, 0/1]\}.$$

$$GOTO(A_9, E) = V_1(E1aE1aE)$$

$$= A_{10} = \{[E \rightarrow E1aE.0, 0/1], [E \rightarrow E.1aE0, 0/1]\}$$

$$GOTO(A_9, b) = A_6$$

$$GOTO(A_{10}, 0) = V_1(E1aE1aE0) = A_{11} = \{[E \rightarrow E1aE0., 0/1]\}$$

$$GOTO(A_{10}, 1) = A_8$$

Colecția canonică este $S = \{A_0, A_1, \dots, A_{11}\}$.

b)

Mulțimea A_0 este consistentă deoarece nu conține linii de forma $[A \rightarrow \beta., u]$.

Mulțimea $A_1 = \{[E' \rightarrow E., \varepsilon], [E \rightarrow E.1aE0, \varepsilon/1]\}$ conține linii de ambele forme.

Deoarece $u = \varepsilon$, $\beta_2 = 1aE0$, $v \in \{\varepsilon, 1\}$, $EFF_1(1aE0) = \{1\}$, $EFF_1(1aE01) = \{1\}$ și $\varepsilon \notin \{1\}$ rezultă că această linie este consistentă.

La fel se arată că toate liniile sunt consistente și deci gramatica este $LR(1)$.

2. a)

	E	T	F	a	$($	$)$	$+$	$*$	$\$$
E						$\doteq$	$\doteq$		
T						$>$	$>$	$\doteq$	$>$

F						$>$	$>$	$>$	$>$
a						$>$	$>$	$>$	$>$
$)$						$>$	$>$	$>$	$>$
$($	$<$ $, \doteq$	$<$	$<$	$<$	$<$		$<$		
$+$		$<, \doteq$	$<$	$<$	$<$				
$*$			$\doteq$	$<$	$<$				
$\$$	$<$	$<$	$<$	$<$	$<$		$<$		

b)

Condiția (1) din definiția 1.40 este satisfăcută. Verificăm condiția (2). Considerăm producțiile $E \rightarrow E + T$, $E \rightarrow +T$ și $E \rightarrow T$; deoarece nu există relații de precedență între E și E și nici între $+$ și E rezultă că este satisfăcută condiția (2). Considerăm acum producțiile $T \rightarrow T * F$ și $T \rightarrow F$; deoarece nu există relații de precedență între $*$ și T , condiția (2) este din nou verificată. Deoarece nu mai sunt reguli care să aibă ca sufix al părții drepte partea dreaptă a unei alte reguli, înseamnă că gramatica este de precedență slabă.

Rezumat: S-au definit gramaticile LR și cele de precedență, s-au dat o metodă de verificare că o gramatică este LR și un algoritm de construire a relațiilor de precedență.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Calcularea relațiilor de precedență simplă

Teme Laborator/Seminar

1. Să se scrie un program care să calculeze relațiile de precedență simplă

Rezumat: *Se calculează relațiile de precedență Wirth-Weber*

Notății

II. PROIECTAREA COMPILATOARELOR

Curs 6
Durata:
2 ore

Curs 6 Teoria traducerii. Analiza lexicală

Descriere Generală

Se analizează metode de traducere a limbajelor și se prezintă principiile de realizare a analizei lexicale.

Obiective

- cunoașterea etapelor ce trebuie parcurse în vederea proiectării unui compilator
- cunoașterea principalelor tehnici de traducere a limbajelor
- însușirea metodologiei de proiectare a unui analizor lexical

Cuprins

- II.1.1. Structura unui compilator
- II.1.2. Traducătoare
 - II.1.2.1 Scheme de traducere orientate de sintaxă
 - II.1.2.2. Traducătoare finite
 - II.1.2.3. Traducătoare pushdown
- II.2. Analiza lexicală
 - II. 2.1. Generalități
 - II. 2.2. Proiectarea unui analizor lexical

Conținut Curs

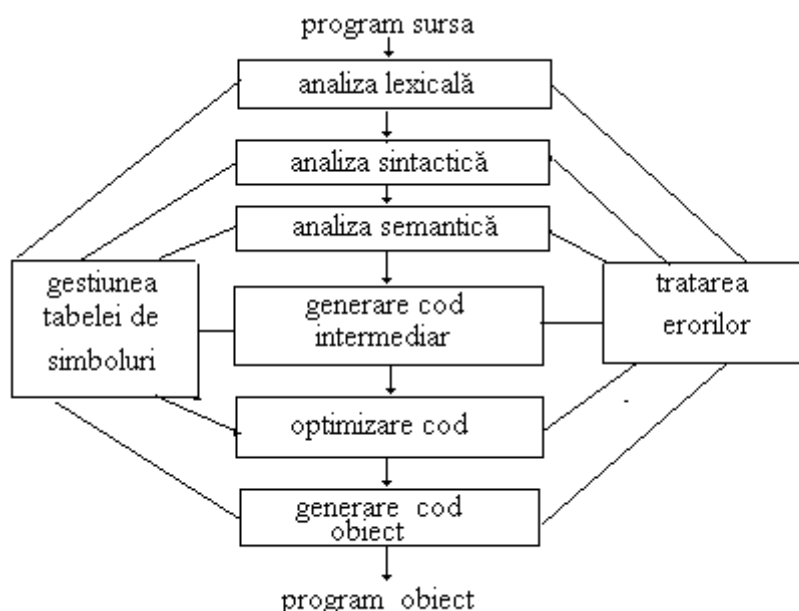
II. 1. TEORIA TRADUCERII

II.1.1. Structura unui compilator

Compilatorul traduce un program scris în limbaj de nivel înalt (FORTRAN, PASCAL, C, C++, JAVA, etc.) în limbaj mașină.

Deci un compilator primește la intrare un program sursă și furnizează la

ieșire un program obiect echivalent (executabil, translatabil, limbaj mașină, etc.) și un fișier de erori. Traducerea programelor sursă în program obiect se face în mai multe faze, așa cum rezultă din figura 1.1.



Schema generală a unui compilator

Figura 1.1

1. *Analiza lexicală* grupează caracterele programului în subșiruri, numite atomi lexicali, care reprezintă : cuvinte cheie, operatori, constante, identificatori. De exemplu, în instrucțiunea de atribuire

$$valfin := valinit + ratia * 60 \quad (1)$$

analiza lexicală depistează următorii atomi lexicali:

- identificatorul *valfin*
- simbolul de atribuire *:=*
- identificatorul *valinit*
- operatorul de adunare *+*
- identificatorul *ratia*
- operatorul de multiplicare ***
- constanta numerică 60.

Spațiile ce separă atomii lexicali sunt eliminate în timpul analizei lexicale. Vom utiliza id_1, id_2 și id_3 pentru *valfin*, *valinit* și *ratia*, respectiv, pentru a sublinia că reprezentarea internă a unui identificator este diferită de secvența de caractere ce formează identificatorul. Reprezentarea internă a instrucțiunii (1) după analiza lexicală este :

Explicarea rolului fiecărei etape de realizare a unui

$$id_1 := id_2 + id_3 * 60.$$

compiler

2. *Analiza sintactică.* Șirul de atomi lexicali este preluat de analiza sintactică; ea depistează în șirul atomilor lexicali structuri sintactice ca: expresii, liste, instrucțiuni, proceduri, plasându-le în arborele sintactic (figura 1.2) ce descrie relațiile de incluziune a structurilor sintactice unele în altele:

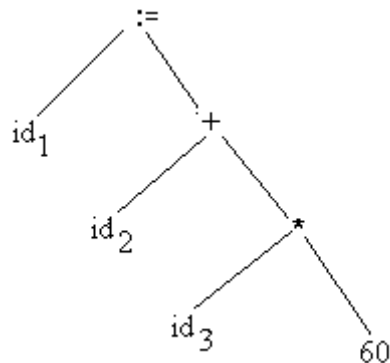


Figura 1.2

3. *Analiza semantică.* Scopul analizei semantice este de a îmbogăți arborele sintactic prin inserarea de informații suplimentare în tabela de simboluri și de a conduce generarea codului intermediar. O componentă importantă a analizei semantice este verificarea tipurilor. Reprezentarea internă a unui număr real este în general diferită de cea a unui număr întreg chiar dacă ele au aceeași valoare. Presupunem, de exemplu, că toți identificatorii din figura 1.3 au fost declarați reali. Operația * se aplică unui număr real *rația* și unui număr întreg 60; are loc conversia tipului întreg la real. Aceasta se realizează prin inserarea unui nod suplimentar *int-in-real*.

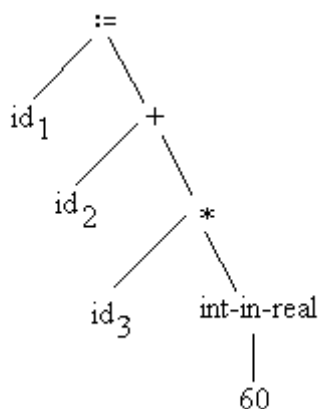


Figura 1.3

4. *Generarea codului intermediar.* Codul intermediar este de obicei, un șir de instrucțiuni simple cu format fix. Operațiile lor se apropie de operațiile calculatorului, iar ordinea lor respectă ordinea execuției. Operanzii instrucțiunilor sunt însă variabile din program sau introduse de compilator și nu regiștri ai calculatorului. De exemplu, arborele semantic din figura 1.3 se traduce în:

$$t_1 := \text{int} - \text{in} - \text{real}(60)$$

$$t_2 := id_3 * t_1$$

$$t_3 := id_2 + t_2$$

$$id_1 := t_3.$$

5. *Optimizarea codului.* În faza de optimizare de cod se elimină redundanțele, calculele și variabilele inutile, pentru a realiza o execuție mai eficientă:

$$t_1 := id_3 * 60.0$$

$$id_1 := id_2 + t_1.$$

6. *Generarea codului.* În această fază se selectează locațiile de memorie pentru fiecare variabilă din program și se translatează codul intermediar într-o secvență de instrucțiuni mașină. Un aspect important constă în alocarea de regiștri pentru variabile:

MOV id₃, r2

MUL #60.0, r2

MOV id₂, r1

ADD r2, r1

MOV r1, id₁

7. *Gestiunea tabelii de simboluri.* O funcție esențială a compilatorului este de a înregistra identificadorii utilizați în programul sursă și de a colecta informații despre diferitele atribute asociate variabilelor. Atributele furnizează informații despre memoria alocată pentru un identificador, tipul său, numărul și tipul parametrilor unei proceduri, modul de transmitere a fiecărui argument. O tabelă de simboluri este o structură de date conținând o înregistrare pentru fiecare identificador, cu câmpuri pentru atribute.

Tabela de simboluri

1	Valfin	
2	Valinit	
3	Rația	
4		

8. *Tratarea erorilor* este o colecție de proceduri ce sunt activate ori de câte ori se depistează o greșeală în program. În acest caz utilizatorul trebuie să primească un mesaj de eroare iar compilarea să continue măcar cu faza de analiză sintactică (în cadrul programului greșit) pentru a detecta și alte eventuale erori.

II.1.2. Traducătoare

Așa cum am precizat anterior, rolul compilatorului este de a traduce programul sursă în program obiect. Traducerea se face în mai multe faze: analiza lexicală, analiza sintactică, etc., fiecare fază fiind la rândul său o traducere. Există mai multe niveluri la care poate fi abordată traducerea. Cel mai general ea este o funcție $f : \Sigma^* \rightarrow \Delta^*$, unde Σ este alfabetul de intrare iar Δ este alfabetul de ieșire. Deci, traducerea este o submulțime $T \subset \Sigma^* \times \Delta^*$; dacă $(x,y) \in T$, atunci y este traducerea lui x .

II.1.2.1 Scheme de traducere orientate de sintaxă

Definiția 1.1. O schemă de traducere orientată de sintaxă (STOS) este un 5-tuplu $S = (N, \Sigma, \Delta, R, S)$, unde :

- N este mulțimea neterminalelor,
- Σ este alfabetul de intrare ,
- Δ este alfabetul de ieșire,
- R este o mulțime de reguli de forma $A \rightarrow \alpha, \beta$ unde $\alpha \in (N \cup \Sigma)^*$, $\beta \in (N \cup \Delta)^*$ iar neterminalele din β sunt (eventual) o permutare a celor din α ,
- S este un neterminal având rolul de simbol inițial.

Definiția
schemei de
traducere

Toate mulțimile de mai sus se presupun a fi finite. Fie $A \rightarrow \alpha, \beta$ o regulă; fiecare neterminal din α este asociat cu unul identic din β . Dacă un neterminal

B apare de mai multe ori în α și β vom folosi indici superiori pentru a pune în evidență asocierea. De exemplu, în regula $A \rightarrow B^{(1)}CB^{(2)}, B^{(2)}B^{(1)}C$, $B^{(1)}$ este asociat cu $B^{(1)}$, $B^{(2)}$ cu $B^{(2)}$ și C cu C .

Definiția 1.2. Numim formă de traducere în STOS $S = (N, \Sigma, \Delta, R, S)$ o pereche $(u, v) \in (N \cup \Sigma)^* \times (N \cup \Delta)^*$ obținută conform regulilor:

(1) (S, S) este o formă de traducere, în care cele două neterminale S sunt asociate;

(2) dacă $(\alpha A \beta, \alpha_1 A \beta_1)$ este o formă de traducere în care cele două simboluri A sunt asociate și $(A \rightarrow \gamma, \gamma_1) \in R$, atunci $(\alpha \gamma \beta, \alpha_1 \gamma_1 \beta_1)$ este o formă de traducere. Neterminalele din γ_1 și γ sunt asociate exact ca în regulă, iar cele din α și β sunt asociate cu cele din α_1 și β_1 exact ca în vechea formă de traducere.

Pentru a defini relația dintre cele două forme de traducere vom scrie:

$$(\alpha A \beta, \alpha_1 A \beta_1) \xRightarrow[S]{*} (\alpha \gamma \beta, \alpha_1 \gamma_1 \beta_1).$$

Vom utiliza notațiile: $\xRightarrow[S]{+}$, $\xRightarrow[S]{*}$ și $\xRightarrow[S]{k}$ pentru închiderea tranzitivă, închiderea reflexivă și tranzitivă și respectiv derivarea în k pași. Dacă nu există pericol de confuzie, nu vom mai specifica schema de traducere S . Traducerea definită de STOS S este

$$\tau(S) = \left\{ (x, y) / (S, S) \xRightarrow{*} (x, y), x \in \Sigma^* \text{ și } y \in \Delta^* \right\}.$$

Traducerea realizată de schema

Definiția 1.3. Fie $S = (N, \Sigma, \Delta, R, S)$ o STOS. Gramatica $G_i = (N, \Sigma, P_i, S)$ unde $P_i = \{ A \rightarrow \alpha \mid (A \rightarrow \alpha, \beta) \in R \}$ se numește gramatica de intrare a schemei S . Gramatica $G_e = (N, \Delta, P_e, S)$ unde $P_e = \{ A \rightarrow \beta \mid (A \rightarrow \alpha, \beta) \in R \}$ se numește gramatica de ieșire a schemei S .

Schema de traducere orientată de sintaxă poate fi privită ca o metodă de transformare a arborilor de derivare din gramatica G_i în arbori de derivare din gramatica G_e .

Algoritmul 1.1. - de transformare a arborilor

Intrare: o STOS $S = (N, \Sigma, \Delta, R, S)$ cu gramatica de intrare $G_i = (N, \Sigma, P_i, S)$, gramatica de ieșire $G_e = (N, \Delta, P_e, S)$ și arborele de derivare D în G_i cu frontiera $x \in \Sigma^*$.

Ieșire: un arbore de derivare D' în G_e cu frontiera $y \in \Delta^*$ astfel încât $(x, y) \in \tau(S)$.

Metoda:

(P1) Aplică pasul (P2), recursiv, începând cu nodul rădăcină al lui D .

(P2) Fie q nodul căruia i se aplică acest pas. Presupunem că q este nod interior și are ca descendenți nodurile $q_1, \dots, q_k$:

(a) Șterge descendenții direcți ai lui q care sunt noduri maximale (etichetate cu ε sau cu terminale).

(b) Fie $A \rightarrow \alpha$ producția din G_i reprezentată de nodul q și descendenții săi; A este eticheta lui q iar α este o secvență formată prin concatenarea etichetelor nodurilor $q_1, \dots, q_k$. Fie $(A \rightarrow \alpha, \beta) \in R$; permută neterminalele din α în ordinea dată de β , subarborii dominați de aceste noduri rămânând neschimbați.

(c) Inserează noduri maximale cu etichete din β în ordinea dată de β .

(d) Aplică pasul (P2) descendenților direcți ai lui q care sunt noduri nemaximale, în ordinea de la stânga la dreapta.

(P3) Subarborele rezultat este Δ' .

Definiția 1.4. O STOS $S = (N, \Sigma, \Delta, R, S)$ astfel încât în fiecare regulă $A \rightarrow \alpha, \beta$ neterminalele asociate apar în aceeași ordine în α și β se numește schemă simplă de traducere orientată de sintaxă (SSTOS).

II.1.2.2. Traducătoare finite

Traducătoarele finite sunt automate finite înzestrate cu ieșiri. Un traducător finit, pornind dintr-o stare dată trece în altă stare și emite o secvență pe un alfabet de ieșire când primește un simbol de intrare sau simbolul vid (notat ε).

Definiția 1.5. Un traducător finit F este un 6-tuplu $(Q, \Sigma, \Delta, \delta, q_0, F)$ unde
- Q este mulțimea stărilor,

Relatia dintre
scheme și
gramatici

- Σ este alfabetul de intrare,
 - Δ este alfabetul de ieșire,
 - δ este funcția de transfer; $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P_f(Q \times \Delta^*)$ unde P_f înseamnă mulțimea părților finite,
 - $q_0 \in Q$ este starea inițială,
 - $F \subseteq Q$ este mulțimea stărilor finale.
- Toate mulțimile de mai sus sunt finite.

Definiția
traducătorului
finit

Definiția 1.6. Numim configurație a traducătorului finit F un triplet (q, x, y) unde :

- q este starea curentă,
- x este porțiunea din secvența de intrare rămasă de citit,
- y este secvența emisă pe banda de ieșire până în momentul curent.

Definim $\vdash$, relație binară între configurațiile lui F . Pentru $q \in Q, a \in \Sigma \cup \{\varepsilon\}, x \in \Sigma^*$ și $y \in \Delta^*$ astfel încât $(r, z) \in \delta(q, a)$ cu $r \in Q$ și $z \in \Delta^*$ scriem $(q, ax, y) \vdash (r, x, yz)$.

Notăm cu $\vdash^+$, $\vdash^*$, $\vdash^i$ închiderea tranzitivă, închiderea reflexivă și tranzitivă și respectiv trecerea în i pași. Dacă pentru $x \in (\Sigma \cup \{\varepsilon\})^*$ există $y \in \Delta^*$ și $q \in F$ astfel încât $(q_0, x, \varepsilon) \vdash^*(q, \varepsilon, y)$ spunem că y este traducerea lui x și notăm $y = \tau(x)$. Traducerea definită de F notată $\tau(F)$ este $\{(x, y) / (q_0, x, \varepsilon) \vdash^*(q, \varepsilon, y) \text{ pentru } q \in F\}$.

Funcționarea
traducătorului
finit

Definiția 1.7. Traducătorul finit F este determinist dacă pentru orice $q \in Q^*$:

- (1) sau $\delta(q, a)$ conține cel mult un element pentru orice $a \in \Sigma$ și $\delta(q, \varepsilon)$ este mulțimea vidă sau
- (2) $\delta(q, \varepsilon)$ conține un element și pentru orice $a \in \Sigma, \delta(q, a)$ este mulțimea vidă.

II.1.2.3. Traducătoare pushdown

O altă clasă importantă de traducătoare o reprezintă traducătoarele pushdown. Un astfel de traducător este un automat pushdown înzestrat cu o

ieşire.

Definiția 1.8. Se numește traducător pushdown un 8-tuplu $P = (Q, \Sigma, \Gamma, \Delta, \delta, q_0, Z_0, F)$, unde toate simbolurile au aceeași semnificație ca în cazul automatului finit, exceptând pe Γ care este alfabetul (memoria) pushdown iar $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow P_f(Q \times \Gamma^* \times \Delta^*)$.

Definiția
traducătorului
pushdown

Definiția 1.9. Se numește configurație a lui P un 4-tuplu (q, x, α, y) unde

- q este starea curentă,
- x este porțiunea din secvența de intrare rămasă de citit,
- α este conținutul memoriei pushdown,
- y este secvența emisă pe banda de ieşire până în momentul curent.

Dacă $(r, \alpha, z) \in \delta(q, a, Z)$ scriem $(q, ax, Z\gamma, y) \vdash (r, x, \alpha\gamma, yz)$ pentru $x \in \Sigma^*, \gamma \in \Gamma^*$ și $y \in \Delta^*$. Spunem că y este traducerea lui x și scriem $y = \tau(x)$, dacă $(q_0, x, Z_0, \varepsilon) \vdash^* (q, \varepsilon, \alpha, y)$ pentru $q \in F$ și $\alpha \in \Gamma^*$. Traducerea definită de P notată $\tau(P)$ este $\{(x, y) \mid (q_0, x, Z_0, \varepsilon) \vdash^* (q, \varepsilon, \alpha, y) \text{ pentru } q \in F \text{ și } \alpha \in \Gamma^*\}$. y este traducerea lui x cu memorie pushdown vidă dacă $(q_0, x, Z_0, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon, y)$ pentru un $q \in Q$. Traducerea definită de P cu memoria pushdown vidă, notată $\tau_\varepsilon(P)$ este $\{(x, y) \mid (q_0, x, Z_0, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon, y) \text{ pentru un } q \in Q\}$.

Funcționarea
traducătorului
pushdown

Definiția 1.10. Traducătorul pushdown $P = (Q, \Sigma, \Gamma, \Delta, \delta, q_0, Z_0, F)$ este determinist dacă :

- (1) pentru orice $q \in Q, a \in \Sigma \cup \{\varepsilon\}$ și $z \in \Gamma$, $\delta(q, a, Z)$ conține cel mult un element și
- (2) dacă $\delta(q, \varepsilon, Z) \neq \emptyset$, atunci $\delta(q, a, Z) = \emptyset$ pentru orice $a \in \Sigma^+$.

La fel ca în cazul automatelor pushdown, are loc echivalența din

Teorema 1.1. Traducerea τ este realizată de traducătorul pushdown P_1 dacă și numai dacă ea este realizată de un traducător pushdown P_2 cu memoria

pushdown vidă.

Teoremele următoare arată că traducătoarele pushdown caracterizează clasa schemelor simple de traducere orientate de sintaxă în aceeași manieră în care automatele pushdown caracterizează limbajele independente de context.

Teorema 1.2. Pentru orice SSTOS există un traducător pushdown P astfel încât $\tau(S) = \tau_\varepsilon(P)$

Demonstrație. Fie $S = (N, \Sigma, \Delta, R, S)$ o SSTOS. Vom avea nevoie să facem distincție între simbolurile lui Σ și cele ale lui Δ . De aceea vom înlocui Δ cu un alfabet Δ' astfel încât $\Sigma \cap \Delta' = \emptyset$. Δ' conține un nou simbol a' pentru fiecare $a \in \Delta$. Fie h homomorfismul definit prin $h(a) = a'$ pentru $a \in \Delta \cap \Sigma$ și $P = (\{q\}, \Sigma, N \cup \Sigma \cup \Delta', \Delta', \delta, q, S, \emptyset)$ unde funcția δ este definită astfel:

(1) dacă $(A \rightarrow x_0 B_1 x_1 \dots B_k x_k, y_0 B_1 y_1 \dots B_k y_k) \in R$ cu $k \geq 0$, $x_i \in \Sigma^*$, $y_i \in \Delta'^*$, $B_i \in N$ atunci $\delta(q, \varepsilon, A)$ conține pe $(q, x_0 h(y_0) B_1 x_1 h(y_1) \dots B_k x_k h(y_k), \varepsilon)$

(2) $\delta(q, a, a) = \{(q, \varepsilon, \varepsilon)\}$ pentru orice $a \in \Sigma$

(3) $\delta(q, \varepsilon, a') = \{(q, \varepsilon, a)\}$ pentru orice $a \in \Delta$.

Se arată, prin inducție după m și n , că

“pentru orice $A \in N$ și $m, n \geq 1$ $(A, A) \xRightarrow{m} (x, y)$ pentru un anumit m

dacă și numai dacă $(q, x, A, \varepsilon) \xRightarrow{n} (q, \varepsilon, \varepsilon, y)$ pentru un anumit n ”.

Luând $A = S$ avem $(S, S) \xRightarrow{*} (x, y)$ dacă și numai dacă $(q, x, S, \varepsilon) \xRightarrow{*} (q, \varepsilon, \varepsilon, y)$, deci $\tau_\varepsilon(P) = \tau(S)$. $\square$

Teorema 1.3. Fie $P = (Q, \Sigma, \Gamma, \Delta, \delta, q_0, Z_0, F)$ un traducător pushdown. Atunci există o SSTOS S astfel încât $\tau(S) = \tau_\varepsilon(P)$.

Demonstrație. Construcția este similară cu cea a obținerii unei gramatici independente de context dintr-un automat pushdown. Fie $S = (N, \Sigma, \Delta, R, S)$ unde

(1) $N = \{[pAq] \mid p, q \in Q, A \in \Gamma\} \cup \{S\}$

(2) R este definită astfel

(a) dacă $(r, x_1 x_2 \dots x_k, y) \in \delta(p, a, A)$ atunci, pentru $k > 0$, R conține regulile $[pAq] \rightarrow a[rx_1 q_1][q_1 x_2 q_2] \dots [q_{k-1} x_k q_k]y[rx_1 q_1][q_1 x_2 q_2] \dots [q_{k-1} x_k q_k]$ pentru orice $q_1, q_2, \dots, q_k \in Q$ și $q = q_k$. Pentru $k = 0$ regula este $[pAr] \rightarrow a, y$

Teoremele 1.2
și 1.3
echivalează
schemele de
traducere cu
traducatoare
pushdown

(b) pentru orice $q \in Q, R$ conține regula $S \rightarrow [q_0 Z_0 q][q_0 Z_0 q]$

Prin inducție după m și n se arată că $([pAq][pAq]) \Rightarrow^m (x, y)$ dacă și numai dacă $(p, x, A, \varepsilon) \vdash^n (q, \varepsilon, \varepsilon, y)$ pentru orice $p, q \in Q$ și $A \in \Gamma$. Apoi avem $(S, S) \Rightarrow ([q_0 Z_0 q][q_0 Z_0 q]) \Rightarrow^+ (x, y)$ dacă și numai dacă $(q_0, x, Z_0, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon, y)$ deci $\tau(S) = \tau_\varepsilon(P)$. $\square$

Teorema 1.4. Dacă S este o SSTOS atunci $t \in \tau(S)$ dacă și numai dacă există un traducător pushdown P astfel încât $t \in \tau(P)$.

Demonstrație. Rezultă din teoremele 1.17, 1.18 și 1.19. $\square$

II.2. ANALIZA LEXICALĂ

II. 2.1. Generalități

Analiza lexicală este prima fază a compilării, rolul său fiind de a citi caracterele de intrare și a produce la ieșire o secvență de atomi lexicali (tokens , în engleză) utilizați apoi în analiza sintactică.

Un atom lexical este un reprezentant al unei clase de șiruri de caractere cu reguli precise de formare. În majoritatea limbajelor de programare, următoarele construcții sunt tratate ca atomi lexicali: identificatori, constante, cuvinte cheie, operatori, șiruri de caractere, semne de punctuație (paranteze, virgulă, punct și virgulă, etc.), etc. Sarcina analizei lexicale este să selecteze în șirul de caractere al programului sursă subșiruri ce respectă regulile de formare a atomilor lexicali, să clasifice aceste subșiruri și să le traducă în atomi lexicali.

Un atom lexical va fi reprezentat printr-o pereche $\langle$ clasa atomului, valoarea sa $\rangle$. Pentru atomii care au o valoare particulară nu vom mai folosi perechea, ci vom pune în evidență numai valoarea. De exemplu, pentru instrucțiunea

$$\text{COST} = (\text{PREȚ} + \text{TAXA}) \times 0.45$$

se pun în evidență următorii atomi lexicali

$\langle \text{id}, \text{COST} \rangle$; = ; (; $\langle \text{id}, \text{PREȚ} \rangle$; + ; $\langle \text{id}, \text{TAXA} \rangle$;) ; $\times$; 0.45

Analiza lexicală trebuie să furnizeze informații mai detaliate care servesc și în alte scopuri; acestea sunt colectate în atributele asociate. În general există un

singur atribut - un pointer spre tabela de simboluri unde se află depusă valoarea atomului. Analiza lexicală poate fi implementată ca o fază separată sau se poate întrepătrunde cu analiza sintactică. Separarea celor două faze are următoarele avantaje :

- proiectarea este mai simplă, conform principiului modularității ;
- sintaxa atomilor lexicali poate fi exprimată printr-o gramatică regulată; deci analiza lexicală poate fi efectuată de un automat finit și nu de unul pushdown, ca în cazul analizei sintactice ;
- analiza lexicală curăță textul de informațiile inutile: comentarii, blancuri.

Analizorul lexical poate fi comandat de cel sintactic; în acest caz el apare ca o rutină a analizorului sintactic pe care acesta o apelează ori de câte ori are nevoie de un nou simbol.

II. 2.2. Proiectarea unui analizor lexical

În cazul unui analizor lexical complex, acesta va fi descompus în „bucăți” ce vor fi legate între ele în vederea obținerii rezultatului final. Exemplificăm cele spuse mai sus folosind următoarea gramatică:

- (G_0)
- (1) $\langle \text{șir atomi} \rangle := \langle \text{atom} \rangle \langle \text{șir atomi} \rangle \langle \text{atom} \rangle$
 - (2) $\langle \text{atom} \rangle := \langle \text{id} \rangle \langle \text{const} \rangle \langle \text{op} \rangle \langle \text{del} \rangle \langle \text{com} \rangle$
 - (3) $\langle \text{id} \rangle := \langle \text{lit} \rangle \langle \text{id} \rangle \langle \text{lit} \rangle \langle \text{id} \rangle \langle \text{cif} \rangle$
 - (4) $\langle \text{const} \rangle := \langle \text{cif} \rangle \langle \text{const} \rangle \langle \text{cif} \rangle$
 - (5) $\langle \text{op} \rangle := + / * \langle \text{com} \rangle \langle \text{del} \rangle \langle \text{op} \rangle \langle \text{com} \rangle \langle \text{del} \rangle \langle \text{op} \rangle$
 - (6) $\langle \text{del} \rangle := \text{; / blanc}$
 - (7) $\langle \text{com} \rangle := * \langle \text{com} \rangle \text{ / orice șir ce nu conține grupul } * \text{ / } *$
 - (8) $\langle \text{lit} \rangle := A / B / C / \dots / Z / a / \dots / z$
 - (9) $\langle \text{cif} \rangle := 0 / 1 / \dots / 9$

*Structura
atomilor
lexicali*

Gramatica (G_0) nu este regulată, dar poate fi transformată în una regulată rescriind producțiile. De exemplu, producția (4) se poate scrie $\langle \text{const} \rangle = 0 / 1 / 2 / \dots / 9 / \langle \text{const} \rangle 0 / \langle \text{const} \rangle 1 / \dots / \langle \text{const} \rangle 9$.

Pentru a ușura proiectarea analizorului lexical, se preferă stratificarea gramaticii (G_0) într-o ierarhie de gramatici mai simple, regulate, care apoi trebuie cuplate astfel încât limbajul generat să rămână același. Stratificarea începe cu partiționarea mulțimii neterminalelor și stabilirea unei ierarhii între elementele partiției. În cazul nostru putem realiza următoarea partiție:

$$\begin{aligned}
N_1 &= \{ \langle siratomi \rangle, \langle atom \rangle \} \\
N_2 &= \{ \langle id \rangle, \langle const \rangle, \langle op \rangle, \langle del \rangle, \langle com \rangle \} \\
N_3 &= \{ \langle lit \rangle, \langle cif \rangle \}
\end{aligned}$$

Cu ajutorul acestei partiții, descompunem gramatica (G_0) în trei gramatici; pentru fiecare gramatică vom considera ca terminale, pe lângă terminalele din (G_0), și neterminalele din grupul imediat inferior în ierarhie. Cele trei gramatici vor fi :

$$(G_1): \langle siratomi \rangle := \langle atom \rangle / \langle siratomi \rangle \langle atom \rangle$$

$$\langle atom \rangle := id/const/op/del/com$$

$$(G_{21}): \langle id \rangle := lit / \langle id \rangle lit / \langle id \rangle cif$$

$$(G_{22}): \langle const \rangle := cif / \langle const \rangle cif$$

$$(G_{23}): \langle op \rangle := + / * / < / <= / > / >= / = / <>$$

$$(G_{24}): \langle del \rangle := ; / \text{blanc}$$

$$(G_{25}): \langle com \rangle := * / \text{orice sir de caractere ce nu contine grupul } * / > * /$$

$$(G_{31}): \langle lit \rangle := A/B/.../Z/a/.../z$$

$$(G_{32}): \langle cif \rangle := 0/1/.../9$$

Proiectarea
unui analizor
lexical prin
stratificarea
gramaticii

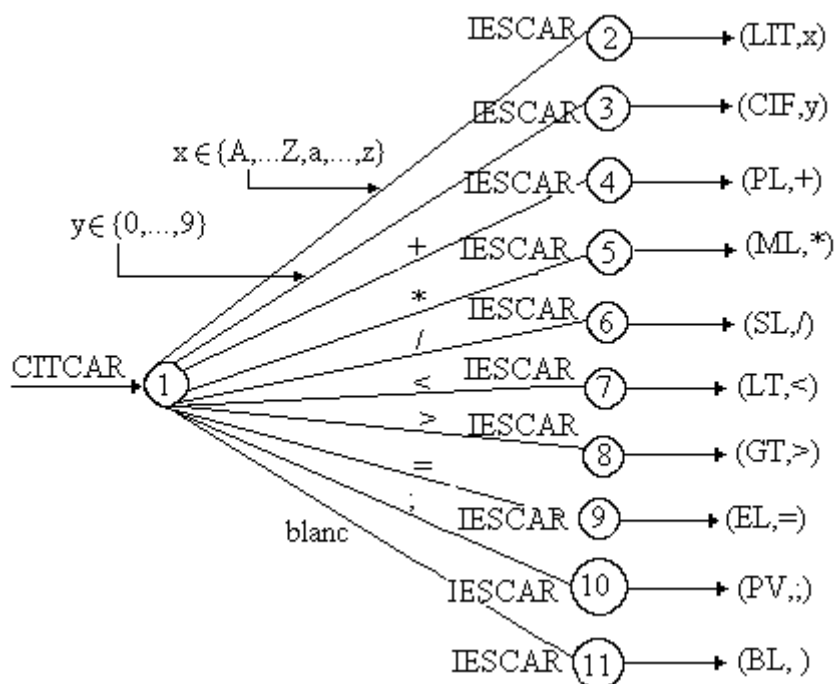
Noile gramatici sunt regulate, cu excepția lui G_1 și G_{25} . Le putem, însă, transforma, la fel cum am transformat anterior producția (4). Fiecare gramatică va fi modelată printr-un automat finit; fiecare automat de pe un nivel inferior trebuie să transmită nivelului superior informația de acceptare a șirului inspectat.

Cuplarea automatelor A_1, A_2 și A_3 se face în serie, ieșirea unuia fiind intrare pentru celălalt. Automatele ce se cuplează trebuie să satisfacă următoarele condiții:

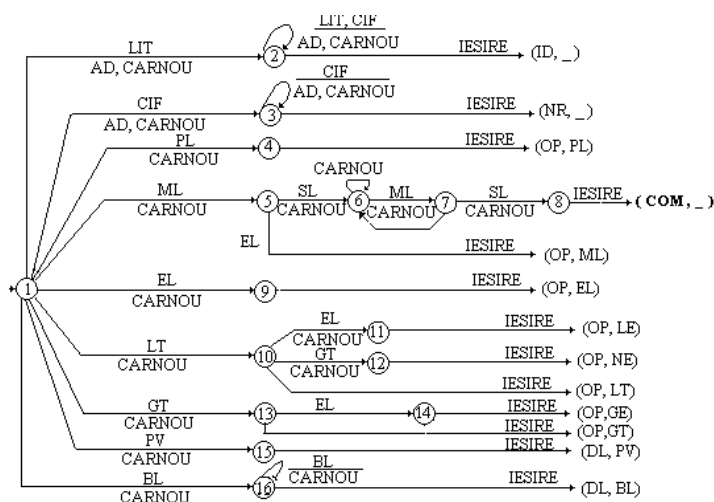
- să fie deterministe;
- orice simbol primit la intrarea unui automat și care nu activează nici o tranziție trebuie să conducă la o situație de eroare.

În cazul nostru nedeterminismul apare în cazul automatului A_2 și se manifestă prin existența a două tranziții din starea inițială pentru simbolul $*$. În diagrama ce o vom prezenta mai târziu vom înlătura acest nedeterminism. Diagramele de tranziții ale automatelor se vor completa cu proceduri semantice, care au ca scop restabilirea la sfârșitul analizei unui atom lexical a unui context adecvat căutării următorului atom, emiterea unei ieșiri corecte care să reprezinte atomul analizat și semnalarea erorilor.

În figura următoare este prezentată diagrama de tranziții a automatului A_3 , completată cu procedurile semantice *CITCAR* și *IESCAR*.



Procedura *CITCAR* este apelată înainte de începutul funcționării automatului și are rolul de a prelua din fișierul de intrare caracterul următor. Procedura *IESCAR* furnizează ieșirea automatului A_3 ; ea este formată din clasa caracterului și caracterul propriu-zis. În figura următoare se prezintă diagrama de tranziții a automatului A_2 completată cu procedurile *AD*, *CARNOU* și *IEȘIRE*.



Procedura *AD* este folosită pentru construcția identificatorilor și constantelor. Ea adaugă într-un tablou de caractere (șir) caracterul primit la intrare. Tabloul acumulează toate caracterele identificatorului sau constantei analizate. *CARNOU* simulează intrarea automatului aducând următorul caracter din textul sursă, dar clasificându-l conform automatului A_3 . Procedura *IEȘIRE* formează ieșirea automatului A_2 , alcătuită din clasa automatului și atomul lexical.

În mod similar se proiectează diagrama de tranziție a automatului A_1 completată cu procedurile semantice *CAUTĂID*, *CAUTĂNR*, *SELECT*. La fel ca în majoritatea limbajelor de programare, comentariile și blaturile sunt eliminate. Procedura *CAUTĂID* caută în tabela de simboluri identificatorul din tabloul șir și dacă nu-l găsește îl introduce. Ea întoarce adresa la care se găsește identificatorul, completând cu ea a doua informație din atomul lexical. Dacă identificatorul este cuvânt cheie, se înlocuiește clasa atomului cu numele cuvântului cheie iar a doua componentă se omite. Procedura *CAUTĂNR* determină valoarea constantei numerice din tabloul șir și caută în tabela de simboluri dacă există o constantă cu aceeași valoare. Dacă nu o găsește, valoarea din șir este plasată în tabelă. Procedura determină adresa intrării corespunzătoare constantei și memorează adresa în atomul lexical. Procedura *SELECT* discerne blaturile de ; .

Teme Curs

Teste Autoevaluare

1. Care este deosebirea dintre analiza sintactică și cea semantică?

..... 1 punct

2. Fie STOS $S = (\{E\}, \{+, \times, a\}, \{+, \times, a\}, R, E)$ unde R este mulțimea de reguli:

$$E \rightarrow E^{(1)} + E^{(2)}, E^{(1)} E^{(2)} +$$

$$E \rightarrow E^{(1)} \times E^{(2)}, E^{(1)} E^{(2)} \times$$

$$E \rightarrow a, a$$

Să se găsească traducerea secvenței $a + a \times a$ 2.5 puncte

3. Considerăm STOS $S = (\{S, A\}, \{0, 1\}, \{a, b\}, R, S)$ unde R este formată din regulile

$$S \rightarrow 0AS, SAa$$

$$A \rightarrow 0SA, ASa$$

$$S \rightarrow 1, b$$

$$A \rightarrow 1, b$$

Folosind algoritmul de transformare a arborilor, să se găsească traducerea secvenței 00111 2.5 puncte

4. Fie traducătorul pushdown $P = (\{q\}, \{a, +, \times\}, \{+, \times, E\}, \{a, +, \times\}, \delta, q, E, \{q\})$ unde funcția de transfer este definită astfel:

$$\delta(q, a, E) = \{(q, \varepsilon, a)\}$$

$$\delta(q, +, E) = \{(q, EE+, \varepsilon)\}$$

$$\delta(q, \times, E) = \{(q, EE\times, \varepsilon)\}$$

$$\delta(q, \varepsilon, +) = \{(q, \varepsilon, +)\}$$

$$\delta(q, \varepsilon, \times) = \{(q, \varepsilon, \times)\}$$

Care este traducerea secvenței $+ \times aaa$? 2.5 puncte

5. Este mai avantajos ca analiza lexicală să fie separată de cea sintactică?

..... 0.5 puncte

Oficiu 1 punct

Răspunsuri

1. Analiza semantică extinde arborele construit în analiza sintactică

2. Avem derivarea:

$$\begin{aligned}(E, E) &\Rightarrow (E^{(1)} + E^{(2)}, E^{(1)} E^{(2)} +) \\&\Rightarrow (E^{(1)} + E^{(3)} \times E^{(4)}, E^{(1)} E^{(3)} E^{(4)} \times +) \\&\Rightarrow (a + E^{(3)} \times E^{(4)}, a E^{(3)} E^{(4)} \times +) \\&\Rightarrow (a + a \times E^{(4)}, aa E^{(4)} \times +) \\&\Rightarrow (a + a \times a, aaa \times +),\end{aligned}$$

deci $\tau(a + a \times a) = aaa \times +$

3. $\tau(00111) = bbbaa$

4. Traducerea secvenței $+ \times aaa$ se desfășoară astfel

$$\begin{aligned}(q, + \times aaa, E, \varepsilon) &\vdash (q, \times aaa, EE+, \varepsilon) \\&\vdash (q, aaa, EE \times E+, \varepsilon) \\&\vdash (q, aa, E \times E+, a) \\&\vdash (q, a, \times E+, aa) \\&\vdash (q, a, E+, aa \times) \\&\vdash (q, \varepsilon, +, aa \times a) \\&\vdash (q, \varepsilon, \varepsilon, aa \times a +).\end{aligned}$$

Deci, traducerea , cu memoria pushdown vidă , a secvenței $+ \times aaa$ este $aa \times a +$

5. DA

Rezumat: S-au prezentat etapele ce trebuie parcurse în vederea proiectării unui compilator, apoi metode de traducere a limbajelor. În partea a doua s-a explicat modul de efectuare a analizei lexicale.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Realizarea unor analizoare lexicale, ca primă etapă a construirii unui compilator

Teme Laborator/Seminar

1. Proiectați un analizor lexical care să recunoască constantele aritmetice și alfanumerice cuprinse între ghilimele.
2. Proiectați un analizor lexical care să recunoască identificatorii și cuvintele cheie

Rezumat: *Se proiectează analizoare lexicale care să recunoască elementele de bază ale unui limbaj de programare.*

Notății

Descriere Generală

Se prezintă principiile de funcționare ale analizei lexicale de tip ascendent și descendent, apoi se dau algoritmi generali de analiză sintactică

Obiective

- însușirea principiilor de funcționare ale analizei sintactice
- cunoașterea principalilor algoritmi de analiză sintactică, ce funcționează pe baza backtrackingului

Cuprins

- II.3.1. Formularea problemei
- II.3.2. Algoritmi generali de analiză sintactică
 - II.3.2.1. Analiză sintactică descendentă
 - II.3.2.2. Analiza sintactică ascendentă
 - II.3.2.3. Algoritmul Cocke -Younger – Kasami (CYK)

Conținut Curs

II.3.1. Formularea problemei

Fie $G = (N, \Sigma, P, S)$ o gramatică independentă de context (GIC) și $\alpha \in L(G)$; spunem că a fost efectuată analiza sintactică a secvenței α dacă a fost găsit cel puțin un arbore generator pentru α . Se lucrează cu analiză sintactică la stânga sau la dreapta. Presupunem că gramatica G are producțiile numerotate de la 1 la p și $\alpha, \beta \in (N \cup \Sigma)^*$. Scriem $\alpha \xRightarrow{i} \beta$ dacă se folosește producția cu numărul i iar neterminalul care derivează este cel mai din stânga. Similar scriem $\alpha \xRightarrow{i} \beta$ dacă se folosește producția cu numărul i iar neterminalul care derivează este cel mai din dreapta. Aceste notații se extind astfel:

- (1) dacă $\alpha \xRightarrow{i_1} \beta$ și $\beta \xRightarrow{i_2} \gamma$ atunci $\alpha \xRightarrow{i_1 i_2} \gamma$
- (2) dacă $\alpha \xRightarrow{i_1} \beta$ și $\beta \xRightarrow{i_2} \gamma$ atunci $\alpha \xRightarrow{i_1 i_2} \gamma$. Dacă $\pi = i_1 i_2 \dots i_n$, notăm

*Definirea
analizei
sintactice*

$\tilde{\pi} = i_n i_{n-1} \dots i_1$. Dacă $\alpha \in L(G)$, se numește analiză sintactică la stânga o secvență $\pi \in \{1, 2, \dots, p\}^*$ astfel încât $S \xRightarrow{\pi} \alpha$. Se numește analiză sintactică la dreapta a cuvântului α , o secvență $\tilde{\pi} \in \{1, 2, \dots, p\}^*$ astfel încât $S \xRightarrow{\tilde{\pi}} \alpha$.

Exemplul 3.1 Fie $G = (\{E, T, F\}, \{a, +, *, (,)\}, \{1, 2, 3, 4, 5, 6\}, E)$

- | | |
|--------------------------|------------------------|
| 1. $E \rightarrow E + T$ | 4. $T \rightarrow F$ |
| 2. $E \rightarrow T$ | 5. $F \rightarrow (E)$ |
| 3. $T \rightarrow T * F$ | 6. $F \rightarrow a$ |

și $\alpha = a + a$. Avem

$$E \xRightarrow{1} E+T \xRightarrow{2} T+T \xRightarrow{4} F+T \xRightarrow{6} a+T \xRightarrow{4} a+F \xRightarrow{6} a+a$$

deci analiza sintactică la stânga este $\pi = 124646$.

$$\text{La fel } E \xRightarrow{1} E+T \xRightarrow{4} E+F \xRightarrow{6} E+a \xRightarrow{2} T+a \xRightarrow{4} F+a \xRightarrow{6} a+a$$

deci analiza sintactică la dreapta este $\tilde{\pi} = 642641$.

Dispozitivele care furnizează analiza sintactică la stânga se numesc descendente (top - down), datorită modului în care construiesc arborele de derivare - de la rădăcină spre nodurile maxime. Analizorii sintactici la dreapta se numesc ascendenți (bottom-up), arborele fiind construit începând cu nodurile maxime și efectuând reduceri până se obține nodul rădăcină.

II.3.2. Algoritmi generali de analiză sintactică

Se pot construi scheme de traducere orientate de sintaxă sau traducătoare pushdown pentru a efectua analiza sintactică. Aceste dispozitive funcționează, însă, nedeterminist, ceea ce impune utilizarea backtracking-ului pentru a putea continua analiza în caz de blocaj. De aceea vom prezenta algoritmi care au backtracking-ul implementat sau vom utiliza gramatici care permit efectuarea analizei sintactice în mod determinist.

II.3.2.1. Analiză sintactică descendentă

Operațiile ce trebuie efectuate de un analizor sintactic bazat pe modelul traducătorului pushdown nedeterminist sunt: înlocuirea unui neterminal A din vârful stivei cu un șir ce reprezintă o parte dreaptă a A -producțiilor gramaticii și ștergerea unui terminal din vârful stivei în cazul în care el coincide

cu terminalul curent de la intrare, urmată de avansul cu o poziție la dreapta pe banda de intrare. Deci, un astfel de analizor sintactic va propune pentru fiecare neterminal alternative de rescriere a lui, operație numită expandare. Fiecare alternativă va reprezenta o structură posibilă pentru următoarele simboluri de intrare.

Dacă structura nu este regăsită în șirul de intrare, analizorul trebuie să propună o altă alternativă. Modelul de analizor prezentat anterior nu încearcă următoarea alternativă de expandare; vom prezenta mai târziu un analizor care are această proprietate. Pentru a prezenta principiul analizei sintactice cu reveniri să considerăm următoarea gramatică

$G = (N, \Sigma, P, S)$, $P = \{S \rightarrow aAd / aB, A \rightarrow b / c, B \rightarrow ccd / ddc\}$ și fie *accd* cuvântul de analizat .

Se pornește cu simbolul inițial ca nod rădăcină , se alege prima alternativă pentru *S* și prima alternativă pentru *A*. Deoarece secvența generată nu se regăsește în șirul de intrare se alege următoarea alternativă pentru *A*. Nici de data aceasta, secvența generată nu se regăsește în cea de intrare; deoarece nu mai sunt alternative pentru *A*, se alege următoarea alternativă pentru *S* și apoi prima pentru *B*.

Algoritmul pe care-l prezentăm utilizează două benzi pushdown (B_1 și B_2) și un indicator pe banda de intrare, ce precizează poziția curentă.

Algoritmul 3.1

Intrare: o GIC, fără recursie stângă $G = (N, \Sigma, P, S)$ și un cuvânt de intrare $w = a_1a_2...a_n, n \geq 0$. Presupunem că producțiile sunt numerotate cu $1, 2, ..., p$.

Ieșire : analiza sintactică la stânga a lui *w*, dacă aceasta există; în caz contrar se afișază „eroare“.

Metoda. Pentru fiecare neterminal *A*, dacă $A \rightarrow \alpha_1 / \alpha_2 ... / \alpha_k$ sunt toate *A*-producțiile, se notează cu A_i alternativa $A \rightarrow \alpha_i$. Algoritmul utilizează configurații de forma (s, i, α, β) unde

- (a) *s* este starea algoritmului și poate lua una din valorile
 - *q* - în cazul funcționării normale
 - *b* - în cazul reluării derivației (backtracking)

*Principiul
analizei
descendente*

- t - în cazul configurației de terminare

(b) $i \in \{1, 2, \dots, n+1\}$ este poziția curentă pe banda de intrare; al $n+1$ -lea simbol de intrare este \$, folosit pe post de delimitator la dreapta.

(c) $\alpha \in (\Sigma \cup \{A_i\})^*$ reprezintă conținutul benzii pushdown B_1 , bandă cu vârful spre dreapta, ce înregistrează alternativele încercate și simbolurile de intrare verificate.

(d) $\beta \in (N \cup \Sigma \cup \{ \$ \})^*$ reprezintă conținutul benzii pushdown B_2 , bandă cu vârful spre stânga. Simbolul din vârf reprezintă nodul activ al arborelui de derivare.

*Algoritm de
analiză
descendentă
bazat pe
backtracking*

Configurația inițială este $(q, 1, \varepsilon, S\$)$, unde S este simbolul inițial al gramaticii. Trecerea de la o configurație la alta se face conform următorilor pași:

Pasul 1 . *expandare*

$$(q, i, \alpha, A\beta) \vdash (q, i, \alpha A_1, \gamma_1\beta)$$

unde $(A \rightarrow \gamma_1) \in P$ și A_1 este prima alternativă pentru A . Acest pas corespunde extinderii arborelui parțial de derivare utilizând prima alternativă pentru neterminalul cel mai din stânga din arbore.

Pasul 2 . *Concordanță între simbolul de intrare și cel derivat*

$$(q, i, \alpha, a\beta) \vdash (q, i+1, \alpha a, \beta)$$

Se efectuează când $a_i = a$, $1 \leq i \leq n$ și constă în mutarea simbolului terminal din vârful benzii B_2 în vârful benzii B_1 și mutarea indicatorului pe banda de intrare.

Pasul 3 . *Obținerea unei configurații de acceptare*

$$(q, n+1, \alpha, \$) \vdash (t, n+1, \alpha, \varepsilon)$$

Ajungerea în această configurație spune că a fost epuizată banda de intrare și a fost găsit un arbore de derivare stângă. Acesta este găsit aplicând homomorfismul h lui α : $h(a) = \varepsilon$ pentru orice $a \in \Sigma$ și $h(A_i) = j$ dacă j este producția $(A \rightarrow \gamma) = A_i$.

Pasul 4 . *Simbolul de intrare nu coincide cu cel derivat*

$$(q, i, \alpha, a\beta) \vdash (b, i, \alpha, a\beta) \text{ dacă } a_i \neq a$$

Pasul 5 . *întoarcerea pe banda de intrare cu o poziție*

$$(b, i, \alpha a, \beta) \vdash (b, i-1, \alpha, a\beta) \quad \forall a \in \Sigma$$

Pasul 6 . încercarea alternativei următoare

$(b, i, \alpha A_j, \gamma_j \beta) \vdash (q, i, \alpha A_{j+1}, \gamma_{j+1} \beta)$ dacă γ_{j+1} este a $j+1$ -a alternativă pentru A

$\vdash$ fără continuare dacă $i=1, A=S$ și există numai j alternative pentru A ; nu există arbore de derivare pentru cuvântul de intrare
 $\vdash (b, i, \alpha, A\beta)$ în caz contrar; au fost epuizate toate alternativele pentru A și se renunță la expandarea acestui nod.

Algoritmul se execută astfel:

Pasul 1 : Se pornește cu configurația inițială și se calculează configurațiile succesive $C_0 \vdash C_1 \vdash \dots \vdash C_r$

Pasul 2 : Dacă ultima configurație este $(t, n+1, \gamma, \varepsilon)$ se emite $h(\gamma)$ și algoritmul se oprește, $h(\gamma)$ fiind primul arbore de derivare găsit. Dacă C_γ este o configurație fără continuare, se emite semnalul „eroare”.

II.3.2.2. Analiza sintactică ascendentă

Principiul de funcționare a unui analizor sintactic ascendent cu reveniri este pus în evidență în figura 3.2, utilizând gramatica cu regulile $S \rightarrow AB, A \rightarrow ab, B \rightarrow aba$ și cuvântul de intrare $w = ababa$

Se folosește pentru reducere prima regulă posibilă, $A \rightarrow ab$. Deoarece nu există nici-o regulă de producție care să aibă în membrul drept secvența AAa , se renunță la ultima reducere și se folosește regula $B \rightarrow aba$; în final secvența AB se reduce la S .

Principiul de funcționare a analizei ascendente

Algoritmul 3.2

Intrare: $G = (N, \Sigma, P, S)$ o GIC, fără cicluri (nu conține producții de forma $A \xRightarrow{+} A$), fără ε -producții, cu producțiile numerotate de la 1 la p și cuvântul de intrare $w = a_1 a_2 \dots a_n, n \geq 1$.

Ieșire : arborele de derivare la dreapta, dacă există, și „eroare” în caz contrar.

Metoda :

- se ordonează arbitrar regulile de producție;
- se lucrează cu configurații de forma (s, i, α, β) unde

a) s reprezintă starea algoritmului și poate lua una din valorile

- q : pentru funcționare normală,
- b : pentru mersul înapoi,
- t : pentru încheierea cu succes a analizei;

b) i indică simbolul curent de pe banda de intrare; pe poziția $n+1$ se află simbolul $\$$

c) $\alpha \in (N \cup \Sigma \cup \{\$\})^*$ este conținutul benzii pushdown B_1 (cu vârful spre dreapta) și reprezintă porțiunea în care a fost redusă secvența de intrare

d) $\beta \in \{1, 2, \dots, p, s\}^*$ reprezintă conținutul benzii pushdown B_2 (cu vârful spre stânga); această bandă ține evidența reducerilor efectuate pentru a se obține conținutul lui B_1 .

- configurația inițială este $(q, 1, \$, \epsilon)$.

Pasul 1 : *încercare de reducere*

$$(q, i, \alpha\beta, \gamma) \mapsto (q, i, \alpha A, j\gamma)$$

unde $A \rightarrow \beta$ este producția cu numărul j . Dacă s-a efectuat pasul 1 se rămâne la acest pas ; în caz contrar se merge la pasul 2 .

Pasul 2 : *trecere*

$$(q, i, \alpha, \gamma) \mapsto (q, i+1, \alpha a_i, s\gamma)$$

cu condiția $i \neq n+1$; apoi se merge la pasul 1.

Dacă $i = n+1$ se merge la pasul 3 . Simbolul s de pe banda pushdown B_2 arată că a avut loc o trecere a unui simbol de pe banda de intrare pe banda B_1 .

Pasul 3 : *acceptare*

$$(q, n+1, \$S, \gamma) \mapsto (t, n+1, \$S, \gamma)$$

Algoritmul se oprește și se emite $h(\gamma)$, unde h este homomorfismul definit prin $h(s) = \epsilon$ și $h(j) = j$ pentru $j \in \{1, 2, \dots, p\}$.

Dacă pasul 3 nu s-a executat se merge la pasul 4.

Pasul 4 : *începutul mersului înapoi*

$$(q, n+1, \alpha, \gamma) \mapsto (b, n+1, \alpha, \gamma)$$

cu condiția $\alpha \neq \$S$. Se merge la pasul 5.

Pasul 5 : *mersul înapoi*

$$(a) \quad (b, i, \alpha A, j\beta) \mapsto (q, i, \alpha' B, k\gamma)$$

unde $A \rightarrow \beta$ este producția cu numărul j ; următoarea producție, în ordinea

*Algoritm de
analiză
ascendentă
bazat pe
backtracking*

stabilită la început, al cărei membru drept este un sufix al lui $\alpha\beta$ este $B \rightarrow \beta'$ și are numărul k ; în acest caz $\alpha\beta = \alpha'\beta'$. După efectuarea acestui pas se merge la pasul 1.

$$(b) \quad (b, n+1, \alpha A, j\gamma) \vdash (b, n+1, \alpha\beta, \gamma)$$

unde $A \rightarrow \beta$ este producția cu numărul j și nu există nici-o altă alternativă de reducere pentru $\alpha\beta$. Apoi, se merge la pasul 5.

$$(c) \quad (b, i, \alpha A, j\gamma) \vdash (q, i+1, \alpha\beta a_i, s\gamma)$$

cu $i \neq n+1$; $A \rightarrow \beta$ este producția cu numărul j și nu există nici-o alternativă de reducere pentru $\alpha\beta$. Apoi, se merge la pasul 1.

$$(d) \quad (b, i, \alpha a, s\gamma) \vdash (b, i-1, \alpha, \gamma)$$

Deci, se renunță la trecerea lui a pe banda B_1 ; suntem cu o poziție în urmă pe banda de intrare și rămânem la pasul 5.

II.3.2.3. Algoritmul Cocke-Younger – Kasami (CYK)

Acest algoritm face trecerea de la analiza globală a lui $w = a_1 a_2 \dots a_n$ la analize locale; timpul de execuție este de ordinul n^3 iar memoria necesară este de ordinul n^2 , unde n este lungimea lui w . Ideea de bază este de a purta pe cuvântul de analizat două paranteze, una stângă notată cu i și alta dreaptă notată cu j , în așa fel încât fiecare porțiune din w poate fi inclusă între aceste paranteze și poate constitui obiect de analiză, independent de celelalte porțiuni ale cuvântului w . Fiecărei pereche (i, j) ce corespunde porțiunii $a_i \dots a_{i+j-1}$, îi atașăm o mulțime de simboluri neterminale $T[i, j]$ astfel încât $A \in T[i, j]$ dacă și numai dacă $A \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}$. Evident, dacă $S \in T[1, n]$ înseamnă că $S \xRightarrow{*} a_1 a_2 \dots a_n$ și deci $w \in L(G)$.

Am pus în evidență două probleme :

- construcția unei mulțimi $T[i, j]$ ce reprezintă structura locală a cuvântului $a_i a_{i+1} \dots a_{i+j-1}$;
- construcția unui algoritm care pentru cuvântul w să genereze o analiză pornind de la structurile locale.

Le analizăm pe rând.

- Pentru $i = 1, 2, \dots, n$ și $j = 1$ avem următoarele mulțimi

$$T[i,1] = \{A / A \xRightarrow{*} a_i\} \text{ iar pentru } j > 1 \text{ avem}$$

$$T[1,j] = \{A / A \xRightarrow{*} a_1 a_2 \dots a_j\}$$

$$T[i,j] = \{A / A \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}\}$$

.....

$$T[n-j+1,j] = \{A / A \xRightarrow{*} a_{n-j+1} \dots a_n\}$$

Mulțimile de mai sus se organizează într-un tabel astfel

$$T = \begin{bmatrix} T[1,1] & T[2,1] \dots & T[n-1,1] & T[n,1] \\ T[1,2] & T[2,2] \dots & T[n-1,2] & \\ & & & \\ T[1,n-1] & T[2,n-1] & & \\ T[1,n] & & & \end{bmatrix}$$

*Analiză
sintactică de
tip local*

Algoritmul se aplică pentru gramatici fără ε - producții și în formă normală Chomsky. În acest caz prima linie din tabelul T devine

$$T[i,1] = \{A / (A \rightarrow a_i) \in P\}, \quad i = 1, 2, \dots, n.$$

Considerăm cazul general al liniei j , adică

$$T[i,j] = \{A / A \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}\}. \text{ Dacă } A \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}, \text{ atunci avem și}$$

$$A \Rightarrow BC \xRightarrow{*} a_i a_{i+1} \dots a_{i+j-1}, \text{ adică există un } k \text{ astfel încât}$$

$$A \Rightarrow BC \xRightarrow{*} a_i a_{i+1} \dots a_{i+k-1} C \xRightarrow{*} a_i a_{i+1} \dots a_{i+k-1} a_{i+k} \dots a_{i+j-1}. \text{ Adică, pentru}$$

$1 \leq k < j$ mulțimea $T[i,j]$ poate fi definită în funcție de mulțimile deja construite $T[i,k]$ și $T[i+k,j-k]$; deci

$$T[i,j] = \{A / (A \rightarrow BC) \in P, B \in T[i,k], C \in T[i+k,j-k]\}.$$

Procedeul descris poate fi ușor transpus într-un algoritm.

b) După ce a fost construit tabelul T , analiza sintactică se poate efectua conform următorului algoritm

Algoritmul 3.3.

Intrare : o GIC în formă normală Chomsky, $G = (N, \Sigma, P, S)$ având producțiile numerotate de la 1 la p , un cuvânt de intrare $w = a_1 a_2 \dots a_n$ și tabelul T al mulțimilor $T[i,j]$.

Ieșire : analiza sintactică la stânga a lui w sau mesajul „eroare”.

Metoda: Se folosește rutina $gen(i,j,A)$ pentru a genera un arbore de derivare

stângă corespunzător derivației $A \xRightarrow{+} a_i a_{i+1} \dots a_{i+j-1}$. Rutina este definită astfel :

1) Dacă $j = 1$ și a m -a producție din P este $A \rightarrow a_i$, atunci se emite numărul m .

2) Dacă $j > 1$ și k este cel mai mic număr întreg, $1 \leq k < j$, astfel încât pentru $B \in T[i, k]$, $C \in T[i + k, j - k]$ și $A \rightarrow BC$ este producția cu numărul m , atunci se emite m și se execută $gen(i, k, B)$ și apoi $gen(i + k, j - k, C)$.

Dacă $S \in T[1, n]$ se execută $gen(1, n, S)$ iar dacă $S \notin T[1, n]$ se emite mesajul „eroare“.

*Algoritmul
analizei locale*

Teme Curs

Teste Autoevaluare

1. Fie gramatica $G = (\{E, T, F, E', T'\}, \{+, *, a, (,)\}, E, P)$ unde P constă din producțiile

1. $E \rightarrow TE'$
2. $E' \rightarrow +TE'$
3. $E' \rightarrow \varepsilon$
4. $T \rightarrow FT'$
5. $T' \rightarrow *FT'$
6. $T' \rightarrow \varepsilon$
7. $F \rightarrow a$
8. $F \rightarrow (E)$

Efectuați analiza sintactică descendentă a cuvântului a 3 puncte

2. Fie $G = (\{E, T, F\}, \{a, +, *, (,)\}, \{1, 2, 3, 4, 5, 6\}, E)$

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow a$

Se cere analiza sintactică ascendentă a cuvântului $w = a * a$ 3 puncte

3. Considerăm gramatica $G = (N, \Sigma, P, S)$ cu producțiile

1. $S \rightarrow AA$
2. $S \rightarrow AS$
3. $S \rightarrow b$
4. $A \rightarrow SA$
5. $A \rightarrow AS$
6. $A \rightarrow a$

și cuvântul de intrare $w = abaab$. Se cere

- a) Tabelul de analiză CYK..... 1.5 puncte
- b) Analiza sintactică de tip CYK..... 1.5 puncte

Oficiu..... 1 punct

Răspunsuri

1. $(q, 1, \varepsilon, E \$) \vdash (q, 1, E_1, TE' \$) \quad (\text{cazul 1})$
 $\vdash (q, 1, E_1 T_1, FT' E' \$) \quad (\text{cazul 1})$
 $\vdash (q, 1, E_1 T_1 F_1, aT' E' \$) \quad (\text{cazul 1})$
 $\vdash (q, 2, E_1 T_1 F_1 a, T' E' \$) \quad (\text{cazul 2})$
 $\vdash (q, 2, E_1 T_1 F_1 a T'_1, * FT' E' \$) \quad (\text{cazul 1})$
 $\vdash (b, 2, E_1 T_1 F_1 a T'_1, * FT' E' \$) \quad (\text{cazul 4})$
 $\vdash (q, 2, E_1 T_1 F_1 a T'_2, E' \$) \quad (\text{cazul 6a})$
 $\vdash (q, 2, E_1 T_1 F_1 a T'_2 E'_2, + TE' \$) \quad (\text{cazul 1})$
 $\vdash (b, 2, E_1 T_1 F_1 a T'_2 E'_1, + TE' \$) \quad (\text{cazul 4})$
 $\vdash (q, 2, E_1 T_1 F_1 a T'_2 E'_2, \$) \quad (\text{cazul 6a})$
 $\vdash (t, 2, E_1 T_1 F_1 a T'_2 E'_2, \varepsilon) \quad (\text{cazul 3})$

Deci, analiza sintactică la stânga a cuvântului a este $h(E_1 T_1 F_1 a T'_2 E'_2) = 14763$.

2. $(q, 1, \$, \varepsilon) \vdash (q, 2, \$a, s)$
 $\vdash (q, 2, \$F, 6s)$
 $\vdash (q, 2, \$T, 46s)$
 $\vdash (q, 2, \$E, 246s)$
 $\vdash (q, 3, \$E^*, s246s)$
 $\vdash (q, 4, \$E^* a, ss246s)$
 $\vdash (q, 4, \$E^* F, 6ss246s)$
 $\vdash (q, 4, \$E^* T, 46ss246s)$
 $\vdash (q, 4, \$E^* E, 246ss246s)$
 $\vdash (b, 4, \$E^* E, 246ss246s)$
 $\vdash (b, 4, \$E^* T, 46ss246s)$
 $\vdash (b, 4, \$E^* F, 6ss246s)$
 $\vdash (b, 4, \$E^* a, ss246s)$
 $\vdash (b, 3, \$E^*, s246s)$

$$\begin{aligned} &\vdash (b, 2, \$E, 246s) \\ &\vdash (q, 3, \$T^*, s46s) \\ &\vdash (q, 4, \$T^* a, ss46s) \\ &\vdash (q, 4, \$T^* F, 6ss46s) \\ &\vdash (q, 4, \$T, 36ss46s) \\ &\vdash (q, 4, \$E, 236ss46s) \\ &\vdash (t, 4, \$E, 236ss46s) \end{aligned}$$

și $h(\gamma) = h(236ss46s) = 23646$.

3. a)

A	S	A	A	S
A,S	A	S	A,S	
A,S	S	A,S		
A,S	A,S			
A,S				

b)

Deoarece $S \in T[1,5]$, $w \in L(G)$. Pentru a obține arborele de derivare chemăm rutina $gen(1,5,S)$. Găsim $A \in T[1,1]$, $A \in T[2,4]$ și $(S \rightarrow AA) \in P$. Emitem 1 (numărul producției $A \rightarrow AA$) și chemăm $gen(1,1,A)$ și $gen(2,4,A)$. $gen(1,1,A)$ dă producția 6. Deoarece $S \in T[2,1]$, $A \in T[3,3]$ și $A \rightarrow SA$ este producția cu numărul 4, $gen(2,4,A)$ emite 4 și cheamă $gen(2,1,S)$ urmată de $gen(3,3,A)$. Continuând în acest fel se obține 164356263; aceste secvențe de reguli dau un arbore de derivare stângă pentru w (notăm faptul că gramatica este ambiguă).

Rezumat: S-au prezentat algoritmi generali de analiză sintactică

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Implementarea unui algoritm de analiză sintactică din cei trei prezențați.

Teme Laborator/Seminar

1. Implementați unul din algoritmi generali de analiză sintactică

Rezumat: <i>implementarea unui algoritm de analiză sintactică</i>
--

Notății

Descriere Generală

Se prezintă algoritmi de analiză sintactică determinați care

- iau decizii privind un anumit număr de simboluri spre dreapta și parcurg secvența de analizat o singură dată
- funcționează pe baza unor relații între simbolurile gramaticii (numite de precedentă) care permit determinarea părții reductibile

Obiective

- cunoașterea a trei algoritmi determinați de analiză sintactică
- cunoașterea unor posibilități de eficientizare a analizei deterministe

Cuprins

II.3.3. Analiza sintactică de tip LL

II. 3.4. Analiza sintactică de tip LR

II.3.5. Analiza sintactică de tip precedentă

II. 3.5.1. Analiza sintactică a gramaticilor de precedentă simplă

II.3.5.2. Analiza sintactică a gramaticilor de precedentă slabă

Conținut Curs

II.3.3. Analiza sintactică de tip LL

Dăm un algoritm de analiză sintactică pentru gramatici de tip $LL(k), k \geq 1$.

Definiția 3.1. Fie $G = (N, \Sigma, P, S)$ o GIC. Pentru fiecare $A \in N$ și $L \subseteq \Sigma^k$ definim funcțiile $T_{A,L}$, numite tabele $LL(k)$ asociate cu A și L , astfel :

- (1) $T_{A,L}(u) = \text{"eroare"}$ dacă nu există nici-o producție $A \rightarrow \alpha$ astfel încât $u \in PRIM_k(\alpha) \oplus_k L$.
- (2) $T_{A,L}(u) = (A \rightarrow \alpha, \langle Y_1, Y_2, \dots, Y_m \rangle)$ dacă există o unică producție $A \rightarrow \alpha$ astfel încât $u \in PRIM_k(\alpha) \oplus_k L$.

Definiția
tabelor $T_{A,L}$

Dacă $\alpha = x_0 B_1 x_1 B_2 \dots B_m x_m, m \geq 0, B_i \in N$ și $x_i \in \Sigma^*$, atunci $Y_i = PRIM_k(x_i B_{i+1} x_{i+1} \dots B_m x_m) \oplus_k L$. Y_i se numește mulțimea local următoare a lui B_i . Dacă $m = 0$ atunci $T_{A,L}(u) = (A \rightarrow \alpha, \emptyset)$.

(3) $T_{A,L}(u)$ este nedefinită dacă există cel puțin două producții $A \rightarrow \alpha_1 / \alpha_2 / \dots / \alpha_n$ astfel încât $u \in PRIM_k(\alpha_i) \oplus_k L, 1 \leq i \leq n, n \geq 2$. Această situație nu apare dacă G este $LL(k)$.

Intuitiv, $T_{A,L}(u)$ eroare spune că nu este posibilă nici-o derivație de forma $Ax \xRightarrow{+} uv$ pentru nici-un $x \in L$ și $v \in \Sigma^*$. Când $T_{A,L}(u) = (A \rightarrow \alpha, \langle Y_1, Y_2, \dots, Y_m \rangle)$ există exact o producție $A \rightarrow \alpha$ care poate fi utilizată la primul pas al derivației $Ax \xRightarrow{+} uv$ pentru orice $x \in L$ și $v \in \Sigma^*$. Fiecare mulțime Y_i dă toate prefixele posibile de lungime cel mult k , formate din terminale, care pot urma un șir derivat din B_i când utilizăm producția $A \rightarrow \alpha$, unde $\alpha = x_0 B_1 x_1 B_2 \dots B_m x_m$ în orice derivație de forma $Ax \xRightarrow{+} \alpha x \xRightarrow{*} uv$, cu $x \in L$.

Algoritmul 3.4. - de construire a tabelor LL

Intrare : o GIC $G = (N, \Sigma, P, S)$ de tip $LL(k)$

Ieșire : mulțimea T a tabelor $LL(k)$

Metoda :

Pasul 1. Se construiește $T_0 = T_{S,\varepsilon}$ și se inițializează $T = \{T_0\}$.

Pasul 2. Pentru fiecare tabel $LL(k)$ T cu intrarea

$T(u) = (A \rightarrow x_0 B_1 x_1 \dots B_m x_m, \langle Y_1, Y_2, \dots, Y_m \rangle)$ se adaugă la T tabelul T_{B_i, Y_i} pentru $1 \leq i \leq m$ dacă el nu există deja în T .

Pasul 3. Se repetă pasul (2) până când nu se mai poate adăuga nici-un tabel la T .

Construirea
tabelor $T_{A,L}$

Analiza sintactică, folosind mulțimea de tabele $LL(k)$ este dată de algoritmul următor.

Algoritmul 3.5.

Intrare : gramatica $G = (N, \Sigma, P, S)$ de tip $LL(k)$ și T

Ieșire : tabelul M de analiză sintactică

Metoda : M este definit pe $(T \cup \Sigma \cup \{S\}) \times \Sigma^{*k}$ astfel :

Pasul 1. dacă $A \rightarrow x_0 B_1 x_1 B_2 \dots B_m x_m$ este producția cu numărul i , $T_{A,L} \in T$ și $T_{A,L}(u) = (A \rightarrow x_0 B_1 x_1 B_2 x_2 \dots B_m x_m, < Y_1, Y_2, \dots, Y_m >)$ atunci M se definește astfel $M(T_{A,L}, u) = (x_0 T_{B_1, Y_1} x_1 \dots T_{B_m, Y_m} x_m, i)$.

Pasul 2. $M(a, av) = \text{reducere}$ pentru orice $v \in \Sigma^{*(k-1)}$.

Pasul 3. $M(\$, \varepsilon) = \text{acceptare}$.

Pasul 4. $M(X, u) = \text{eroare}$, în alte cazuri.

Analiza
sintactică
 $LL(k)$

Configurația inițială este $(T_0 \$, w, \varepsilon)$ iar cea finală este $(\$, \varepsilon, \pi)$ unde w este cuvântul de analizat iar π este analiza sintactică la stânga a lui w .

Analiza sintactică a gramaticilor $LL(k)$ se simplifică în cazul $k = 1$.

Definiția 3.7. O gramatică $G = (N, \Sigma, P, S)$ este $LL(1)$ dacă pentru orice neterminat A și orice două A -producții distincte $A \rightarrow \alpha$ și $A \rightarrow \beta$ este verificată condiția

$$PRIM_1(\alpha URM_1(A)) \cap PRIM_1(\beta URM_1(A)) = \emptyset.$$

O formulare mai simplă a condiției de mai sus este: pentru orice reguli $A \rightarrow \alpha_1 / \alpha_2 / \dots / \alpha_n$:

1. $PRIM_1(\alpha_i) \cap PRIM_1(\alpha_j) = \emptyset$ pentru $i \neq j$
2. dacă $\alpha_i \xRightarrow{*} \varepsilon$ atunci $PRIM_1(\alpha_j) \cap URM_1(A) = \emptyset$ pentru $i \neq j$

Analiza sitactică se face cu ajutorul funcției

$$M(A, a) = \begin{cases} \text{reducere} & \text{dacă } A = a \in \Sigma \\ \text{acceptare} & \text{dacă } A = \$ \text{ si } a = \varepsilon \\ (\alpha, i) & \text{dacă } a \in PRIM_1(\alpha) \text{ și } (A \rightarrow \alpha) = i \in P \\ (\alpha, i) & \text{dacă } a \in URM_1(A) \text{ și } (A \rightarrow \alpha) = i \in P \\ & \text{iar } \varepsilon \in PRIM_1(\alpha) \\ \text{eroare} & \text{în alte cazuri} \end{cases}$$

Analiza
 $LL(1)$

II. 3.4. Analiza sintactică de tip LR

Descriem un algoritm de analiză sintactică pentru gramatici $LR(k)$, care furnizează o analiză la dreapta privind k simboluri în față și funcționând în mod determinist.

Definiția 3.8. Fie $G = (N, \Sigma, P, S)$ o GIC și S colecția canonică de mulțimi de linii $LR(k)$. $T(A)$, tabelul $LR(k)$ asociat mulțimii $A \in S$, este o pereche de funcții $\langle f, g \rangle$ unde f este funcția de trecere iar g este funcția GOTO:

(1) funcția f este definită astfel

- (a) $f(u) = \text{trecere}$ dacă $[A \rightarrow \beta_1 \beta_2 v] \in A$, $\beta_2 \neq \varepsilon$ și $u \in EFF_k(\beta_2 v)$
- (b) $f(u) = i$ dacă $[A \rightarrow \beta., u] \in A$ și $A \rightarrow \beta$ este producția cu numărul i .
- (c) $f(\varepsilon) = \text{acceptare}$ dacă $[S' \rightarrow S., \varepsilon] \in A$
- (d) $f(u) = \text{eroare}$ în alte cazuri.

(2) funcția g determină următorul tabel ce va fi folosit în analiză; ea pune în corespondență unui element din $N \cup \Sigma$ un tabel $LR(k)$ sau eroare:

$$g(x) = GOTO(A, x) \text{ dacă } GOTO(A, x) \neq \emptyset$$

$$g(x) = \text{eroare} \text{ dacă } GOTO(A, x) = \emptyset.$$

Algoritmul 3.6.

Intrare: Mulțimea T de tabele $LR(k)$ corespunzătoare gramaticii $G = (N, \Sigma, P, S)$ cu tabelul inițial $T_0 = T(A_0)$, $A_0 = V_k^G(\varepsilon)$ și cuvântul inițial $w \in \Sigma^*$.

Ieșire: Analiza sintactică la dreapta dacă $w \in L(G)$ și „eroare” în caz contrar.

Metoda: Algoritmul lucrează cu o bandă de intrare, o bandă de ieșire și una pushdown. Configurația inițială este (T_0, w, ε) . Se execută pașii (1) și (2) până când se obține un răspuns de acceptare sau de eroare.

Pasul 1. Se determină $u = PRIM_k$ (porțiunea din banda de intrare rămasă de citit).

Pasul 2. Fie $T_i = \langle f, g \rangle$ linia înscrisă în vârful benzii pushdown;

(a) Dacă $f(u) = \text{trecere}$, atunci primul simbol disponibil x de pe banda de intrare se trece în vârful benzii pushdown. Calculează $g(x) = T_j$ și înscrie T_j

Funcții LR

în vârful benzii pushdown și apoi treci la pasul (1).

(b) Dacă $f(u) = i$ și $A \rightarrow \alpha$ este regula i din P , atunci șterge $2|\alpha|$ simboluri de pe banda pushdown și înscrie i pe banda de ieșire. Fie $T_j = \langle f_j, g_j \rangle$ tabelul rămas în vârful benzii pushdown; determină $g_j(A) = T'$, înscrie pe banda pushdown pe AT' și mergi la pasul (1). Dacă $g_j(A) = \text{eroare}$, oprește algoritmul și cheamă, eventual, o rutină de tratare a erorii.

(c) Dacă $f(u) = \text{eroare}$, oprește algoritmul și cheamă, eventual, o rutină de tratare a erorii.

(d) Dacă $f(u) = \text{acceptare}$, oprește algoritmul și emite $\tilde{\pi}$, unde π este secvența de pe banda de ieșire.

În locul analizei $LR(1)$, se preferă analiza $LALR(1)$, bazată pe gramatici $LALR$ (în engleză lookahead LR grammars). Ea este mai avantajoasă deoarece tabelul $LALR(1)$ este mai mic în comparație cu cel $LR(1)$. De asemenea, gramaticile $LALR(1)$ acoperă o clasă mare de limbaje ce includ practic toate construcțiile sintactice folosite în limbajele de programare cunoscute.

Fie $A_i = \{[A \rightarrow \alpha., a]\}$ și $A_j = \{[A \rightarrow \alpha., b]\}$ două elemente ale colecției canonice de linii $LR(1)$. Liniile din cele două mulțimi au același nucleu, dar diferă prin șirul de anticipare (a și respectiv b). După reducere, în funcție de elementul din vârful stivei și de simbolul de intrare, se ajunge în stări diferite.

Dacă se renunță la verificarea simbolului de intrare, cele două mulțimi de linii LR , A_i și A_j sunt echivalente; putem să le înlocuim cu o nouă mulțime $A_{i,j} = \{[A \rightarrow \alpha., a/b]\}$. Mai general, două mulțimi din colecția canonică $LR(1)$ pot fuziona dacă au aceleași nuclee ale elementelor componente. Repetând această operație până când nu mai există stări cu nuclee identice, ajungem la un tabel $LR(1)$. Deoarece funcția $GOTO$ depinde doar de nucleu, toate referirile la T_i și T_j din definiția funcției g vor fi înlocuite prin $T_{i,j}$.

II.3.5. Analiza sintactică de tip precedență

II. 3.5.1. Analiza sintactică a gramaticilor de precedență simplă

Plecând de la relațiile de precedență trebuie stabilit mai întâi când are loc o trecere a unui simbol de pe banda de intrare pe banda pushdown și când are loc o operație de reducere.

Analiza LR

Analiza
 $LALR(1)$

Algoritmul 3.7

Intrare: o gramatică de precedență simplă $G = (N, \Sigma, P, S)$ cu producțiile numerotate de la 1 la p .

Ieșire: funcțiile f și g .

Metoda: Simbolul $\$ \notin (N \cup \Sigma)$ marchează sfârșitul benzii pushdown și ultimul simbol de pe banda de intrare. Funcția f depinde de simbolul din vârful benzii pushdown și de cel curent de pe banda de intrare; ea este definită astfel:

- a) $f(X, a) = \text{trecere}$, dacă $X < a$ sau $X = a$
- b) $f(X, a) = \text{reducere}$, dacă $X > a$
- c) $f(S, \$) = \text{acceptare}$
- d) $f(X, a) = \text{eroare}$ în alte cazuri.

Regula c) este prioritară față de regulile a) și b) când $X = S$ și $a = \$$

Funcția g nu depinde de conținutul benzii de intrare; ea depinde numai de simbolurile din vârful benzii pushdown ce formează partea reductibilă, plus un simbol la stânga:

- a') $g(X_{k+1} X_k \cdots X_1, \varepsilon) = i$ dacă $X_{k+1} < X_k, X_{j+1} = X_j$ pentru $1 \leq j < k$ și $A \rightarrow X_k \cdots X_1$ este producția cu numărul i .
- b') $g(\alpha, \varepsilon) = \text{eroare}$ în alte cazuri.

Deci f este definită pe $(N \cup \Sigma \cup \{\$\}) \times (\Sigma \cup \{\$\})$ iar g este definită pe $(N \cup \Sigma \cup \{\$\})^*$. Algoritmul de analiză folosește o bandă pushdown pe care $\$$ arată capătul din stânga, o bandă de intrare pe care $\$$ arată capătul din dreapta și o bandă de ieșire. Deci, o configurație este de forma: $(\$X_1 \cdots X_m, a_1 \cdots a_q \$, i_1 \cdots i_n)$ unde:

- $\$X_1 \cdots X_m$ reprezintă conținutul benzii pushdown cu X_m în vârf;
- $a_1 \cdots a_q$ este porțiunea din banda de intrare rămasă de citit, iar a_1 este simbolul curent;
- $i_1 \cdots i_n$ indică secvența regulilor de producție utilizate pentru a reduce cuvântul inițial la $X_1 \cdots X_m a_1 \cdots a_q$.

Pentru ca legătura dintre transformarea configurațiilor și funcțiile f și g să fie mai clară, vom considera că funcțiile f și g sunt extinse astfel:

Funcțiile de
precedență
simplă

$$f : V^* \times (\Sigma \cup \{\$ \})^* \rightarrow \{trecere, reducere, eroare, acceptare\}$$

$$g : V^* \times (\Sigma \cup \{\$ \})^* \rightarrow \{1, 2, \dots, p, eroare\},$$

unde $V = N \cup \Sigma \cup \{\$ \}$.

Trecerea de la o configurație la alta se definește astfel:

- (1) dacă $f(\alpha, aw) = trecere$ atunci $(\alpha, aw, \pi) \vdash (\alpha a, w, \pi)$;
- (2) dacă $f(\alpha, w) = reducere$, $g(\alpha, w) = i$ și $A \rightarrow \beta$ este producția cu numărul i atunci $(\alpha, w, \pi) \vdash (\gamma A, w, \pi i)$, cu $\alpha = \gamma \beta$;
- (3) dacă $f(\alpha, w) = acceptare$ atunci $(\alpha, w, \pi) \vdash acceptare$
- (4) $(\alpha, w, \pi) \vdash eroare$ în alte cazuri.

Dacă $(\$, w \$, \varepsilon) \vdash^* (\$, S \$, \pi) \vdash acceptare$, atunci $S \xrightarrow[\alpha]{\tilde{\pi}} w$.

Exemplul 3.2. Considerăm gramatica din Lecția 5. Funcțiile f și g se deduc imediat urmărind tabelul prezentat în acest exemplu. Să efectuăm analiza sintactică a cuvântului $w = babaacac$. Avem:

$$\begin{aligned}
 (\$, babaacac \$, \varepsilon) &\vdash (\$, abaacac \$, \varepsilon) \\
 &\vdash (\$, ba, baacac \$, \varepsilon) \\
 &\vdash (\$, bA, baacac \$, 3) \\
 &\vdash (\$, bAb, aacac \$, 3) \\
 &\vdash (\$, bAba, acac \$, 3) \\
 &\vdash (\$, bAbA, acac \$, 33) \\
 &\vdash (\$, bAbAa, cac \$, 33) \\
 &\vdash (\$, bAbAA, cac \$, 333) \\
 &\vdash (\$, bAbAAc, ac \$, 333) \\
 &\vdash (\$, bAS, ac \$, 3331) \\
 &\vdash (\$, bASa, c \$, 3331) \\
 &\vdash (\$, bAA, c \$, 33312) \\
 &\vdash (\$, bAAc, \$, 33312) \\
 &\vdash (\$, S \$, 333121) \\
 &\vdash acceptare
 \end{aligned}$$

deci $\pi = 333121$ și $S \xrightarrow[\alpha]{\tilde{\pi}} w$.

Analiza de
precedență
simplă

II.3.5.2. Analiza sintactică a gramaticilor de precedență slabă

Următoarea teoremă arată cum se alege regula folosită pentru reducere.

Teorema 3.1. Fie $G = (N, \Sigma, P, S)$ o gramatică de precedență slabă, $(B \rightarrow \beta) \in P$ și $\$S\$ \xRightarrow{*} \gamma Cw \Rightarrow \delta X\beta w$. Dacă $(A \rightarrow \alpha X\beta) \in P$, atunci ultima producție folosită în derivarea de mai sus nu este $B \rightarrow \beta$.

Demonstrație. Presupunem că ultima producție folosită a fost $B \rightarrow \beta$. Atunci, avem $\$S\$ \xRightarrow{*} \gamma Bw = \delta XBw \Rightarrow \delta X\beta w$. Conform teoremei 7.6 din I, avem $X < B$ sau $X \doteq B$, ceea ce este în contradicție cu definiția gramaticilor de precedență slabă. $\square$

Din această teoremă rezultă că într-o gramatică de precedență slabă, odată izolată limita dreaptă a părții reductibile, reducerea este determinată de producția a cărei cea mai lungă parte dreaptă filtrează vârful benzii pushdown. În cazul când condiția (1) din definiția precedenței simple (definiția 7.17 din I) nu este satisfăcută, gramatica poate fi modificată astfel încât să fie eliminat conflictul. De exemplu, dacă $X \doteq Y$ și $X > Y$ atunci $X \doteq Y$ înseamnă că există producția $A \rightarrow \alpha XY\beta$. Eliminăm relația $X \doteq Y$ astfel:

- înlocuim pe X cu un nou neterminal B ; deci producția $A \rightarrow \alpha XY\beta$ devine $A \rightarrow \alpha BY\beta$;
- adăugăm la producțiile existente producția $B \rightarrow X$.

Modificările înlătură conflictele, dar trebuie verificat dacă se mai păstrează proprietatea de unic invertibilitate.

Exemplul 3.3. Fie gramatica cu producțiile:

$$E \rightarrow a/(E)/a(F)$$

$$F \rightarrow E/F, E$$

Avem $E \doteq$ și $E >$. Dacă înlocuim $E \rightarrow (E)$ cu $E \rightarrow (A)$ și adăugăm $A \rightarrow E$, conflictul dispare. Dar, noua gramatică:

Realizarea
precedenței
slabe

$$E \rightarrow a/(A)/a(F)$$

$$F \rightarrow E/F, E$$

$$A \rightarrow E$$

nu mai este unic-invertibilă datorită producțiilor $F \rightarrow E$ și $A \rightarrow E$. În cazul gramaticii de mai sus, o soluție mai bună constă în evitarea relației $E >)$,

datorată lui $F \doteq)$, astfel:

$$E \rightarrow a/(E)/a(E)/a(F, E)$$

$$F \rightarrow E/F, E$$

Acum avem peste tot $E \doteq)$, dar mai sunt și alte conflicte ce trebuie eliminate:

$(\doteq E$ și $(\prec E$ sau $(\doteq F$ și $(\prec F$.

Teme Curs

Teste Autoevaluare

1. Considerăm gramatica cu producțiile

1. $S \rightarrow aAaa$

2. $S \rightarrow bAba$

3. $A \rightarrow b$

4. $A \rightarrow \varepsilon$

Știind că este $LL(2)$, să se efectueze analiza sintactică a cuvântului
 $w = bbba$ 5 puncte

2. Fie gramatica $G = (\{S, A, B, C\}, \{a, b, c, d, e, f\}, P, S)$ unde producțiile sunt :

1. $S \rightarrow aAbc$

2. $A \rightarrow BA$

3. $A \rightarrow \varepsilon$

4. $B \rightarrow dcCe$

5. $B \rightarrow fa$

6. $C \rightarrow cC$

7. $C \rightarrow \varepsilon$

Să se verifice că este $LL(1)$ și să se efectueze analiza sintactică a cuvântului
 $w = abc$ 4 puncte

Oficiu..... 1 punct

Răspunsuri

1. a) Construim mai întâi mulțimea de tabele T.
..... 3 puncte

Deoarece $(S \rightarrow aAaa) \in P$ calculăm $PRIM_k(aAaa) \oplus_2 \{\varepsilon\} = \{aa, ab\}$;

deoarece $(S \rightarrow bAba) \in P$ calculăm $PRIM_2(bAba) \oplus_2 \{\varepsilon\} = \{bb\}$.

Atunci $T_0(aa) = (S \rightarrow aAaa, Y)$, unde $Y = PRIM_2(aa) \oplus_2 \{\varepsilon\} = \{aa\}$.

Continuând în acest mod obținem tabelul $T_0 = T_{S, \{\varepsilon\}}$

u	Producția	mulțimile următoare
aa	$S \rightarrow aAaa$	$\{aa\}$
ab	$S \rightarrow aAaa$	$\{aa\}$
bb	$S \rightarrow bAba$	$\{ba\}$

Deoarece $T_0(aa) = (S \rightarrow aAaa, \{aa\})$ adăugăm la T tabelul $T_1 = T_{A, \{aa\}}$

u	Producția	mulțimile următoare
ba	$A \rightarrow b$	$\emptyset$
aa	$A \rightarrow \varepsilon$	$\emptyset$

Deoarece $T_0(bb) = (S \rightarrow bAba, \{ba\})$, adăugăm la T tabelul $T_2 = T_{A, \{ba\}}$

u	Producția	mulțimile următoare
ba	$A \rightarrow \varepsilon$	$\emptyset$
bb	$A \rightarrow b$	$\emptyset$

- b) Construirea tabelului M 1.5 puncte

	aa	ab	a	ba	bb	b	ε
T_0	$aT_1aa, 1$	$aT_1aa, 1$			$bT_2ba, 2$		
T_1	$\varepsilon, 4$			$b, 3$			
T_2				$\varepsilon, 4$	$b, 3$		
a	R	R	R				
b				R	R	R	
\$							A

unde R înseamnă “reducere” iar A “acceptare”.

c) Analiza..... 0.5 puncte

$(T_0 \$, bbba, \varepsilon) \vdash (bT_2ba \$, bbba, 2) \vdash (T_2ba \$, bba, 2) \vdash (bba \$, bba, 23) \vdash$
 $(ba \$, ba, 23) \vdash (a \$, a, 23) \vdash (\$, \varepsilon, 23)$

deci $\pi = 23$.

2. a) verificarea condiției..... 1 punct

Deoarece $PRIM_1(aAbc) = \{a\}$, $PRIM_1(BA) = \{d, f\}$, $PRIM_1(\varepsilon) = \{\varepsilon\}$,
 $PRIM_1(dcCe) = \{d\}$, $PRIM_1(fa) = \{f\}$, $PRIM_1(cC) = \{c\}$ și
 $URM_1(A) = \{b\}$, $URM_1(B) = \{b\}$, $URM_1(C) = \{e\}$ rezultă că gramatica
 este $LL(1)$.

b) tabelul 2 puncte

	a	b	c	d	e	f	ε
S	(aAbc,1)						
A		(ε ,3)		(BA,2)		(BA,2)	
B				(dcCe,4)		(fa,5)	
C			(cC,6)		(ε ,7)		
a	R						
b		R					
c			R				
d				R			
e					R		
f						R	
\$							A

c) analiza 1 punct

$(S \$, abc, \varepsilon) \vdash (aAbc \$, abc, 1)$
 $\vdash (Abc \$, bc, 1)$
 $\vdash (bc \$, bc, 13)$

$\vdash (c\$,c,13)$
 $\vdash (\$, \epsilon, 13)$
 $\vdash$ acceptare.

Rezumat: *S-au descris algoritmi determinați de analiza sintactică ascendentă și descendentă*

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se vor implementa doi algoritmi de analiză sintactică corespunzător celor două tipuri: ascendent și descendent

Teme Laborator/Seminar

1. Implementați analiza sintactică de tip precedență simplă
2. Implementați analiza sintactică LL(1)

Rezumat: *se implementează algoritmi de tip precedență și LL(1)*

Notatii

Descriere Generală

Se definesc tipurile de atribute și rolul lor în specificarea semanticii unui limbaj de programare. Se arată modul de utilizare a gramaticilor L-atributate în specificarea semanticii iar în final se dă un exemplu pentru un limbaj concret.

Obiective

- cunoașterea tipurilor de atribute și a semnificației lor
- cunoașterea rolului gramaticilor atributate în specificarea semanticii unui limbaj de programare

Cuprins

II.4.1. Specificarea semanticii limbajelor

II.4.1.1. Specificarea semanticii folosind gramatici cu atribute

II.4.1.2. Specificarea semanticii folosind gramatici L-atributate

II.4.2. Model de analiză semantică

Conținut Curs

II.4.1. Specificarea semanticii limbajelor

Analiza lexicală și cea sintactică extrag structura programului sursă. Analiza semantică completează structura sintactică cu valorile atributelor asociate componentelor acestei structuri. Pe baza valorilor atributelor, se realizează, pe considerente semantice, corectitudinea programului sursă și se alcătuiește codul intermediar echivalent. Rezolvarea acestor două sarcini necesită, de obicei, mai multe parcurgeri ale arborelui de derivare. Numărul acestor parcurgeri poate fi fix sau variabil în funcție de structura programului analizat. Prima situație este preferabilă și, de obicei, prin modificări ale definiției limbajului sau impunerea unor restricții, numărul parcurgerilor poate fi limitat, putându-se ajunge la situația ideală: o singură parcurgere.

Numărul de parcurgeri rezultă din ordinea de evaluare a atributelor și este

Generalități

influențat de resursele de memorie disponibile. Pentru a economisi spațiu de memorie, arborele de derivare nu este explicit construit; în locul lui se preferă arborele sintactic sau forme echivalente acestuia. În cazul analizei într-o singură trecere, informațiile privind structura programului sunt concentrate în șirul acțiunilor analizorului sintactic și arborele devine inutil. În practică, analiza semantică se desfășoară în paralel cu cea sintactică, asociind acțiunilor analizorului sintactic acțiuni referitoare la atributele componentelor sintactice.

Knuth a propus o metodă de specificare a semanticii limbajelor de programare, definind înțelesuri ale cuvintelor unui limbaj independent de context cu ajutorul atributelor asociate simbolurilor din arborele de derivare.

II.4.1.1. Specificarea semanticii folosind gramatici cu atribute

Definiția 4.1. O gramatică cu atribute (GA) constă din:

a) o gramatică independentă de context $G = (N, \Sigma, P, S)$ având producțiile numerotate sub forma

$$p.X_{p_0} \rightarrow X_{p_1} X_{p_2} \dots X_{p_{n_p}}$$

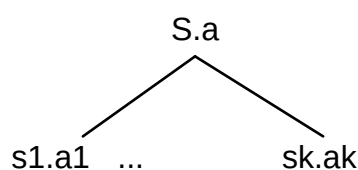
în care simbolul inițial S nu apare în partea dreaptă a nici unei producții;

b) o mulțime de atribute A_i . Fiecare atribut $a \in A_i$ este caracterizat printr-o mulțime de valori $V(a)$ la fel cum un tip de date este caracterizat prin mulțimea valorilor sale. Fiecărui simbol $X \in N \cup \Sigma$ îi corespunde o mulțime finită de atribute $A(X)$ formată din două mulțimi disjuncte $S(X)$ și $M(X)$, numite mulțimea atributelor sintetizate și respectiv moștenite ale lui X . Un atribut poate reprezenta o entitate oarecare: o valoare numerică, un tip, un câmp de caractere, o adresă de memorie, etc. Atributele din $A(X)$ însoțesc simbolul X în toate producțiile în care acesta apare.

c) o mulțime F de reguli semantice, folosite pentru evaluarea fiecărei producții:

- pentru un atribut sintetizat, valoarea sa se calculează în funcție de valorile atributelor asociate fiilor acestui nod

Gramatică cu
atribute

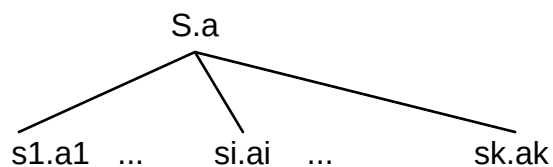


$S \rightarrow s_1 \cdots s_k$ este regula de producție

a = atributul sintetizat asociat lui S și se calculează în funcție de attributele $a_1, \dots, a_k$ asociate respectiv lui $S_1, \dots, S_k$: $a = f(a_1, \dots, a_k)$

Simbolurile terminale nu pot avea attribute sintetizate.

- pentru un atribut moștenit, valoarea se calculează utilizând valorile atributelor asociate fraților sau părinților acestui nod



$S \rightarrow s_1 \dots s_k$ este regula de producție

a_i = atribut moștenit asociat lui s_i ; $a_i = f(a, a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_k)$

Dacă existența atributelor sintetizate rezultă din faptul că înțelesul unei componente sintactice rezultă în primul rând din structura ei, necesitatea celor moștenite poate fi pusă la îndoială.

Există însă situații când attributele moștenite nu pot fi evitate, deoarece ele captează influența asupra înțelesului unei componente sintactice a contextului în care apare componenta respectivă. Este posibil ca numărul de parcurgeri să fie infinit, aceasta însemnând că valoarea unui atribut rezultă dintr-un calcul în care este implicată ea însăși. Knuth a dat un algoritm de verificare a existenței unei astfel de situații.

Dacă b este un atribut, asociat nodului a , ce depinde de atributul c , atunci regula semantică pentru b trebuie evaluată după regula semantică ce definește pe c . Interdependența dintre attributele sintetizate și cele moștenite asociate nodurilor arborelui de derivare poate fi înfățișată prin graful de dependență:

for fiecare nod n din arborele de derivare do

for fiecare atribut a asociat nodului n do

Construiește un nod în graful de dependențe pentru a

for fiecare nod n din arborele de derivare do

for fiecare regulă semantică $b := f(c_1, c_2, \dots, c_k)$

asociată producției corespunzătoare nodului n do
for $i:=1$ to k do
 Construește o muchie de la nodul
 corespunzător lui c_i la cel corespunzător lui b .

De exemplu, fie $A.a = f(X.x, Y.y)$ o regulă semantică pentru producția $A \rightarrow XY$; această regulă definește atributul sintetizat $A.a$ care depinde de atributele $X.x$ și $Y.y$. Se construiesc nodurile $A.a$, $X.x$ și $Y.y$, apoi o muchie de la $X.x$ la $A.a$ și încă una de la $Y.y$ la $A.a$. Dacă producția $A \rightarrow XY$ are regula semantică $X.x := g(A.a, Y.y)$ atunci se construiește o muchie de la $A.a$ la $X.x$ și încă una de la $Y.y$ la $X.x$.

Exemplul 4.1.

producția

$$E \rightarrow E_1 + E_2$$

regula semantică

$$E.val := E_1.val + E_2.val$$

*Graf de
dependență*

Cele trei noduri din graful de dependență marcate prin $\bullet$ corespund atributelor $E.val$, $E_1.val$ și $E_2.val$

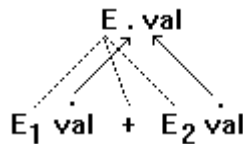


Figura 4.2 - Exemplu de graf de dependență

II.4.1.2. Specificarea semanticii folosind gramatici L-atributate

Funcțiilor semantice, fie ele simple transferuri de valori fie calcule oricât de complexe ale valorilor atributelor, li se asociază acțiuni semantice; astfel analiza semantică devine o înlănțuire a acțiunilor semantice în ordinea impusă de acțiunile sintactice. Simbolurile de acțiune se intercalează printre simbolurile din partea dreaptă a producțiilor în funcție de necesitățile de evaluare și transmitere a atributelor. De exemplu, producția $A \rightarrow XYZ$ poate fi completată cu simboluri de acțiune astfel:

$A \rightarrow XY@TZ$, unde notația $@$ introduce un simbol de acțiune.

Definiția 4.2. Fie $G = (N, \Sigma, P, S)$ o GIC și Ω o mulțime de simboluri (numite de acțiune) astfel încât $\Omega \cap (N \cup \Sigma) = \emptyset$. Atunci:

Introducerea
simbolurilor
de acțiune

a) Fie A_i o mulțime de attribute, fiecărui atribut $a \in A_i$ corespunzându-i o mulțime de valori $V(a)$. Pentru fiecare simbol $X \in N \cup \Sigma \cup \Omega$ există o mulțime de attribute asociate $A(X) \subset A_i$. $A(X) = M(X) \cup S(X)$; $m \in M(X)$ se numește atribut moștenit al lui X iar $s \in S(X)$ se numește atribut sintetizat al lui X .

b) Pentru fiecare producție din P există o rescriere numită producție atributată. În cadrul ei pot să apară în partea dreaptă și simboluri din Ω .

c) Pentru fiecare producție atributată există o mulțime de reguli de calculare a valorilor atributelor conform regulilor:

c1) valoarea unui atribut moștenit ce apare în partea dreaptă a unei reguli de producție este calculată în funcție de alte attribute ale producției. Pentru attributele moștenite ale simbolului de start se dau valori inițiale.

c2) valoarea unui atribut sintetizat asociat neterminalului din stânga producției se calculează în funcție de alte attribute ce apar în regulă.

c3) valoarea unui atribut sintetizat asociat unui simbol de acțiune se calculează în funcție de alte attribute ale simbolului de acțiune.

Condițiile de mai sus definesc o gramatică de traducere atributată.

Pentru a exemplifica această definiție considerăm regula de mai jos, unde attributele moștenite sunt precedate de caracterul $\downarrow$ iar cele sintetizate sunt precedate de $\uparrow$

$$\langle \text{terms} \rangle_{\downarrow s \uparrow t} ::= + \langle \text{term} \rangle_{\uparrow a} \langle \text{terms} \rangle_{\downarrow b \uparrow c}$$

$$b \leftarrow s + a; t \leftarrow c$$

Definiția 4.3. O gramatică de traducere atributată se numește L-atributată dacă specificațiile a) și b) se mențin nemodificate iar c) se modifică în:

c'_1) valoarea unui atribut moștenit ce apare în partea dreaptă a unei reguli de producție este calculată în funcție de attributele moștenite ale simbolului din stânga regulii și attribute arbitrare ale simbolurilor din dreapta regulii de producție ce apar la stânga simbolului a cărui apariție de atribut o considerăm;

c'_2) valoarea unui atribut sintetizat asociat neterminalului din stânga, se calculează în funcție de attributele moștenite ale simbolului din stânga și

Gramatica L-
atributată

atributele arbitrare ale simbolurilor din dreapta ;

c'_2) valoarea unui atribut sintetizat asociat unui simbol de acțiune se calculează în funcție de alte atribute moștenite ale simbolului de acțiune.

Definiția 4.4. O gramatică L-atributată se numește în formă simplă dacă cerințele c'_1) și c'_2) se modifică în:

c''_1) valoarea unui atribut moștenit asociat unui simbol din partea dreaptă este o constantă, valoarea unui atribut moștenit al simbolului din stânga sau valoarea unui atribut sintetizat al unui simbol care apare la stânga simbolului considerat, în partea dreaptă;

c''_2) valoarea unui atribut sintetizat asociat neterminalului din stânga este o constantă, valoarea unui atribut al neterminalului din stânga sau valoarea unui atribut sintetizat al unui simbol din partea dreaptă.

Gramatica L-atributată în formă simplă

Fiind dată o gramatică L-atributată, se poate construi o gramatică L-atributată în formă simplă echivalentă, prin adăugarea de simboluri de acțiune.

$$\langle terms \rangle_{\downarrow s \uparrow t} ::= + \langle term \rangle_{\uparrow a} @add_{\downarrow b, c \uparrow d} \langle terms \rangle_{\downarrow e \uparrow f}$$

$$b \leftarrow s, c \leftarrow a, e \leftarrow d, t \leftarrow f$$

Diferența esențială dintre cele două gramatici constă în înlocuirea regulilor de evaluare prin simboluri de acțiune. De exemplu, simbolul de acțiune $@add$ invocă rutina „add” care primește ca parametri de intrare atributele moștenite b și c și generează ca parametru de ieșire atributul sintetizat $d : d \leftarrow b + c$. Este posibil să se înlocuiască atribuirea explicită prin atribuirea implicită care se realizează folosind atribute variabile. De exemplu, producția anterioară poate fi scrisă ca

$$\langle terms \rangle_{\downarrow s \uparrow t} ::= + \langle term \rangle_{\uparrow a} @add_{\downarrow s, a \uparrow d} \langle terms \rangle_{\downarrow d \uparrow f}$$

Regulile ce guvernează această atribuire sunt următoarele:

1) un atribut moștenit, să zicem a , situat în partea dreaptă primește ca valoare atributul sintetizat sau moștenit având același nume și situat la stânga lui a .

Referindu-ne la regula analizată avem:

- atributul moștenit s din partea dreaptă primește valoare de la atributul moștenit s situat în stânga regulii;
- atributul moștenit d primește valoare de la atributul sintetizat d asociat

simbolului de acțiune $@add$.

2) un atribut moștenit, să zicem a , din partea stângă moștenește valoarea prin aplicarea unei producții care invocă partea stângă. De exemplu, atributul moștenit s din stânga regulii 4 primește valoare prin aplicarea producțiilor care conțin $\langle terms \rangle$ în partea dreaptă.

3) un atribut sintetizat ce apare în partea dreaptă a unei reguli primește valoare fie prin aplicarea unei rutine de acțiune fie prin sintetizarea informației produse prin procesul de construire a arborelui de derivare.

De exemplu, atributul sintetizat d din dreapta producției primește valoare prin invocarea rutinei de acțiune $@add$ iar atributul sintetizat f din dreapta primește valoare când $\langle terms \rangle_{\downarrow e \uparrow f}$ se unifică eventual cu cuvântul vid și f primește valoarea lui s așa cum rezultă din producția 3;

4) un atribut sintetizat din partea stângă primește valoare de la atributul sintetizat cu același nume situat în dreapta producției. Deci, atributul sintetizat f care apare în stânga producției primește valoare de la atributul sintetizat f din partea dreaptă.

Regula următoare este în forma simplă

$$\langle terms \rangle_{\downarrow s \uparrow f} ::= + \langle term \rangle_{\uparrow a} @add_{\downarrow s, a \uparrow d} \langle terms \rangle_{\downarrow d \uparrow f}$$

II.4.2. Model de analiză semantică

Definim gramatica independentă de context ce definește limbajul cu care vom lucra, apoi vom atribui această gramatică. Mulțimea simbolurilor terminale este $T = \{func, var, endf, rezultat, +, -, *, /, (,), \text{virgulă}, ;, =, \text{lit}, v, n, f\}$. Primele patru simboluri sunt cuvinte rezervate: *func* (este analogul lui *function* din Pascal și servește la definirea unei noi funcții), *var* (se utilizează la definirea variabilelor), *endf* (indică sfârșitul definiției unei funcții) și *rezultat* (specifică variabila sau parametrul formal a cărui valoare este returnată de funcție). Următoarele patru simboluri terminale corespund celor patru operații aritmetice; urmează parantezele rotunde și virgula, care servesc la delimitarea șirului de parametri ai unei funcții. Simbolul punct și virgulă se utilizează pentru semnalarea sfârșitului unei instrucțiuni sau a unei declarații, iar semnul egal servește la delimitarea părții stângi a unei atribuirii, de partea dreaptă a acesteia. Ultimele patru simboluri au următoarea semnificație:

- *lit* corespunde unui literal numeric;
- *n*, *v*, și *f* corespund fiecare unui identificator.

La întâlnirea unui identificator, analizorul lexical returnează simbolul terminal *n* dacă identificatorul este întâlnit pentru prima dată sau nu are încă atributele completate (este încă nedefinit), simbolul *v* dacă din tabela de simboluri reiese că este o variabilă sau parametru formal al unei funcții, respectiv simbolul terminal *f* dacă s-a întâlnit un nume de funcție.

Mulțimea simbolurilor neterminale este

$$N = \{Start, Vars, Func, Corp, Listparform, Sflist1, Atrib, Expr, Termen, Sftermen, Fact, Sffact, Listparact, Sflist2\}$$

Simbolul de start este *Start*, iar producțiile sunt următoarele:

1. $Start \rightarrow Vars \text{ Func } Corp$
2. $Vars \rightarrow \varepsilon$
3. $Vars \rightarrow var \ n \ Sflist1;$
4. $Sflist1 \rightarrow \varepsilon$
5. $Sflist1 \rightarrow ,n \ Sflist1;$
6. $Corp \rightarrow Atrib \ rezultat \ v;$
7. $Atrib \rightarrow \varepsilon$
8. $Atrib \rightarrow v = Expr; \ Atrib$
9. $Func \rightarrow func \ n \ Listparform;$
 $Vars \text{ Func } Corp \ endf \ Func$
10. $Func \rightarrow \varepsilon$
11. $Listparform \rightarrow \varepsilon$
12. $Listparform \rightarrow (\ n \ Sflist1)$
13. $Expr \rightarrow Termen \ Sfterm$
14. $Sfterm \rightarrow \varepsilon$
15. $Sfterm \rightarrow +Expr$
16. $Sfterm \rightarrow -Expr$
17. $Termen \rightarrow Fact \ Sffact$
18. $Sffact \rightarrow \varepsilon$
19. $Sffact \rightarrow * \ Termen$
20. $Sffact \rightarrow / \ Termen$
21. $Fact \rightarrow v$

Gramatica
limbajului

- 22. $Fact \rightarrow lit$
- 23. $Fact \rightarrow (Expr)$
- 24. $Fact \rightarrow f \quad Listparact$
- 25. $Listparact \rightarrow (Expr \ Sflist2)$
- 26. $Listparact \rightarrow \varepsilon$
- 27. $Sflist2 \rightarrow \varepsilon$
- 28. $Sflist2 \rightarrow , Expr \ Sflist2$

Un program definit de această gramatică se compune din definirea variabilelor, a funcțiilor și din programul principal care are aceeași sintaxă ca și corpul unei funcții.

Variabilele sunt definite înaintea funcțiilor pentru ca ele să fie vizibile și în corpul funcțiilor. Definirea variabilelor începe cu cuvântul cheie *var* urmat de o listă de identificatori, separați prin virgulă, și terminată cu punct și virgulă. Corpul unei funcții (precum și programul principal) constă dintr-o listă de atribuiri urmată de cuvântul cheie *rezultat* care precede un nume de variabilă (sau parametru formal, care la nivelul sintaxei este echivalent cu o variabilă). Din producția 9 rezultă felul cum se pot defini funcțiile: cuvântul rezervat *func* arată că începe o definiție de funcții; după *func* urmează numele funcției, urmat de lista, eventual vidă, a parametrilor formali și punct și virgulă. Urmează definirea variabilelor locale, definirea funcțiilor locale funcției în curs de definire (funcțiile imbricate), după care urmează corpul funcției. Definirea unei funcții se termină cu cuvântul rezervat *endf*. După definiția unei funcții poate urma o altă definiție de funcție; această funcție va fi la același nivel cu prima. Producția 10 arată că funcțiile locale pot să lipsească.

Producțiile 13-28 definesc sintaxa unei expresii aritmetice. În definirea sintaxei unei expresii s-a făcut deosebire între neterminalele *Termen* și *Factor*; primul apare în sume și diferențe iar al doilea în produse și cături. Această deosebire este necesară datorită priorităților diferite asociate operatorilor aritmetici. Dacă nu ar fi fost problema priorităților, sintaxa expresiilor aritmetice s-ar fi putut descrie și cu producțiile

- 13') $Expr \rightarrow Fact \ Sffact$
- 14') $Sffact \rightarrow Oper \ Fact$
- 15') $Sffact \rightarrow \varepsilon$

16') $Oper \rightarrow +$

17') $Oper \rightarrow -$

18') $Oper \rightarrow *$

19') $Oper \rightarrow /$

care puteau înlocui producțiile 13-20.

În definirea sintaxei limbajului de programare s-a folosit simbolul terminal *lit* pe care îl va returna analizorul lexical de fiecare dată când întâlnește în textul sursă un literal numeric, adică o succesiune de cifre. Acest fapt poate fi detaliat mai mult la nivelul gramaticii, considerând că *lit* este un neterminat (notat în continuare cu *Lit*) și introducând producțiile

$Lit \rightarrow 0 \ Sflit$ $Lit \rightarrow 4 \ Sflit$ $Lit \rightarrow 8 \ Sflit$

$Lit \rightarrow 1 \ Sflit$ $Lit \rightarrow 5 \ Sflit$ $Lit \rightarrow 9 \ Sflit$

$Lit \rightarrow 2 \ Sflit$ $Lit \rightarrow 6 \ Sflit$ $Sflit \rightarrow \varepsilon$

$Lit \rightarrow 3 \ Sflit$ $Lit \rightarrow 7 \ Sflit$ $Sflit \rightarrow lit$

Gramatica precedentă va fi *L*-atributată astfel încât attributele și acțiunile semantice să permită:

- verificarea coincidenței numărului de parametri actuali cu numărul parametrilor formali, la apelul unei funcții;
- asigurarea regulilor de valabilitate pentru structura de blocuri, adică schimbarea contextelor și permiterea redefinirii locale a numelor;
- alocarea memoriei pentru variabilele și parametri formali ai funcțiilor.

Valorile
atributelor

Pentru aceasta s-au definit următoarele mulțimi de valori

VAL - corespunde valorii unui literal numeric;

DIM - corespunde numărului total de parametri și variabilele locale;

OFF - corespunde offsetului în cadrul articolului de activare: practic acest offset este numărul de ordine al identificatorului în lista concatenată a parametrilor formali și variabilelor locale ale funcției în ordinea apariției lor în textul sursă;

PAR - corespunde numărului de parametri formali sau actuali;

TIPV - corespunde tipului variabilei : variabilă locală sau parametru formal ;

NIS - corespunde nivelului de imbricare statică a unui identificator;

NUME = mulțimea identificatorilor admiși.

Valorile atributelor sunt: $V(VAL) = N$, $V(DIM) = N$,

$V(OFF) = N$, $V(PAR) = N$, $V(NIS) = N$, $V(TIPV) = \{var, par\}$;

Atributele asociate diferitelor simboluri gramaticale sunt:

$Start_{\uparrow OFF}$ - arată numărul de variabile definite la nivelul programului principal;

$Vars_{\downarrow OFF \uparrow OFF}$ - arată offsetul primei variabile din lista variabilelor locale, respectiv numărul total de variabile;

$Sflist1_{\downarrow OFF \downarrow TIPV \uparrow OFF}$ - arată offsetul și tipul următoarei variabile din listă, respectiv numărul total de variabile;

$Listparform_{\uparrow OFF}$ - arată numărul de parametri formali ai funcției;

$Listparact_{\downarrow PAR \uparrow PAR}$ - arată numărul inițial de parametri actuali (adică 0) respectiv numărul total de parametri actuali;

$Sflist2_{\downarrow PAR \uparrow PAR}$ - arată numărul de parametri actuali deja analizați, respectiv numărul total al lor;

$lit_{\uparrow VAL}$ - arată valoarea literalului

$v_{\uparrow TIPV \uparrow NUME \uparrow OFF \uparrow NIS}$ - arată tipul, numele și offsetul variabilei, respectiv nivelul de imbricare statică;

$n_{\uparrow NUME}$ - un identificator încă nedefinit nu are altă informație completată în tabela de simboluri, decât numele;

$f_{\uparrow NUME \uparrow PAR \uparrow DIM \uparrow NIS}$ - arată numele, numărul de parametri formali, numărul total de variabile, respectiv nivelul de imbricare.

Cu aceste notații, gramatica L-atributată este :

1. $Start_o \rightarrow Vars_{o,o} Func Corp$

2. $Vars_{o,o} \rightarrow \epsilon$

3. $Vars_{o,o1} \rightarrow var@Taien_n^{var,n,o}@Ins$
 $Sflist1_{o+1,var,o1}; @Re fa$

4. $Sflist1_{o,t,o} \rightarrow \epsilon$

5. $Sflist1_{o,t,o1} \rightarrow, n_n^{t,n,o}@Ins Sflist1_{o+1,t,o1};$

6. $Corp \rightarrow Atrib rezultat v_{t,n,o,i}; Atrib$

7. $Atrib \rightarrow \epsilon$

8. $Atrib \rightarrow v_{t,n,o,i} = Expr; Atrib$

9. $Func \rightarrow func @Taie n_n @Refă @Context @Taie Listparform_o$

$@Refă; Vars_{o,o1} Func Corp, endf @Exit^{n,o,o1}@Insfunc Func$

10. $Func \rightarrow \epsilon$

11. $Listparform_o \rightarrow \epsilon$

12. $Listparform_o \rightarrow (n_n^{par,n,o}@Ins Sflist1_{1,par,o})$

Atributele
asociate

Gramatica L-
atributată

13. $Expr \rightarrow Termen\ Sfterm$
14. $Sfterm \rightarrow \varepsilon$
15. $Sfterm \rightarrow +Expr$
16. $Sfterm \rightarrow -Expr$
17. $Termen \rightarrow Fact\ Sffact$
18. $Sffact \rightarrow \varepsilon$
19. $Sffact \rightarrow * Termen$
20. $Sffact \rightarrow / Termen$
21. $Fact \rightarrow v_{t,n,o,i}$
22. $Fact \rightarrow lit_v$
23. $Fact \rightarrow (Expr)$
24. $Fact \rightarrow f_{n,p,d,i} Listparact_{0,a}^{p,a} @Verif$
25. $Listparact_{p,p1} \rightarrow (Expr\ Sflist2_{p+1,p1})$
26. $Listparact_{p,p} \rightarrow \varepsilon$
27. $Sflist2_{p,p} \rightarrow \varepsilon$
28. $Sflist2_{p,p1} \rightarrow Expr Sflist2_{p+1,p1}$

Acțiunile semantice au următorul rol :

$@Taie$ - taie legătura de la blocul context curent la contextul tată, pentru a permite redefinirea locală a numelor;

$@Refă$ - reface legătura la contextul tată, pentru a permite folosirea numelor din blocurile înglobate;

$TIPV, NUME, OFF @Ins$ - inserează în blocul curent al tabelii de simboluri o variabilă de tip $TIPV$ cu numele $NUME$ și offsetul OFF ;

$@Context$ - crează un context imbricat, initial vid;

$NUME, PAR, DIM @Insfunc$ - inserează în blocul curent al tabelii de simboluri o funcție cu numele $NUME$, cu numărul parametrilor egal cu PAR și cu numărul total de variabile plus parametri formali egal cu DIM ;

$PAR, PAR @Verif$ - verifică dacă numărul parametrilor actuali este egal cu numărul celor formali;

$@Exit$ - determină ieșirea din contextul curent în contextul tată.

Acțiuni
semantice

Teme Curs

Teste Autoevaluare

1. De cine depinde numărul de parcurgeri ale arborelui de derivare necesar pentru calcularea atributelor?..... 1 punct
 2. Prin ce metode poate fi modificat numărul parcurgerilor?..... 1 punct
 3. Care sunt elementele ce definesc o gramatică cu atribute?..... 1 punct
 4. Câte tipuri de atribute există și prin ce se caracterizează?..... 2 puncte
 5. Care atribute nu pot fi eliminate și de ce?..... 1 punct
 6. În ce parte a unei reguli de producție se plasează simbolurile de acțiune?..... 1 punct
 7. Dați un exemplu de situație în care parcurgerea arborelui de derivare, în vederea calculării atributelor, duce la ciclare?..... 1 punct
 8. Care este atributul care trebuie să fie obligatoriu prezent?..... 1 punct
- Oficiu..... 1 punct

Răspunsuri

1. De ordinea de evaluare a atributelor și de resursele de memorie disponibile
2. Prin modificări ale definiției limbajului sau impunerea unor restricții
3. Gramatica independentă de context, mulțimea atributelor și mulțimea regulilor semantice
4. Atribute sintetizate și atribute moștenite. Pentru un atribut sintetizat, valoarea sa se calculează în funcție de valorile atributelor asociate fiilor acestui nod. Pentru un atribut moștenit, valoarea se calculează utilizând valorile atributelor asociate fraților sau părinților acestui nod
5. Atributele sintetizate, deoarece existența lor rezultă din faptul că înțelesul unei componente sintactice este dat în primul rând de structura ei
6. În partea dreaptă
7. Funcția care calculează atributul a are ca argument pe a
8. Adresa de memorie

Rezumat: S-au definit noțiunile de atribut, gramatică L -atributată și s-a prezentat modul de utilizare a gramaticii L -atributate pentru specificarea semanticii unui limbaj; în final s-a dat un exemplu de gramatică L -atributată pentru un limbaj de programare.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se implementează un program simplu care să verifice prin tehnici semantice transmiterea corectă a parametrilor formali

Teme Laborator/Seminar

1. Implementati analiza semantica pentru verificarea transmiterii corecte a parametrilor actuali.

Rezumat: *se implementează un model de analiză semantică*

Notății

Descriere Generală

Se prezintă trei forme de reprezentare a codului intermediar: forma poloneză, arbori sintactici și triplete. Ca model se prezintă codul intermediat pentru expresii booleene.

Obiective

- cunoașterea semnificației și importanței codului intermediar
- cunoașterea tipurilor de cod intermediar și a diferențelor dintre ele
- cunoașterea posibilităților de implementare a codului cu trei adrese
- înțelegerea modului de generare a codului cu trei adrese

Cuprins

- II.5.1. Forma poloneză
- II. 5.2. Arbori sintactici
- II.5.3. Cod intermediar cu trei adrese
 - II.5.3.1. Triplete
 - II.5.3.2. Cuadrule
 - II.5.3.2.1. Expresii booleene

Conținut Curs

Rezultatul analizei sintactice și semantice constă dintr-un „fișier“ conținând traducerea programului într-un limbaj intermediar. Acesta este mai apropiat de limbajul de asamblare decât de cel sursă. Astfel, programul este o succesiune de operații împreună cu operanzii asociați. Operațiile sunt în majoritate similare celor din limbajul de asamblare: operații aritmetice, atribuire, teste, salturi, iar ordinea lor din program este cea în care se execută. Din program lipsesc declarațiile, descrierea operanzilor găsindu-se în tabela de simboluri. În același timp, codul intermediar se deosebește de limbajele de asamblare prin aceea că operanzii nu sunt regiștri sau cuvinte de memorie, ci referințe la intrări în tabela de simboluri. În afara referințelor propriu-zise, operanzii mai pot conține și informații sumare privind natura lor: variabile simple, variabile indexate,

Generalități

variabile temporare, constante, apeluri de funcții sau informații privind modul de adresare: direct sau indirect. Structura secvenței operațiilor, ca și modul de reprezentare a unei instrucțiuni sunt dependente de soluția adoptată pentru codul intermediar: forma poloneză, arbori sintactici, triplete, cuadrupe.

II.5.1. Forma poloneză

Acest tip de cod intermediar este foarte utilizat pentru limbaje constituite în general din expresii aritmetice și structuri de control rudimentare. O expresie în formă poloneză, sau notație postfixată se caracterizează prin aceea că operatorii apar în ordinea în care se execută operațiile pentru calculul expresiei. De aceea, evaluarea unei expresii în formă poloneză se face parcurgând într-un singur sens expresia și executând operațiile ținând seama de aritatea lor. De exemplu, expresia $a * (b + c)$ se traduce în formă poloneză în $abc + *$. În parcurgerea ei spre dreapta întâlnim întâi $+$ care, fiind o operație binară, determină efectuarea adunării $b + c$. Urmează $*$ care, precedat de a și de rezultatul lui $b + c$ (să-l notăm cu t), determină operația $a * t$.

Definiția 5.1. O expresie în formă poloneză (EFP) se definește astfel:

- 1) orice operand a este EFP;
- 2) dacă $e_1, e_2, \dots, e_n$ sunt EFP și $*$ este o operație n -ară ($n \geq 1$), atunci $e_1 e_2 \dots e_n *$ este EFP;
- 3) orice expresie formată altfel ca la 1) și 2) nu este EFP.

Definiția
formei
poloneze

Transcrierea unei expresii aritmetice în formă poloneză se face cu următorul algoritm :

Algoritmul 5.1

Intrare : o expresie aritmetică

Ieșire : forma poloneză postfixată a expresiei aritmetice

Metoda : se definesc prioritățile pentru operatori :

- $\$$ și $($ au prioritatea 0
- $+$ și $-$ au prioritatea 1
- $*$ și $/$ au prioritatea 2

$\$$ este un simbol special, iar pentru $)$ nu avem nevoie de priorități.

Lucrăm cu triplete (α, β, γ) , unde α este șirul de intrare, β este stiva de lucru folosită pentru depozitarea temporară a operatorilor iar γ este șirul de ieșire. Configurația inițială este $(\alpha, \$, \varepsilon)$ iar cea finală este $(\varepsilon, \$, \bar{\alpha})$.

Pasul 1. $(\alpha\alpha, b\beta, \gamma) \vdash (\alpha, b\beta, \gamma a)$ dacă a este operand

$\vdash (\alpha, ab\beta, \gamma)$ dacă a este operator și $P(a) > P(b)$

sau $a = ($

$\vdash (\alpha\alpha, \beta, \gamma b)$ dacă a este operator și $P(a) \leq P(b)$

Pasul 2. $(\alpha, b\beta, \gamma) \vdash (\alpha, \beta, \gamma)$ dacă $b = ($

$\vdash (\alpha, \beta, \gamma b)$ dacă $b \neq ($

Pasul 3. $(\varepsilon, b\beta, \gamma) \vdash (\varepsilon, \beta, \gamma b)$ dacă $b \neq \$$,

unde $P(x)$ este prioritatea operatorului x .

Așa cum am subliniat anterior, avantajul folosirii forme poloneze ca limbaj intermediar constă în proprietatea că expresia poate fi evaluată printr-o singură trecere prin textul ei.

Simplitatea algoritmului de evaluare recomandă folosirea forme poloneze ca limbaj intermediar în compilare. Dar, în cazul limbajelor de programare există și alte operații ce trebuie traduse în cod intermediar: atribuirii, teste, salturi, indexări, etc. De aici, necesitatea extinderii reprezentării și la asemenea operații sau folosirii altor forme de reprezentare.

Se pot adăuga, de exemplu, următoarele operații:

a) $L \text{ goto}$ - operație ce indică o instrucțiune de salt din programul sursă. Această etichetă se află într-o intrare în tabela de simboluri care se actualizează la un moment dat cu echivalentul etichetei în forma poloneză: un index în șirul $fpol$.

b) $E_1 E_2 \dots E_n \ A \ \text{INDEX}$ - operație cu număr variabil de operanzi reprezentând indexarea tabloului A cu rezultatele expresiilor $E_1, \dots, E_n$. Operația INDEX, pe baza simbolului A din vârful stivei, determină din tabela simbolurilor numărul n al dimensiunilor lui A și extrage din stivă, pentru indexare, cele n valori ale expresiilor.

c) $E_1 E_2 \dots E_n \ P \ \text{PREL}$ - operator cu număr variabil de operanzi reprezentând apelul procedurii P cu parametri efectivi dați de expresiile $E_1, \dots, E_n$.

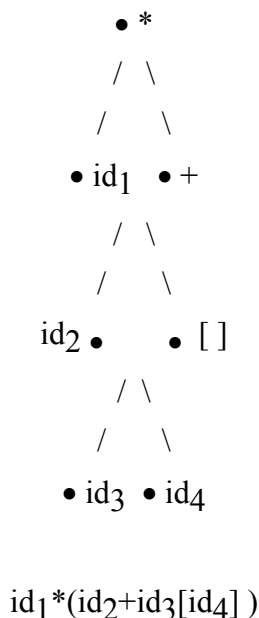
d) $I_1 S_1 I_2 S_2 \dots I_n S_n \ A \ \text{TDECL}$ - operație cu număr variabil de operanzi reprezentând declarația tabloului A . I_j și S_j reprezintă expresiile ce dau limita

inferioară și respectiv superioară pentru dimensiunea j .

II. 5.2. Arbori sintactici

Reprezentarea programului intermediar ca un arbore este forma de reprezentare cea mai apropiată de structura sintactică a programului sursă, fiind mai îndepărtată de structura programului obiect. Ea conține puține elemente noi față de programul sursă, fiind o formă concentrată a acestuia. Din această cauză reprezentarea arborescentă este utilă în faza de optimizare a codului. În general, arborele folosit nu este cel al derivării, ci o variantă a sa care nu conține redundanțe. Un astfel de arbore se numește arbore sintactic sau abstract. În figura 5.1 se prezintă arborii sintactici ai unei expresii aritmetice și ai unei instrucțiuni condiționale. Se observă că nodurile interioare sunt etichetate cu operatori, iar frunzele cu operanzi.

Operațiile nu sunt numai aritmetice, ele pot fi: indexări, selectări, comparații, atribuirii, salturi, etc. De asemenea, ele pot fi: unare, binare sau ternare. Aritatea operațiilor dă numărul de descendenți ai nodului etichetat cu operația respectivă. Totuși, în unele cazuri se preferă folosirea arborilor binari care au reprezentarea mai uniformă și sunt mai ușor de parcurs.



*Exemplu de
arbore
sintactic*

Figura 5.1

II.5.3. Cod intermediar cu trei adrese

Cel mai adesea, codul intermediar este văzut ca o secvență de instrucțiuni de tipul $A := B \text{ op } C$, unde A, B și C sunt identificatori din program, constante sau variabile temporare generate de compilator, iar op este o operație aritmetică sau logică. Instrucțiunile complexe sunt fragmentate în instrucțiuni simple conținând un singur operator. Variabilele temporare sunt create pentru a memora rezultatele intermediare. Codul cu trei adrese poate fi implementat sub formă de triplete, triplete indirecte sau cuadrule.

II.5.3.1. Triplete

Sunt reprezentate prin structuri cu trei câmpuri conținând operatorul și cei doi operanzi. Instrucțiunea $A := B \text{ op } C$ se reprezintă printr-o structură ale cărei câmpuri conțin : op, B și C . Cele trei câmpuri sunt pointeri către tabela de simboluri sau către structura tripletelor. Pointerii către structura tripletelor se vor reprezenta prin numere între paranteze rotunde.

Exemplul 5.1. Expresia $A := (B + C) * D$ se reprezintă astfel:

număr triplet	Operator	Operand stânga	operand dreapta
(0)	+	B	C
(1)	*	(0)	D
(2)	:=	A	(1)

Cod intermediar sub forma de triplete directe

În faza de optimizare a codului au loc, frecvent, operații de suprimare sau deplasare a instrucțiunilor cu trei adrese. În caz de deplasare trebuie modificați toți pointerii către aceste instrucțiuni. Trebuie parcursă structura tripletelor și modificate toate tripletele care utilizează variabila temporară asociată tripletului deplasat. Pentru a economisi timp se preferă folosirea *tripletelor indirecte*. În acest caz, structuri tripletelor i se asociază o listă de pointeri, care dau ordinea de execuție a tripletelor. Când au loc modificări în ordinea de execuție a tripletelor, este suficient să se reordoneze numai lista pointerilor.

Exemplul 5.2. Expresia $A := (B + C) * D$ se reprezintă prin triplete indirecte astfel:

Instrucțiuni		operator	operand stânga	operand dreapta	
(0)	(100)	(100)	+	B	C
(1)	(101)	(101)	*	(0)	D
(2)	(102)	(102)	:=	A	(1)

*Cod
intermediar
sub forma de
triplete
indirecte*

II.5.3.2. Cuadrule

Cuadrulele sunt structuri cu patru câmpuri conținând operatorul, cei doi operanzi și rezultatul. Câmpurile corespunzătoare operanzilor și rezultatului pointează către tabela de simboluri. Trebuie, deci, ca variabilele temporare să fie memorate în tabela de simboluri la fel cum sunt memorați identificatorii unui program sursă.

Exemplul 5.3. Expresia $A := (B + C) * D$ se reprezintă prin cuadrule astfel:

	operator	operand stânga	operand dreapta	rezultat
(0)	+	B	C	T ₁
(1)	*	T ₁	D	T ₂
(2)	:=	A	T ₂	

*Cod
intermediar
sub formă de
cuadrule*

În continuare ne vom ocupa numai de generarea codului intermediar sub formă de cuadrule. Instrucțiunile cu trei adrese pe care le utilizăm în continuare sunt :

- instrucțiuni de atribuire :
 - $A := B \text{ op } C$ cu *op* operator aritmetic sau logic binar;
 - $A := \text{op} B$ cu *op* operator aritmetic sau logic unar;
 - $A := B$
- instrucțiuni de salt necondițional:

- *goto* Q ;
- instrucțiuni de salt condițional:
 - *if* $A \text{ oprel } B \text{ goto } Q$ unde *oprel* este un operator relațional; dacă relația este satisfăcută se execută instrucțiunea cu trei adrese etichetată cu Q ;
- apel de subprograme :
 - *Param* Pa - specifică faptul că Pa este parametru;
 - *Call* Pr, Np - specifică apelul procedurii Pr cu Np parametri;
- atribuirea indexată:
 - $A := B[i]$ și $A[i] := B$
- atribuirea prin pointeri și adrese:
 - $A := *B$ și $*A := B$;
 - $A := \text{Adr } B$;

*Instrucțiuni cu
trei adrese*

Exemplul 5.4.

$A := B \text{ or } C$ $T_1 := B \text{ or } C$
 $A := T_1$

$\text{Prod}(A, B)$ (1) *Param* A
 (2) *Param* B
 (3) *Call* $\text{Prod}, 2$

În tabelul următor se dă corespondența dintre instrucțiunile cu trei adrese și cuadruple:

Instrucțiunea cu trei adrese	Cuadruplul
$A := B \text{ op } C$	(op, B, C, A)
$A := \text{op } B$	$(\text{op}, B, _, A)$
$A := B$	$(:=, B, _, A)$
<i>Param</i> Pa	$(\text{Param}, Pa, _, _)$
<i>Call</i> Pr, Np	$(\text{Call}, Pr, Np, _)$
<i>goto</i> Q	$(\text{goto}, _, _, Q)$
<i>If</i> $A \text{ oprel } B \text{ goto } Q$	(oprel, A, B, Q)

Vom exemplifica, în continuare, modul de generare a codului intermediar

pentru expresii booleene.

II.5.3.2.1. Expresii booleene

Gramatica ce generează expresii booleene este:

$G = (\{ E \}, \{ id, or, and, not, oprel, :=, (,) \}, P, E)$ cu următoarele producții :

1. $E \rightarrow E \text{ or } E$
2. $E \rightarrow E \text{ and } E$
3. $E \rightarrow not \ E$
4. $E \rightarrow (\ E \)$
5. $E \rightarrow id$
6. $E \rightarrow id \ oprel \ id$

Reprezentare prin valorile fals și adevărat

Pentru a reprezenta valoarea unei expresii booleene vom folosi valorile 0 pentru fals și 1 pentru adevărat. Expresiile sunt evaluate de la stânga la dreapta ținând seama de prioritatea operatorilor: operatorul *not* este prioritar în raport cu *and*, care este prioritar în raport cu *or*. În acțiunile semantice asociate regulilor gramaticii, presupunem că toți identificatorii sunt de tip boolean. Gramatica cu atributele și acțiunile semantice asociate sunt :

- 1) $E \rightarrow E_1 \text{ or } E_2$ @1
- 2) $E \rightarrow E_1 \text{ and } E_2$ @2
- 3) $E \rightarrow not \ E_1$ @3
- 4) $E \rightarrow (\ E_1 \)$ @4
- 5) $E \rightarrow id$ @5
- 6) $E \rightarrow id_1 \ oprel \ id_2$ @6

@1 $T := Var_Temp ()$
 $E.intrare := T$
 $Gen_cuadr (or, E_1.intrare, E_2.intrare, T)$

@2 $T := Var_Temp ()$
 $E.intrare := T$
 $Gen_cuadr (and, E_1.intrare, E_2.intrare, T)$

@3 $T := Var_Temp ()$

Gramatica


```

E.intrare := T
Gen_cuadr (not , E1 .intrare , _ , T )

@4   E.intrare := E1 .intrare

@5   E.intrare := id.intrare

@6   T := Var_Temp ( )
      E.intrare := T
      Gen_cuadr (oprel , id1 .intrare , id2 .intrare , Cuadr+3 )
      Gen_cuadr ( := , 0 , _ , T )
      Gen_cuadr ( goto , _ , _ , Cuadr+2 )
      Gen_cuadr ( := , 1 , _ , T )

```

atributată
pentru expresii
booleene

Cuadr trimite către prima intrare liberă din tabela de cuadrupe. Această variabilă este incrementată automat la fiecare creare a unui nou cuadrupe, adică la fiecare apel al funcției Gen_cuadr().

Reprezentarea printr-o poziție de atins

Această metodă, numită și evaluare prin scurt-circuit, permite generarea unui cod corespunzător unei expresii ce va fi evaluată în funcție de valoarea sub-expresiilor. Dacă avem expresia E_1 or E_2 numai E_1 va fi evaluată dacă ea este adevărată. Acest tip de reprezentare este interesant deoarece expresia booleană este o condiție într-o instrucțiune iterativă sau condițională; de exemplu :

```

if E then S else S      sau
while E do S             sau
repeat S until E

```

Expresia următoare conține un exemplu de expresie booleană

```

if  A < B or A > D
then  A := A+1;

```

Ea se traduce prin cuadrupe astfel :

```

( 0 )  if  A < B  goto ( 3 )
( 1 )  if  A > D  goto ( 3 )
( 2 )  goto ( 5 )
( 3 )  T1 := A+1
( 4 )  A := T1

```

Codul generat

(5)

Dacă utilizăm o analiză ascendentă, când este generat un cuadru plu nu se cunoaște încă intrarea în tabela cuadru plurilor ce corespunde ieșirii adevărat; nu putem, deci, completa acest cuadru plu. Codul generat pentru expresii booleene va conține un număr de cuadru ple de salt (condițional și/sau necondițional) care vor rămne incomplete. Ele se vor completa în funcție de context. Cuadru ple incomplete trebuie memorate într-o listă. În realitate lucrăm cu două liste asociate unei expresii booleene: una pentru ieșirile adevărate și una pentru cele false. Fiecare listă conține și cuadru ple incomplete. Pentru a manipula aceste liste este nevoie de următoarele funcții:

- *Crează_lista(n)*: crează o listă conținând cuadru plu incomplet cu indicele n în tabela cuadru plurilor ;
- *Concatenează_liste ($L1, L2$)* : concatenează listele definite de pointerii $L1$ și $L2$, și returnează un pointer spre noua listă ;
- *Completează_lista (L, n)* : completează lista definită de pointerul L cu indicele n al tabelii de cuadru ple.

Folosim atributele $E.true$ și $E.false$ care conțin pointerii catre listele asociate ieșirilor “adevărat” și respectiv “fals”.

Considerăm regula $E \rightarrow E_1 \text{ or } E_2$ și avem :

- dacă E_1 este adevărată atunci E este adevărată fără a evalua E_2 ; deci ieșirile adevărate ale lui E sunt aceleași cu cele ale lui E_1 ;
- dacă E_1 este falsă atunci trebuie evaluată E_2 ; deci ieșirile false ale lui E_1 corespund cu primul cuadru plu al lui E_2 ;
- dacă E_1 este falsă atunci ieșirile lui E corespund cu cele ale lui E_2 ;

Acțiunile semantice vor fi:

- completarea listei $E_1.false$ cu indicele primului cuadru plu al lui E_2 din tabloul cuadru plurilor ;
- concatenarea listelor $E_1.true$ și $E_2.true$ pentru a crea lista $E.true$;
- punerea în corespondență a ieșirilor false ale lui E_2 cu cele ale lui E .

Pentru regula $E \rightarrow E_1 \text{ and } E_2$ avem :

- dacă E_1 este falsă atunci E este falsă fără a evalua E_2 ; deci ieșirile false ale lui E sunt aceleași cu cele ale lui E_1 ;
- dacă E_1 este adevărată atunci trebuie evaluată și expresia E_2 ; ieșirile adevărate ale lui E_1 corespund primului cuadru plu al lui E_2 ;
- dacă E_1 este adevărată, atunci ieșirile lui E corespund cu cele ale lui E_2 ;

Acțiunile semantice sunt :

- completarea listei $E_1.true$ cu indicele primului cuadru al lui E_2 ;
- concatenarea listelor $E_1.false$ și $E_2.false$ pentru a crea lista $E.false$;
- punerea în corespondență a ieșirilor adevărate ale lui E_2 cu cele ale lui E .

Pentru regula $E \rightarrow not E_1$ este suficient să se inverseze ieșirile lui E_1 pentru a obține pe cele ale lui E .

Pentru regula $E \rightarrow E_1 or E_2$, trebuie completată lista $E_1.false$ cu indicele din tabloul cuadrualelor unde apare primul cuadru al lui E_2 . Dar această listă este cunoscută numai după ce s-a utilizat această regulă de derivare. Trebuie modificată regula astfel încât primul cuadru al lui E_2 să poată fi accesibil atunci când acțiunea semantică are nevoie de el. Adăugăm un simbol neterminal M căruia îi atașăm un atribut ce memorează primul cuadru care urmează codului generat pentru E_1 . Acesta ne permite să completăm lista $E_1.false$ când codul corespunzător lui E_2 a fost generat. Vom avea :

$E \rightarrow E_1 or ME_2$ @1

$M \rightarrow \varepsilon$ @2

@1

Completează_Lista ($E_1.false$, M.cuadru)

.....

@2 M.cuadru :=Cuadru_nou

Deci variabilei M îi asociem atributul $M.cuadru$ care memorează indicele primei intrări libere din tabloul cuadrualelor; acesta este indicele primului cuadru al lui E_2 . Raționând ca mai sus, rezultă următoarea asociere a acțiunilor semantice pentru regulile de mai jos :

$E \rightarrow E_1 or ME_2$ @1

$E \rightarrow E_1 and ME_2$ @2

$E \rightarrow not E_1$ @3

$E \rightarrow (E_1)$ @4

$E \rightarrow id$ @5

$E \rightarrow id_1 oprel id_2$ @6

$M \rightarrow \varepsilon$ @7

- @1 E.true := Concatenează_liste (E₁.true, E₂.true)
 E.false := E₂.false
 Completează_lista (E₁.false , M.cuadruplu)
- @2 E.true := E₂.true
 E.false := Concatenează_liste(E₁.false , E₂.false)
 Completează_lista (E₁.true , M.cuadruplu)
- @3 E.true := E₁.false
 E.false := E₁.true
- @4 E.true := E₁.true
 E.false := E₁.false
- @5 E.true := Crează_Lista (Cuadruplu_nou)
 E.false := Crează_Lista (Cuadruplu_nou+1)
 Gen_cuadr (oprel , id.intrare , _ , _)
 Gen_cuadr (goto , _ , _ , _)
- @6 E.true := Crează_Lista (Cuadruplu_nou)
 E.false := Crează_Lista (Cuadruplu_nou+1)
 Gen_cuadr (oprel , id₁.intrare , id₂.intrare , _)
 Gen_cuadr (goto , _ , _ , _)
- @7 M.cuadruplu := Cuadruplu_nou

*O nouă
gramatică
atributată
pentru expresii
booleene*

Teme Curs

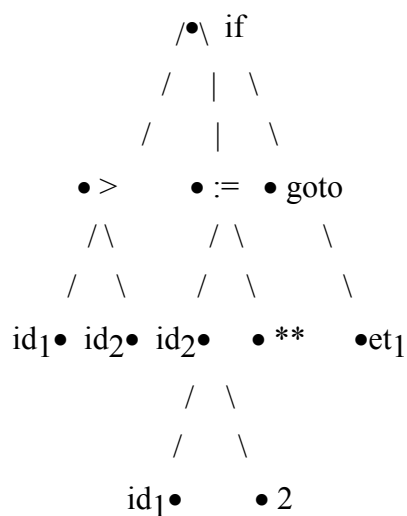
Teste Autoevaluare

1. Aplicați algoritmul de aducere la forma poloneză pentru expresia $c * (a + b)$ 1 punct
 2. Generați codul de tip arbore pentru instrucțiunea
if $id_1 > id_2$ then $id_2 := id_1 ** 2$ 1 punct
 3. Transpuneți în cod cu trei adrese instrucțiunea $A < B$ or C
..... 1 punct
 4. Generați codul intermediar pentru expresia $A > B$, folosind algoritmul care utilizează valorile fals și adevărat..... 1 punct
 5. Aplicați algoritmul de tip scurt circuitare pentru a genera codul corespunzător expresiei A or not B and C 5 puncte
- Oficiu 1 punct

Răspunsuri

1. $(c * (a + b), \$, \varepsilon) \vdash (* (a + b), \$, c) \vdash ((a + b), * \$, c)$
 $\vdash (a + b), (* \$, c) \vdash (+ b), (* \$, ca) \vdash (b), + (* \$, ca)$
 $\vdash (), + (* \$, cab) \vdash (), (* \$, cab +) \vdash (\varepsilon, * \$, cab +)$
 $\vdash (\varepsilon, \$, cab + *)$

2.



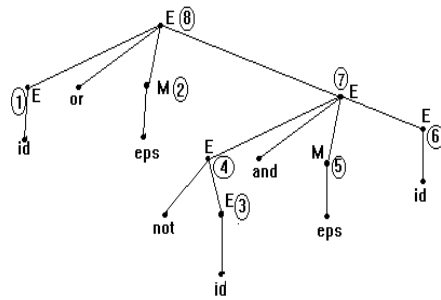
3.

- (0) if A < B goto (3)
- (1) $T_1 := 0$
- (2) goto (4)
- (3) $T_1 := 1$
- (4) $T_2 := T_1$ or C

4.

- (0) if A > B goto (3)
- (1) $T1 := 0$
- (2) goto (4)
- (3) $T1 := 1$
- (4) ...

5.



Evaluăm expresia descrisă de arbore parcurgându-l ascendent și executând acțiunile semantice asociate fiecărui nod. Presupunând că valoarea lui Cuadruplu_nou este 100 , avem :

Nodul 1 : E.true := { 100 }

E.false := { 101 }

(100) if A goto _

(101) goto _

Se crează listele E.true și E.false, și două cuadrupe incomplete la 100 și 101 .

Nodul 2 : M.cuadruplu := 102

Indicele 102 conține primul cuadruplu al lui E_2 , adică „not B and C“

Nodul 3: E.true := { 102 }

E.false := { 103 }

(102) if B goto _

(103) goto _

Se crează listele E.true și E.false, și două cuadrupe incomplete la 102 și 103 .

Nodul 4: E.true := { 103 }

E.false := { 102 }

Se inversează listele corespunzătoare ieșirilor adevărat și fals, deoarece se testează expresia „not B“.

Nodul 5 : M.cuadruplu := 104

Indicele 104 conține primul cuadruplu al lui E_2 , adică „C“.

Nodul 6 : E.true := { 104 }

E.false := { 105 }

(104) if C goto _

(105) goto _

Se crează listele E.true și E.false și două cuadrupe incomplete la 104 și 105.

Nodul 7: E.true := { 104 }

E.false := { 102 ,105 }

Se crează listele E.true și E.false și se completează cuadruplul de la (103) prin valoarea M.cuadruplu, adică 104:

(103) goto 104

Nodul 8: E.true := { 100 , 104 }

E.false := { 102 , 105 }

Se crează listele E.true și E.false și se completează cuadruplele listei E.false, adică cuadruplul cu indicele 101, prin valoarea M.cuadruplu =102

(101) goto 102

Obținem la ieșire listele de cuadrupe incomplete :

E.true := { 100 , 104 }

E.false := { 102 , 105 }

și următoarea listă de cuadrupe :

(100) if A goto _

(101) goto 102

(102) if B goto _

(103) goto 104

(104) if C goto _

(105) goto _

Rezumat: *S-au prezentat trei forme de reprezentare a codului intermediar și un exemplu pentru expresii booleene.*

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se va implementa codul intermediar sub una din formele alese pentru a continua etapele următoare ale construirii compilatorului

Teme Laborator/Seminar

1. Scrieți un program care să implementeze codul intermediar sub formă de arbori sintactici sau cod cu trei adrese

Rezumat: *se implementează codul intermediar sub o formă la alegere*

Notății

Descriere Generală

Se prezintă trei metode de optimizare a codului: simple, locale și globale. Se detaliază modul de lucru la nivelul fiecăreia și se explică pe exemple avantajele optimizării respective.

Obiective

Cunoașterea unor metode de îmbunătățire a codului intermediar, cu privire la timp de rulare și memorie ocupată

Cuprins

- II.6.1. Optimizări simple
- II.6.2. Optimizări globale
- II.6.3. Optimizări locale

Conținut Curs

Optimizarea codului este o fază opțională și are ca scop rearanjarea codului intermediar sau obiect în vederea obținerii unui program mai eficient. Eficiența se referă atât la memoria folosită la execuție dar, mai ales, la timpul execuției. Denumirea de optimizare este improprie, deoarece se pot obține programe mai bune dar foarte rar sunt și optime. Iată câteva dintre posibilele surse ale optimizării:

- un bun algoritm de programare
- o alocare inteligentă a regiștrilor
- compatibilizarea codului cu structura mașinii
- propagarea constantelor
- utilizarea identităților algebrice
- reducerea aritmetică a operatorilor
- eliminarea subexpresiilor comune
- reordonarea codului.

Dacă optimizările sunt structurate modular, ele nu vor duce la o creștere

Generalități

substanțială a complexității programului. Îmbunătățirea codului poate fi realizată în paralel cu analiza semantică, în paralel cu generarea de cod sau într-un pas separat. Așa cum am precizat, optimizarea se poate realiza atât asupra codului intermediar cât și asupra codului obiect. În ultimul caz, îmbunătățiri importante ale eficienței execuției se pot obține dacă în paralel cu generarea de cod se urmărește optimizarea alocării regiștrilor sau folosirea codului de instrucțiuni ale mașinii. De aceea, metodele folosite pentru optimizarea codului obiect sunt puternic dependente de mașină și, deci, mai dificil de prezentat într-un mod unitar. În cele ce urmează ne vom ocupa de optimizarea codului intermediar.

II.6.1. Optimizări simple

Una din cele mai simple optimizări este *aplatizarea*: ea constă în înlocuirea expresiilor ce pot fi evaluate în timpul compilării prin valorile lor. De exemplu, expresia $A = 2 + 3 + A + C$ poate fi înlocuită prin $A = 5 + A + C$, unde 5 înlocuiește expresia $2 + 3$.

Pentru aplatizarea codului se poate ține seama de :

- unele identități algebrice ; de exemplu

$$X + 0 = X$$

$$0 + X = X$$

$$X * 1 = X$$

$$1 * X = X$$

$$0/X = 0$$

$$X - 0 = X$$

- proprietățile de asociativitate și comutativitate ale unor operatori; de exemplu, expresia $5 + A + B + 7$ se poate înlocui cu $12 + A + B$.

În strânsă legătură cu identitățile algebrice se află și reducerea aritmetică a operatorilor, care constă în înlocuirea unui operator cu altul mai puțin costisitor; de exemplu

$$1. \quad i * 2 = 2 * i = i + i$$

$$2. \quad x / 2 = x * 0.5 .$$

O altă optimizare simplă este *propagarea constantelor* și constă în înlocuirea variabilelor cu valorile lor, dacă aceste valori sunt cunoscute la compilare. De exemplu secvența de program

$$PI := 3.141592$$

$$D := PI / 180.0$$

Aplatizarea

*Propagarea
constantelor*

poate fi rescrisă ca

$$PI: = 3.141592$$

$$D: = 3.141592/180.0$$

sau încă

$$PI: = 3.141592$$

$$D: = 0.0174644$$

II.6.2. Optimizări globale

Cea mai mare parte a timpului de optimizare a codului intermediar este destinat optimizării buclelor. Pentru a determina diferitele bucle ale unui program se utilizează graful de flux. El este un graf orientat ce definește relațiile dintre blocurile de bază. Un bloc de bază este format din instrucțiuni consecutive ale programului, care alcătuiesc o zonă accesibilă la execuție doar prin instrucțiunea de început și sunt executate una după alta exact în ordinea în care apar în program. Astfel, controlul execuției părăsește blocul de bază prin ultima instrucțiune din bloc. Doar într-un bloc de bază se pot controla variabilele fără perturbații exterioare; deci, împărțirea unui program în blocuri de bază este absolut necesară. Două blocuri de bază dintr-un program nu pot avea instrucțiuni comune; înseamnă că împărțirea unui program în blocuri de bază este echivalentă cu partiționarea sa.

Având o secvență de instrucțiuni (cu trei adrese) putem obține lista blocurilor de bază în două etape:

- a) se determină primele instrucțiuni ale blocurilor
- b) pentru fiecare primă instrucțiune se construiește blocul ei.

Bloc de bază

Dacă în urma acestor operații rămân instrucțiuni neincluse în vreun bloc ele pot fi eliminate deoarece nu vor fi executate niciodată. Determinarea primelor instrucțiuni se face astfel:

- prima instrucțiune din program este prima instrucțiune a unui bloc
- o instrucțiune ce urmează unei instrucțiuni de transfer este o primă instrucțiune
- o instrucțiune la care trimite o instrucțiune de transfer este o primă instrucțiune.

Blocul corespunzător unei prime instrucțiuni conține această instrucțiune precum și cele care urmează până la următoarea primă instrucțiune, exclusiv aceasta.

Prezentăm în continuare diferite tipuri de optimizare la nivelul unui bloc de bază, luând ca exemplu algoritmul ce efectuează produsul scalar a doi vectori:

```

produs := 0
indice := 1
repeat
    produs := produs + A[indice] * B[indice]
    indice := indice + 1
until indice > 20

```

Presupunând că unui cuvânt îi corespund patru octeți, programului de mai sus îi corespunde următoarea secvență de cadruple:

- (1) produs := 0
- (2) indice := 1
- (3) T1 := 4 * indice
- (4) T2 := adresa (A) - 4
- (5) T3 := T2[T1]
- (6) T4 := 4 * indice
- (7) T5 := adresa (B) - 4
- (8) T6 := T5[T4]
- (9) T7 := T3 * T6
- (10) T8 := produs + T7
- (11) produs := T8
- (12) T9 := indice + 1
- (13) indice := T9
- (14) if indice <= 20 goto (3)
- (15) -

Graful de flux asociat este cel din figura 6.1.

O primă optimizare constă în determinarea *invariantilor* din bucle și scoaterea lor în afară. Un invariant este un calcul ce dă același rezultat la fiecare iterație. Prin scoaterea în afară a invariantilor aceștia se execută o singură dată înainte de intrarea în buclă și deci numărul cadruplelor ce se execută se micșorează. În blocul BL2, cadruplele

- (4) T2 := adresa (A) - 4 și
- (7) T5 := adresa (B) - 4

reprezintă invarianti dacă spațiul de memorie rezervat vectorilor *A* și *B* este alocat în mod static.

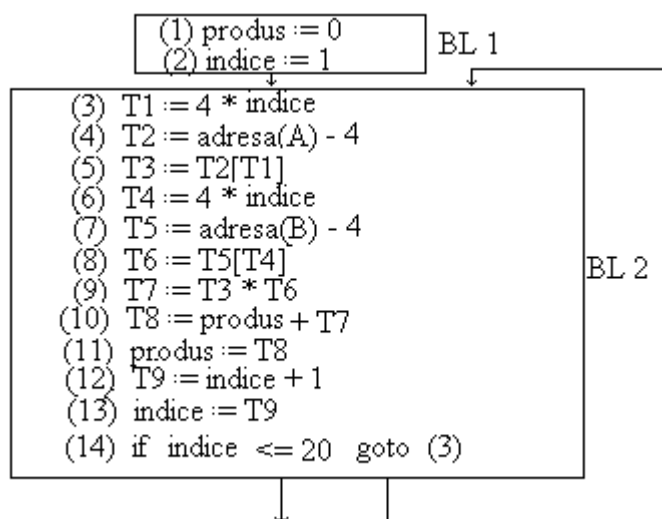
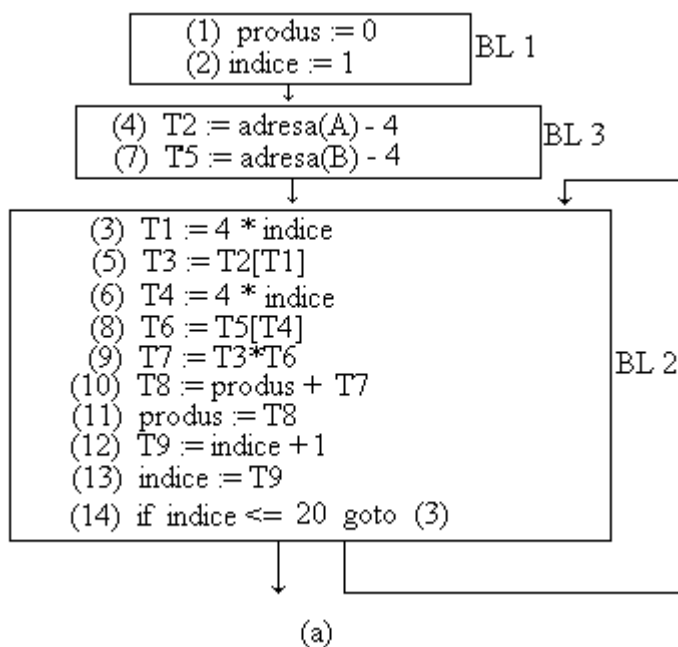


Figura 6.1

Dacă scoatem din blocul *BL2* cuadruplele (4) și (7) obținem graful de flux din figura 6.2.a. Aceste cuadruple sunt plasate înaintea lui *BL2* și formează un bloc *BL3*.



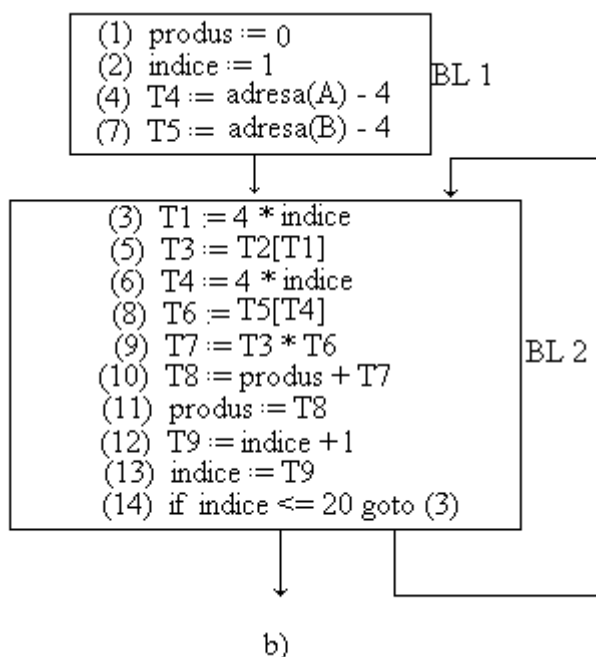


Figura 6.2

Blocurile *BL1* și *BL3* pot fi combinate într-un singur bloc deoarece, *BL1* este singurul predecesor al lui *BL3* iar *BL2* este singurul succesor al lui *BL1* (figura 6.2.b)

Scoțând cuadrupele (4) și (7), numărul cuadrupelelor din buclă scade de la 12 la 10 iar numărul de cuadrupe executate scade de la $2+12*20 = 242$ la $4+10*20 = 204$. Codul obținut poate fi optimizat în continuare prin eliminarea unor *variabile induse*. Acestea sunt variabile ale căror valori formează pe parcursul execuției repetate a ciclului, o progresie aritmetică. În exemplul nostru, variabila *indice* crește de la 1 la 20 cu pasul 1 iar *T1* crește de la 4 la 80 cu pasul 4. Variabila care se elimină este, în general, cea care este utilizată pentru calculul alteia. În cazul nostru se poate elimina variabila *indice*; cuadruplul

(3) $T1 := 4 * indice$

se va înlocui cu

(3) $T1 := T1 + 4$

Variabila *T1* nu are valori inițiale; de aceea trebuie adăugat un cuadruplu pentru a o inițializa la valoarea 0, deoarece la prima iterație ea are valoarea 4. Acest cuadruplu va fi inserat înaintea blocului *BL2*. Cuadrupele (2), (12) și (13), care utilizează variabila *indice*, se elimină. Cuadruplul (14) se modifică pentru a utiliza variabila *T1* în locul variabilei *indice*. La ultima execuție, *T1* trebuie să aibă valoarea 80, deci noul cuadruplu (14) este

(14) if $T1 <= 76$ goto (3)

*Eliminarea
variabilelor
induse*

Astfel, la ultima iterație $T1$ va avea valoarea 80. Se crează un nou bloc $BL3$ pentru a inițializa variabilele $T1$ și $T4$ iar blocurile $BL1$ și $BL3$ se combină într-unul singur, deoarece $BL1$ este singurul predecesor al lui $BL3$ iar $BL3$ este singurul succesor al lui $BL1$; astfel se obține graful de flux cu variabilele induse eliminate

II.6.3. Optimizări locale

Optimizările locale constă în eliminarea instrucțiunilor inutile, date de existența a două sau mai multe subexpresii comune, adică subexpresii echivalente sau care produc același rezultat.

În cazul subexpresiilor comune este suficient să calculăm rezultatul o singură dată și apoi doar să-l referim.

Exemplul 6.1. Fie secvența de program

```
x = (1 + 20) * (-x)
y = x * x + (x/y)
y = z = (x/y) / (x * x)
```

Codul intermediar generat este:

- (1) $T1 := (1+20)$
- (2) $T2 := -x$
- (3) $x := T1 * T2$
- (4) $T3 := x * x$
- (5) $T4 := x/y$
- (6) $y := T3 + T4$
- (7) $T5 := x/y$
- (8) $T6 := x * x$
- (9) $z := T5 / T6$
- (10) $y := z$

Expresia $x * x$ se calculează de două ori, în instrucțiunile (4) și (8); vom elimina a doua operație de calculare a acestei valori. În liniile (5) și (7) avem aceeași operație, x/y . Valoarea lui x rămâne neschimbată, dar a lui y se modifică în linia 6 și deci x/y nu este subexpresie comună.

Pentru a elimina instrucțiunile inutile se utilizează o structură de date particulară în vederea analizării unui bloc de bază: graful orientat fără cicluri

(*GOFC*). El descrie modul cum valoarea calculată pentru fiecare cuadru este utilizată în alte instrucțiuni ale blocului. El permite detectarea subexpresiilor comune unui bloc, a identificatorilor utilizați într-un bloc și evaluați în altul și a cuadrelor ale căror valori sunt utilizate în afara blocului. Într-un *GOFC*, nodurile sunt etichetate astfel:

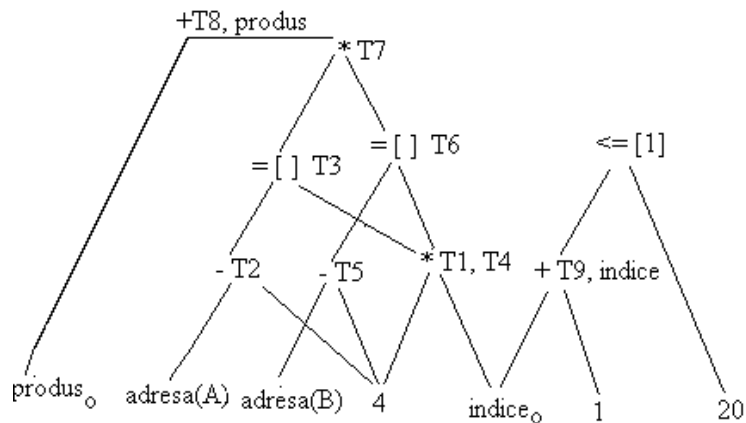
- fiecare terminal este etichetat cu un identificator (nume de variabilă sau de constantă) indicat prin valoarea 0; indicele precizează că este vorba de o valoare inițială a identificatorului;
- un nod interior este etichetat printr-un operator; el reprezintă valoarea calculată pentru expresia corespunzătoare acestui nod;
- nodurile interioare pot fi etichetate, în plus, printr-o mulțime de identificatori care au valoarea calculată în acel nod.

Nu trebuie confundat graful de flux cu un *GOFC*. Fiecare nod al unui graf de flux poate fi reprezentat printr-un *GOFC*. Pentru a construi un *GOFC*, se ia fiecare bloc de bază și se tratează toate cuadrelor. Când se întâlnește un cuadru de tipul $A := B \text{ op } C$, se examinează nodurile reprezentând valorile curente ale identificatorilor B și C . Se crează un nou nod etichetat op cu doi fii, la stânga nodul asociat lui B și la dreapta cel asociat lui C . Apoi se adaugă eticheta A nodului op . Dacă există deja un nod reprezentând pe $B \text{ op } C$, nu se mai crează un nou nod, doar se adaugă A la lista identificatorilor asociați acestui nod. Dacă A a etichetat în prealabil un alt nod, care nu e terminal, se elimină această etichetă, căci valoarea curentă a lui A este valoarea noului nod creat. Pentru cuadru $A := B$ nu se crează un nou nod, ci se adaugă eticheta A la nodul ce corespunde valorii curente B . Pentru a defini funcția de creare a unui *GOFC*, ce returnează un pointer către ultimul nod creat, utilizăm următoarele funcții :

- $Nod(identificator)$: returnează un pointer către nodul cu eticheta *identificator* dacă există, în caz contrar returnează *Nil*;
- $Nod_operator(op, pointer_st, pointer_dr)$: returnează un pointer către nodul cu eticheta op care are ca descendent stâng nodul definit de $pointer_st$ iar ca descendent drept nodul definit de $pointer_dr$; în caz contrar returnează *Nil*;

Vom aplica această funcție pentru blocul *BL2* din figura 6.1;

*Graf orientat
fără cicluri*



Se observă că :

- subexpresiile comune apar în același nod; $T1$ și $T4$, de exemplu
- variabilele temporare inutile de tipul

$T8 := produs + T7$

$produs := T8$

apar în același nod.

GOFC permite reconstruirea listei simplificate a cuadrupelilor eliminând subexpresiile comune și cuadruplele de tipul $A := B$ cu excepția celor care sunt necesare. În cazul când lista asociată unui nod operator conține numai variabile temporare se alege una la întâmplare, fiind foarte probabil ca ele să fie utilizate numai în blocul de bază curent. De exemplu, în cazul nodului $*T1, T4$ se poate reține oricare din variabilele $T1$ și $T4$. Dacă lista conține și o variabilă din program, de exemplu $T1, T4, A$, ar trebui să optăm pentru A deoarece este posibil ca aceasta să fie folosită în blocurile următoare. Pentru a ști care variabilă este utilizată în blocurile următoare, trebuie făcută o analiză globală, pe graful de flux, a transferului de valori ale variabilelor între blocuri. Dacă în urma acestei analize rezultă că, din lista de identificatori asociată unui nod, este posibil să avem nevoie în blocurile următoare de mai multe variabile, atunci alegem dintre acești identificatori unul la întâmplare, fie el A , și pentru ceilalți, fie ei $B_1, B_2, \dots, B_k$, introducem atribuirile $B_1 := A, B_2 := A, \dots, B_k := A$.

Teme Curs

Teste Autoevaluare

1. Ilustrați grafic (prin arbori sintactici) efectul aplatizării expresiei $(2 * i) * 3$ 1 punct
2. Generați codul intermediar cu trei adrese corespunzător programului de mai jos, apoi realizați împărțirea în blocuri de bază și construiți graful de flux.

```
FACT ← 1
for i = 2 to n do
  FACT ← FACT * i
FACTORIAL ← FACT
```

..... 4 (=2+1+1) puncte

3. Efectuați optimizări simple și locale pentru următoarea secvență de cod intermediar

- (1) $T1 := (1+20)$
- (2) $T2 := -x$
- (3) $x := T1 * T2$
- (4) $T3 := x * x$
- (5) $T4 := x / y$
- (6) $y := T3 + T4$
- (7) $T5 := x / y$
- (8) $T6 := x * x$
- (9) $z := T5 / T6$
- (10) $y := z$

..... 2 puncte

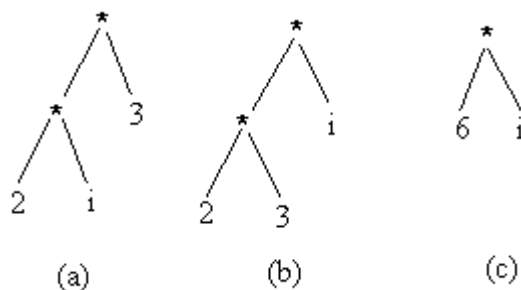
4. Care este deosebirea dintre graful de flux și GOFC?..... 1 punct

5. Ce se reduce prin optimizările globale (memorie, timp)?..... 1 punct

Oficiu..... 1 punct

Răspunsuri

1.

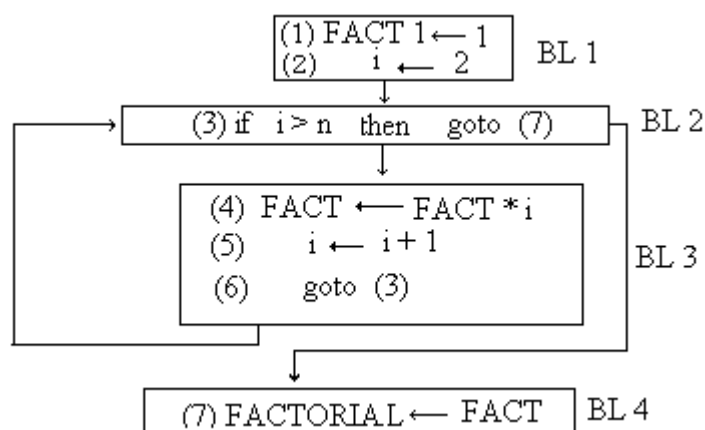


2.

Codul intermediar

- (1) $FACT \leftarrow 1$
- (2) $i \leftarrow 2$
- (3) if $i > n$ then goto (7)
- (4) $FACT \leftarrow FACT * i$
- (5) $i \leftarrow i + 1$
- (6) goto (3)
- (7) $FACTORIAL \leftarrow FACT$

Împărțirea în blocuri de bază



Graful de flux

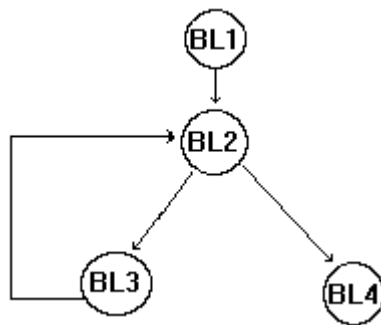


Figura 6.3

3. După propagarea constantelor și eliminarea expresiilor comune, codul devine:

$T2 := -x$

$x := 21 * T2$

$T3 := x * x$

$T4 := x/y$

$y := T3 + T4$

$T5 := x/y$

$z := T5/T3$

$y := z$

4. Fiecare nod al unui graf de flux poate fi reprezentat printr-un *GOFC*

5. Memorie și timp

Rezumat: *S-au prezentat modalități de realizare a optimizărilor de tip: local, global, simplu*

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Pentru fiecare tip de optimizare studiat se cere să se scrie câte un program care să implementeze metoda respectivă.

Teme Laborator/Seminar

1. Scrieți un program care să realizeze împărțirea în blocuri de bază.
2. Scrieți un program care, pentru o secvență de cod intermediar cu trei adrese, să efectueze optimizările simple și locale
3. Scrieți un program care efectuează optimizări globale la nivelul unui bloc de bază

Rezumat: *se implementează tipurile de optimizări studiate*

Notății

Descriere Generală

Se prezintă trei forme de reprezentare a codului intermediar: forma poloneză, arbori sintactici și triplete. Ca model se prezintă codul intermediar pentru expresii booleene

Obiective

- cunoașterea tipurilor de cod obiect
- cunoașterea generării codului pentru calculatoare cu un singur registru
- cunoașterea metodelor de generare a codului pentru calculatoare cu regiștri generali
- modalități de îmbunătățire a codului obiect

Cuprins

II.7.1. Generalități

II.7.2. Generarea codului pentru calculatoare cu registru acumulator

II. 7.3. Generarea codului pentru calculatoare cu regiștri generali

II.7.3.1. Gestiunea regiștrilor

II.7.3.2. Utilizarea GOFc în generarea codului obiect

Conținut Curs

II.7.1. Generalități

Ultima fază a compilării are ca scop sinteza programului obiect, program executabil sau aproape executabil, în sensul că el poate fi preluat de un alt procesor de limbaj implementat pe calculatorul țintă și tradus de acesta în cod executabil. Generarea codului este una din fazele cele mai importante și mai dificile. Pentru implementarea sa sunt necesare cunoștințe aprofundate despre mașina cu care se lucrează.

Forma luată de programul obiect este, de obicei, una din următoarele:

1) *Program executabil* : rezultatul obținut este direct executabil. El este stocat într-o zonă de memorie fixă și toate adresele sunt puse la zi. Această formă este

recomandată pentru programele mici. Inconvenientul său constă în lipsa de flexibilitate. Compilarea modulară este imposibilă, toate modulele trebuind să fie compilate simultan.

*Tipuri de cod
obiect*

2) *Program obiect.* Această formă este numită și translatabilă; înainte de execuție este necesară faza de editare de legături, care permite și legarea unor rutine din bibliotecă sau realizate de utilizatori. Este soluția cea mai des întâlnită în cazul compilatoarelor comerciale.

3) *Program în limbaj de asamblare.* Este forma cea mai simplă de generat, deoarece instrucțiunile sunt foarte apropiate de cele cu trei adrese. Programul este reprezentat de o mulțime de instrucțiuni simbolice care necesită o fază de asamblare înainte de execuție. Deci, generarea codului este simplificată, dar rezultatul nu poate fi direct utilizabil.

4) *Program într-un alt limbaj,* care simplifică mult generarea de cod, dar necesită cel puțin o compilare suplimentară pentru a putea fi executat. Este cazul preprocesoarelor de limbaje.

Generarea codului este dependentă de calculatorul țintă ca și de sistemul de operare al acestuia. De aceea prezentarea sa necesită unele presupuneri privind structura calculatorului țintă și a setului său de instrucțiuni. Pe de altă parte, ea este influențată de forma codului intermediar precum și de gradul în care compilatorul a rezolvat problemele traducerii: a făcut verificările semantice, a introdus conversii, a optimizat codul intermediar, l-a structurat pe blocuri de bază, etc. În cele ce urmează vom analiza generarea de cod pentru două tipuri de calculatoare:

- a) calculatoare cu acumulator
- b) calculatoare cu un număr de regiștri generali.

Presupunem că în fazele anterioare s-au realizat toate verificările semantice, s-a optimizat codul intermediar, etc. Pentru fiecare variabilă știm adresa ei la execuție, prin perechea (registru ce conține o adresă de început a înregistrării de activare, deplasamentul în această zonă). Mai presupunem că toate operațiile din codul intermediar au un corespondent în setul de instrucțiuni ale calculatorului iar variabilele sunt simple. Pentru ușurința înțelegerii vom genera codul obiect în limbaj de asamblare în care variabilele apar prin numele lor și nu prin referințe, deplasamente, etc. Generarea codului obiect apare ca un proces în care se parcurge instrucțiune cu instrucțiune forma intermediară, apelându-se pentru fiecare tip de instrucțiune proceduri de generare

corespunzătoare.

Pentru a genera un cod eficient trebuie să se țină seama de unele detalii legate de calculatorul țintă:

1) majoritatea calculatoarelor permit efectuarea unor calcule în mai multe feluri utilizând diferite instrucțiuni; de exemplu $nr := nr + 1$ se poate traduce prin

MOV AX, nr ; încarcă valoarea lui nr în registrul AX

INC AX ; incrementează 1 la conținutul registrului AX

MOV nr, AX ; se stochează conținutul registrului AX în locația corespunzătoare lui nr

sau

ADD nr, 1 ; se adună 1 la conținutul zonei de memorie nr

2) diferitele forme de traducere a aceleiași secvențe de instrucțiuni nu folosesc, în general, același număr de regiștri.

Costul unei instrucțiuni constă în lungimea sa în biți sau în timpul de execuție. Strategiile optimizării tind să se concentreze pe reducerea costului de execuție, deoarece majoritatea utilizatorilor apreciază mai mult viteza de execuție decât spațiul ocupat. Instrucțiunile care lucrează cu regiștri, fără a face apel la memorie, au un timp de execuție mai mic și ocupă mai puțină memorie. O altă posibilitate de a optimiza codul constă în eliberarea zonelor de memorie ocupate de variabile nefolositoare. În momentul când o variabilă devine nefolositoare și valoarea sa se află într-un registru, ea poate fi ștearsă din acesta. Când se ajunge la finalul unui bloc de bază, valoarea fiecărei variabile folositoare trebuie păstrată fie într-un registru, fie în memoria principală prin intermediul variabilelor temporare.

II.7.2. Generarea codului pentru calculatoare cu registru acumulator

Toate calculele trebuie efectuate într-un singur registru, numit adesea acumulator. Acumulatorul va fi continuu alocat la diferite variabile astfel încât să se minimizeze numărul de instrucțiuni ale codului generat. Să luăm mai întâi un exemplu simplu care arată cum poate fi utilizat în mod eficient registrul acumulator. Printre instrucțiunile existente în limbajul de asamblare sunt următoarele:

LOD X ; încarcă valoarea X în registrul acumulator

STO X ; memorează conținutul registrului acumulator într-un

Generalități

cuvânt de memorie notat cu X

ADD X ; adună valoarea lui X la valoarea acumulatorului

SUB X ; valoarea variabilei X se scade din valoarea acumulatorului

MUL X ; valoarea variabilei X este multiplicată prin valoarea
acumulatorului

DIV X ; valoarea acumulatorului se împarte la valoarea variabilei X

Toate cele patru operații aritmetice plasează rezultatul în acumulator și lasă neschimbat conținutul lui X. Compilatorul utilizează forma poloneză pentru reprezentarea codului intermediar. Codul pentru operatorii aritmetici binari este generat conform următorului algoritm :

- 1) se încarcă primul operand în acumulator;
- 2) se aplică operatorul folosind al doilea operand și lăsând rezultatul în acumulator;
- 3) se memorează rezultatul într-o variabilă temporară.

De exemplu, expresia $x+y$, care se reprezintă în forma poloneză ca $xy+$, se translatează conform acestui algoritm în

```

LOD x
ADD y
STO T1

```

unde T1 este adresa unei locații de memorie care conține valoarea unui rezultat intermediar. Codul pentru operatorul de asignare „:=” este generat conform următorului algoritm simplu :

1. se încarcă valoarea părții drepte a instrucțiunii de asignare în acumulator;
2. se memorează acest rezultat în variabila specificată.

Algoritmul parcurge liniar șirul, aflat în formă poloneză. Când este întâlnit un operator, se selectează ultimele două simboluri din stivă, se execută operația indicată de operator, iar rezultatul obținut este plasat în stivă. Deoarece operatorii sunt binari, se afișază un mesaj de eroare dacă stiva nu conține doi operanzi la întâlnirea unui operator. Dacă algoritmul s-a terminat și au mai rămas operanzi în stivă, expresia nu este o instrucțiune de atribuire validă.

Considerăm instrucțiunea $X \leftarrow A + (B * C + D)$ care se reprezintă în forma poloneză postfixă ca $XABC*D++\leftarrow$

Algoritmul precedent produce următorul cod :

```

LOD B
MUL C
STO T1
LOD T1

```

```
ADD D
STO T2
LOD A
ADD T2
STO T3
LOD T3
STO X
```

Codul generat nu este optim , căci secvențe de tipul

```
STO Ti
LOD Ti
```

sunt inutile. De asemenea, codul poate fi rearanjat folosind proprietatea de comutativitate a operatorilor de adunare și înmulțire. Astfel, secvența :

```
STO T2
LOD A
ADD T2
```

poate fi rescrisă sub forma

```
STO T2
LOD T2
ADD A
```

deoarece $T2 + A$ are aceeași valoare cu $A + T2$.

II. 7.3. Generarea codului pentru calculatoare cu regiștri generali

II.7.3.1. Gestiunea regiștrilor

În vederea generării codului obiect pentru calculatoare cu regiștri generali vom considera codul intermediar sub forma instrucțiunilor cu trei adrese. Pentru ușurința exprimării vom lucra cu expresii aritmetice pentru care sunt verificate condițiile de mai jos.

a) Variabilele sunt simple și alocate static. În vederea alocării regiștrilor trebuie cunoscută starea lor: un registru poate fi disponibil, adică memorează o valoare nesemnificativă, sau poate fi ocupat, când conține o valoare utilizabilă în continuare. Notăm cu $Var(R)$ mulțimea variabilelor a căror valoare se află în registrul R .

b) Pentru fiecare variabilă este necesar să se cunoască locul unde se află valoarea sa curentă. Notăm cu $Loc(X)$ mulțimea locurilor în care se află la un moment

dat valoarea variabilei X .

c) Pentru utilizarea eficientă a regiștrilor este necesară cunoașterea, pentru fiecare apariție a unei variabile în codul intermediar, a următoarei ei utilizări. Procedura de determinare a următoarei utilizări poate fi simplificată dacă ne limităm la un bloc de bază și considerăm că la ieșirea dintr-un bloc toate variabilele sunt utilizate în continuare. Notăm cu $Ni(A)$ numărul următoarei instrucțiuni din bloc ce folosește valoarea curentă a variabilei A . Dacă $Ni(A) = 0$, înseamnă că A nu are o utilizare următoare în blocul curent.

d) Gestiunea regiștrilor generali este asigurată de procedura *GESTREG* care întoarce o locație L ce urmează să fie folosită pentru memorarea unei valori a unei variabile A obținută prin instrucțiunea $A := B \text{ op } C$. Locația poate fi un registru sau o locație de memorie. Procedura determină locația L cercetând succesiv următoarele cazuri :

1) B se află într-un registru R și este ultima ei utilizare înaintea unei atribuirii $B := \dots$

Vom folosi pentru A chiar registrul folosit de B ; deci $L \leftarrow R$;

2) există un registru R disponibil; se ia $L = R$;

3) variabila A este utilizată în continuare în blocul curent și se găsește un registru care poate fi eliberat (este posibil ca toți regiștri să fie ocupați); fie acesta R . Se eliberează registrul R prin salvarea valorii sale într-o locație de memorie și se ia $L = R$.

4) trebuie folosită chiar locația de memorie rezervată pentru A .

Algoritmul de generare a codului este următorul

Algoritmul 7.1 - (*GENCOD1*)

Intrare : O instrucțiune cu trei adrese de tipul $A := B \text{ op } C$

Ieșire : Codul obiect corespunzător acestei instrucțiuni

Metoda :

1. Cheamă funcția *GESTREG* pentru a determina locația L unde se va executa operația $B \text{ op } C$; în mod obișnuit L este un registru, dar poate fi și o locație de memorie.
2. Consultă mulțimea $Loc(B)$ pentru a determina locația curentă B' a lui B . Preferăm un registru pentru B' dacă valoarea lui B se află atât în memorie cât și într-un registru. Dacă valoarea lui B nu se află deja în L , generează instrucțiunea :

Informații
necesare la
generarea
codului
obiect

Algoritm de
generare cod
obiect,
folosind
procedura
GESTREG

MOV B', L

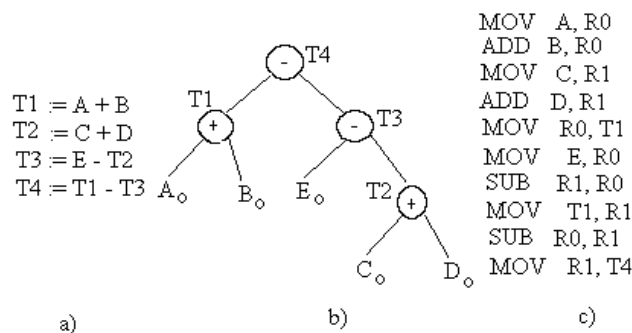
3. Generează instrucțiunea *op C', L* unde *C'* este locația curentă a lui *C*. Din nou preferăm un registru dacă valoarea lui *C* se află și în memorie și într-unul sau mai mulți regiștri.

II.7.3.2. Utilizarea GOFC în generarea codului obiect

a) Reordonarea nodurilor.

Avantajul utilizării grafurilor orientate fără cicluri în generarea codului obiect constă în posibilitatea alegerii unei secvențe de instrucțiuni cu trei adrese într-o ordine mai convenabilă pentru obținerea unui cod eficient: program mai scurt și variabile temporare mai puține.

Exemplul 7.1. Fie expresia $(A + B) - (E - (C + D))$. în figura 7.1.a este prezentat codul intermediar, în 7.1.b GOFC corespunzător iar în 7.1.c codul generat conform algoritmului anterior, considerând că avem disponibili doi regiștri.



Optimizarea
codului prin
rearanjarea
nodurilor

Să rearanjăm codul intermediar astfel încât T1 să apară imediat înaintea lui T4

T2 := C+D
T3 := E - T2
T1 := A+B
T4 := T1 - T3

Folosind din nou algoritmul *GENCOD1* obținem

MOV C, R0
ADD D, R0
MOV E, R1
SUB R0, R1

```

MOV A, R0
ADD B, R0
SUB R1, R0
MOV R0, T4.

```

Din exemplul anterior rezultă că îmbunătățirea codului generat s-a obținut prin plasarea instrucțiunii de calcul a lui $T1$ imediat înaintea instrucțiunii de calcul a lui $T4$, în cadrul căreia $T1$ este primul operand. Pe baza acestei observații putem afirma că, de regulă, obținem un cod mai bun dacă plasăm calculul celui mai din stânga operand chiar înaintea instrucțiunii care conține operația. Astfel operandul se va găsi într-un registru în momentul când avem nevoie de el, economisind astfel o memorare și o încărcare în registru. Pentru a obține lista nodurilor din GOFC ordonate conform acestei observații, se parcurge arborele de la rădăcină spre frunze listând întâi nodurile rezultat. Apoi, listăm nodurile operanți începând cu cel mai din stânga dacă toți părinții lui au fost evaluați. Ne interesează numai nodurile interioare, cărora le corespund temporare. Se obține secvența optimizată a instrucțiunilor în ordinea inversă.

Algoritmul 7.2 - de listare a nodurilor (LISTNOD)

Intrare : un GOFC

Ieșire : secvența optimizată a instrucțiunilor în ordine inversă

Metoda :

```

while mai există noduri interioare nelistate do
  begin
    selectează un nod interior nelistat  $n$ , ai cărui părinți au fost listați
    listează  $n$ 
    while cel mai din stânga descendent  $m$  al lui  $n$  are toți părinții
      listați și  $m$  nu este nod frunză do
      begin
        listează  $m$ 
         $n \leftarrow m$ 
      end
    end
  end
end

```

*Algoritmul de
optimizare
prin
rearanjarea
nodurilor*

b) Etichetarea nodurilor

Evitarea memorării rezultatelor parțiale, păstrând pe cât posibil aceste

valori în regiștri mașinii, necesită cunoașterea numărului de regiștri necesari generării codului pentru diferite secvențe de instrucțiuni cu trei adrese. Stabilirea acestui număr se numește *etichetare*. Etichetarea se realizează vizitând nodurile de jos în sus astfel încât un nod nu este vizitat atâta timp cât descendenții săi nu au fost etichetați. Etichetarea nodurilor se face conform algoritmului următor:

Algoritmul 7.3 - de etichetare

Intrare : nodul n împreună cu descendenții săi etichetați

Ieșire : eticheta lui n

Metoda :

```

if  $n$  este nod terminal
    then if  $n$  este descendentul cel mai din stânga al părintelui său
        then
            eticheta( $n$ )  $\leftarrow$  1
        else
            eticheta( $n$ )  $\leftarrow$  0
    else
        begin
            fie  $n_1, n_2, \dots, n_k$  descendenții lui  $n$  în ordinea dată de etichete :
                eticheta( $n_1$ )  $\geq$  eticheta( $n_2$ )  $\geq \dots \geq$  eticheta( $n_k$ )
            eticheta( $n$ )  $\leftarrow \max_{1 \leq i \leq k} (\text{eticheta}(n_i) + i - 1)$ 
        end
    end

```

*Determinarea
numărului de
regiștri
necesari*

În cazul când n este un nod binar cu descendenții având etichetele n_1 și n_2 , formula de etichetare a sa devine

$$\text{eticheta}(n) = \begin{cases} \max(n_1, n_2) & \text{dacă } n_1 \neq n_2 \\ n_1 + 1 & \text{dacă } n_1 = n_2 \end{cases}$$

Odată etichetat arborele, se poate genera codul folosind procedura *GENCOD2* pe care o prezentăm în continuare pentru cazul arborilor binari; extinderea sa la arbori cu mai mult de doi descendenți se poate face fără dificultate.

Algoritmul 7.4 - GENCOD2

Intrare : un arbore de derivare etichetat

Ieșire : codul obiect asociat arborelui

Metoda : Algoritmul este o procedură recursivă care aplicată unui nod generează codul arborelui dominat de acel nod .

Se utilizează :

a) o stivă *Stiva* care reține toți regiștri disponibili la un moment dat, din cei r ai mașinii; inițial stiva conține toți cei r regiștri. Pentru a lucra cu o stivă s se folosesc :

- procedura $PUSH(s, x)$ care depune în stiva s valoarea x
- funcția $POP(s)$ care descarcă stiva s
- funcția $VARF(s)$ care întoarce valoarea din vârful stivei
- procedura $PERM(s)$ care permută două valori din vârful stivei.

La ieșirea din $GENCOD2(n)$, registrul din vârful stivei este cel care conține valoarea calculată pentru nodul n .

b) o stivă *Temp* care menține lista variabilelor temporare $T0, T1, \dots$, pe care le poate folosi în continuare. Când cei r regiștri sunt ocupați, algoritmul ia o celulă temporară din vârful stivei.

procedure $GENCOD2(n)$

begin

if n este un nod frunză reprezentând operandul $nume$
și este cel mai din stânga descendent al tatălui său

then

{ cazul 1 }

write (' MOV ', $nume$, $VARF(Stiva)$)

else if n este un nod interior cu operatorul op ,

descendentul stâng n_1 și cel drept n_2

then

{ cazul 2 }

if $eticheta(n_2) = 0$

then

begin

fie $nume$ operandul reprezentat de n_2

call $GENCOD2(n_1)$

write (op , $nume$, $VARF(Stiva)$)

end

else if $1 \leq eticheta(n_1) < eticheta(n_2)$ și

*Algoritm
recursiv de
generare cod*

```

eticheta( $n_1$ ) <  $r$ 
    then
    { cazul 3 }
    begin
        call PERM ( Stiva )
        call GENCOD2 ( $n_2$ )
        reg  $\leftarrow$  POP ( Stiva )
        call GENCOD2 ( $n_1$ )
        write (op, reg, VÂRF(Stiva) )
        call PUSH (Stiva, reg )
        call PERM (Stiva )
    end
else if  $1 \leq eticheta(n_2) \leq eticheta(n_1)$  și
     $eticheta(n_2) < r$ 
    then
    { cazul 4 }
    begin
        call GENCOD2 ( $n_1$ )
        reg  $\leftarrow$  POP ( Stiva )
        call GENCOD2 ( $n_2$ )
        write (op, VÂRF(Stiva) , reg)
        call PUSH (Stiva, reg )
    end
else
    { cazul 5: ambele etichete sunt mai mari decât  $n$  }
    begin
        call GENCOD2 ( $n_2$ )
        t  $\leftarrow$  POP ( Temp)
        write ('MOV ', VÂRF(Stiva), t )
        call GENCOD2 ( $n_1$ )
        call PUSH (Temp, t )
        write( op, t, VÂRF (Stiva) )
    end
end

```

În funcție de nodul întâlnit, algoritmul se poate afla în una din situațiile următoare :

1) Nod frunză, fiind cel mai din stânga descendent al nodurilor tată. El are eticheta 1, deci necesită o încărcare într-un registru a valorii reprezentate de acest nod. Extragem registrul din vârful stivei *Stiva* și executăm instrucțiunea *MOV* de încărcare în el a operandului indicat de nod.

2) Nod interior, având ca descendent dreapta un nod frunză. Operația indicată de nodul curent poate fi realizată cu condiția evaluării în prealabil a descendentului stâng. Se apelează recursiv algoritmul *GENCOD2* pentru descendentul stâng, după care se generează codul operației nodului curent folosind registrul din vârful stivei *Stiva*.

3) Nod interior, având ca operanzi subarborii dominați de nodurile n_1 și n_2 cu $eticheta(n_1) < eticheta(n_2)$ și $eticheta(n_1) < r$. Procedura se apelează recursiv pentru a genera codul operandului doi și apoi al primului. Inversarea ordinei de generare este precedată de permutarea a doi regiștri din vârful stivei *Stiva*; este vorba de acei regiștri ce vor conține rezultatele operanzilor. Înainte de ieșirea din procedură, stiva se reface repermurând regiștri din vârful său.

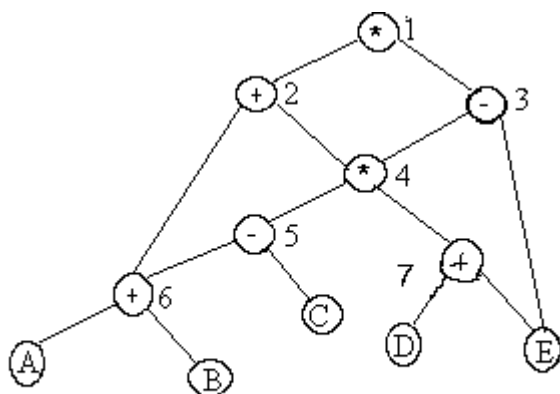
4) Nod interior în condițiile de la 3), doar că $eticheta(n_1) \geq eticheta(n_2)$ și $eticheta(n_2) < r$. Se execută operațiile de la 3) fără a mai fi nevoie de permutarea și refacerea celor doi regiștri din vârful stivei.

5) Nod interior, având operanzii subarbori dominați de nodurile n_1 și n_2 cu $eticheta(n_i) \geq r, i \in \{1, 2\}$. După generarea codului pentru al doilea operand se salvează conținutul registrului din vârful stivei (cel ce conține rezultatul operandului) într-o variabilă temporară extrasă din stiva *Temp*. După generarea codului pentru primul operand, variabila este reîntoarsă în *Temp*, iar codul de operație generat va conține referințe la variabila temporară și la registrul din vârful stivei.

Teme Curs

Teste Autoevaluare

1. Optimizați codul corespunzător expresiei $XABC * D + + \leftarrow$, în cazul când se folosește un singur registru..... 1 punct
2. Fiind dat codul cu trei adrese
 $T1 := A - B$
 $T2 := A - C$
 $T3 := T1 + T2$
 $T4 := T3 + T2$
să se genereze codul obiect folosind algoritmul GENCOD1..... 2 puncte
3. Folosind algoritmul de listare a nodurilor, rearanjați codul intermediar dat de arborele următor



..... 2 puncte

4. Folosind algoritmul GENCOD2, generați codul corespunzător expresiei $(A + B) - (E - (C + D))$ 4 puncte
- Oficiu..... 1 punct

Răspunsuri

1.

```
LOD  B
MUL  C
ADD  D
ADD  A
STO  X
```

2.

```
MOV  A, R0
SUB  B, R0
MOV  A, R1
SUB  C, R1
ADD  R1, R0
ADD  R1, R0
MOV  R0, D
```

3.

```
T7 := D+E
T6 := A+B
T5 := T6 - C
T4 := T5 * T8
T3 := T4 - E
T2 := T6 + T4
T1 := T2 * T3
```

4.

```
GENCOD2 (T4)      [R1 R0]
GENCOD2 (T3)      [R0 R1]
GENCOD2 (E)       [R0 R1]
MOV  E, R1
GENCOD2 (T2)  [R0]
GENCOD2 (C)  [R0]
MOV  C, R0
ADD  D, R0
SUB  R0, R1
```

GENCOD2 (T1)	[R0]
GENCOD2 (A)	[R0]
MOV A, R0	
ADD B, R0	
SUB R1, R0	

Rezumat: *S-au prezentat doi algoritmi de generare a codului obiect și câte un algoritm de optimizare din punct de vedere al numărului de instrucțiuni și al numărului de regiștri.*

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se vor implementa algoritmii de optimizare a codului folosind graful orientat fără cicluri și un algoritm de generare a codului obiect

Teme Laborator/Seminar

1. Implementați algoritmul de listare a nodurilor
2. Implementați algoritmul de etichetare a nodurilor
3. Implementați algoritmul GENCOD2

Rezumat: *se implementează un algoritm de generare cod obiect și algoritmii de optimizare*

Notatii

Descriere Generală

Se prezintă noțiuni generale cu privire la importanța tabelii de simboluri, diferite moduri de implementare și gestiune a tabelii de simboluri și reprezentarea identificatorilor ținând seama de domeniul de valabilitate.

Obiective

- cunoașterea structurii și utilității tabelii de simboluri
- cunoașterea principalelor moduri de organizare a tabelii de simboluri

Cuprins

II.8.1. Generalități

II.8.2. Organizarea tabelii de simboluri

II.8.2.1. Tabele neordonate

II.8.2.2. Tabele ordonate alfabetic

II.8.2.3. Tabele arborescente

II.8.2.4. Tabele dispersate

II.8.3. Reprezentarea identificatorilor ținând seama de domeniul de valabilitate

Conținut Curs

II.8.1. Generalități

Informația selectată de compilator în legătură cu numele simbolice care apar în programul sursă se află, de obicei, concentrată într-o structură numită tabela de simboluri. În cea mai simplă formă, tabela de simboluri apare ca un tablou de înregistrări, dar există și alte forme de reprezentare a sa: arbori, liste, etc. Tabela de simboluri este divizată în două părți :

- numele, care este un șir de caractere reprezentând identificatorul și
- informația care conține
 - tipul identificatorului: integer, real, record, array, etc.
 - utilizarea sa : parametru formal, etichetă, subprogram, etc.

*Generalități cu
privire la
structura și
utilizarea*

- domeniul de definiție: pentru un tablou, de exemplu, numărul dimensiunilor, limitele fiecărei dimensiuni, etc.
- adresa de memorie, etc.

Această divizare a tabelei de simboluri se face cu scopul de a economisi memorie. Numele simbolice devin la un moment dat inutile, cum se întâmplă la ieșirea din domeniul de valabilitate al variabilelor desemnate prin aceste nume. Spațiul folosit pentru păstrarea numelor poate fi reutilizat în cazul când numele memorate nu mai apar în program. Tabela de simboluri poate fi constituită dintr-o mulțime de tabele, fie specializate pentru variabile, proceduri, constante, etichete, etc., fie diferențiate pe baza lungimii numelui simbolic: tabele cu nume de cel mult patru caractere, tabele cu nume de cel mult 5-8 caractere, etc.

Tabela de simboluri este utilizată în diferite faze ale compilării:

- în faza de analiză lexicală și sintactică, când, la întâlnirea unui nume în program, compilatorul (de obicei analizorul lexical) verifică dacă numele respectiv se află memorat sau nu în tabelă. Dacă numele nu se află în tabelă, el este introdus, reținându-se adresa intrării;
- în faza de analiză semantică, pentru a verifica dacă utilizarea identificatorilor este în concordanță cu declarațiile;
- în faza generării de cod pentru a determina lungimea zonelor de memorie alocate variabilelor;
- în faza de tratare a erorilor, pentru a evita mesaje redundante.

Diferitele acțiuni executate de compilator asupra tabelei de simboluri sunt:

- verificarea dacă un nume este întâlnit pentru prima dată sau nu;
- adăugarea unui nume întâlnit pentru prima dată;
- adăugarea unei informații la un nume;
- ștergerea unui nume sau grup de nume.

Deoarece performanțele unui compilator depind în mare măsură de eficiența căutării în tabela de simboluri, iar căutarea are loc pentru fiecare simbol întâlnit, este foarte importantă metoda de organizare a tabelei de simboluri.

II.8.2. Organizarea tabelei de simboluri

O primă clasificare a tabelei de simboluri este dată de existența sau inexistența unor criterii de căutare în tabelă. Dacă asemenea criterii nu există, tabela este neordonată; dacă există un mecanism care să grăbească găsirea unui

simbol în tabelă, aceasta este ordonată. Tabelele ordonate se clasifică la rândul lor după mecanismul folosit pentru accesul la intrări.

II.8.2.1. Tabele neordonate

Organizarea cea mai simplă a unei tabele de simboluri constă în adăugarea secvențială de noi intrări pe măsură ce ele apar, în ordinea apariției. Organizarea tabelei de simboluri depinde de limbaj. Pentru un limbaj ce impune o lungime maximă asupra numelor, memorarea va fi diferită față de un limbaj care nu impune această cerință.

Când lungimea este limitată, se poate rezerva un spațiu maxim pentru a memora numele. Implementarea se face ca în figura 8.1.

Nume	Informații
Identifl	întreg,....., variabila simplă
A	șir,....., etichetă
B	real,....., variabilă simplă

*Tabele
neordonate*

Figura 8.1

Când lungimea nu este limitată, nu ne putem permite să rezervăm un spațiu maxim pentru a stoca numele, deoarece spațiul pierdut ar fi prea mare. În acest caz vom stoca numele sub forma unui pointer către tabela numelor. Vom avea, deci două tabele: una care conține numele și alta care conține informațiile și pointerii către prima. Lungimea numelor se stochează în tabela informațiilor sau în tabela numelor sau va fi implicită.

Dezavantajul reprezentării tabelei de simboluri sub formă de tablou constă în faptul că pentru identificarea unui nume este necesară căutarea sa începând cu prima înregistrare până când este găsit sau până la sfârșit. Căutarea poate fi optimizată dacă plasăm cele mai frecvent utilizate nume la începutul tabelei. Pentru aceasta se suprapune peste structura secvențială a tabelei, o structură de listă, ca în figura următoare. În acest caz, căutarea se face în ordinea dată de legături.

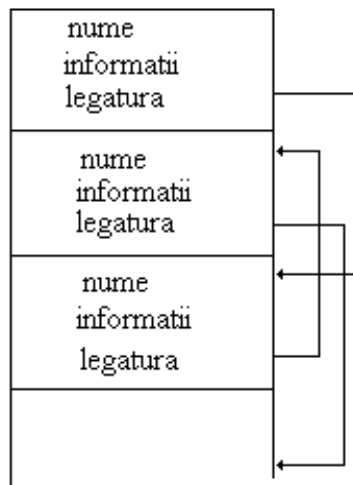


Figura 8.2

II.8.2.2. Tabele ordonate alfabetic

În tabelele ordonate alfabetic ordinea intrărilor este dată de ordinea alfabetică a numelor. Dacă tabela este un tablou de înregistrări, cea mai adecvată căutare este cea binară. Presupunând că sunt ocupate n intrări din tabelă, căutarea se face cu funcția Caută_bin(x, k)

```

p ← 1
u ← n
găsit ← fals
repeat
    i ← [(p+u)/2]
    if x < Nume[i]
    then u ← i-1
    else if x > Nume[i]
    then p ← i+1
    else begin
        găsit ← true
        k ← i
    end
until găsit or p > u
if not găsit then k ← 0

```

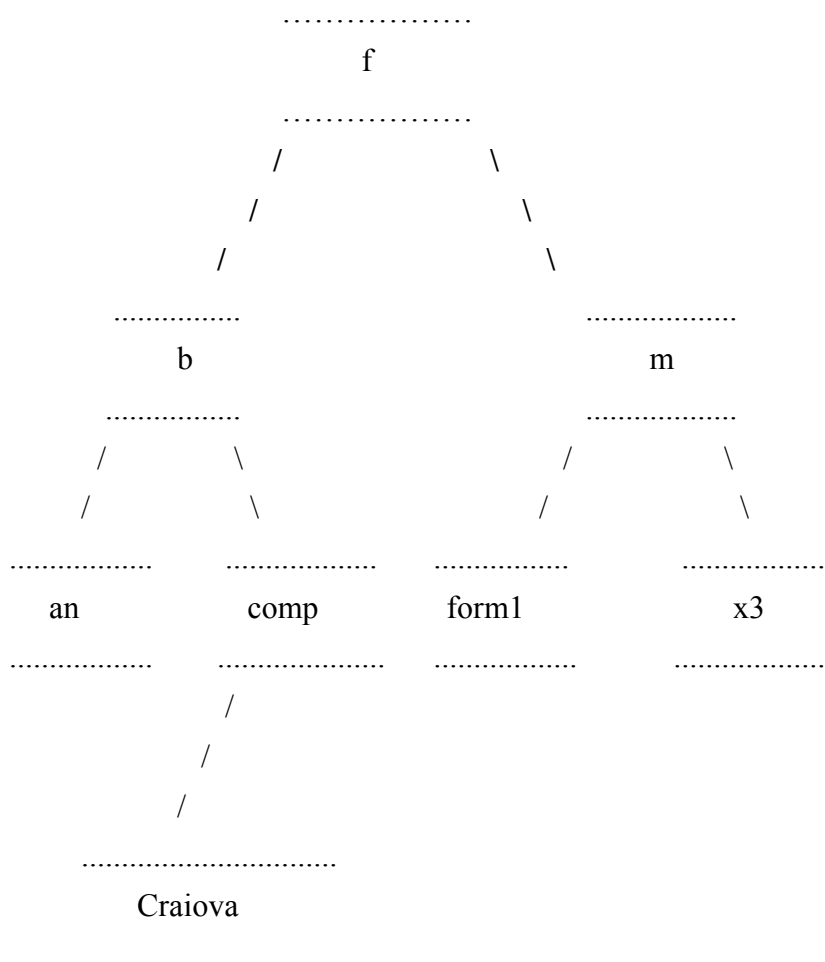
*Algoritmul de
căutare binară
într-o tabelă
ordonată
alfabetic*

unde $[x]$ reprezintă partea întreagă a lui x .

Procedura primește la intrare numele x și întoarce indexul k al intrării în care a găsit numele sau $k=0$, dacă nu l-a găsit. Înregistrările din tabelă sunt $R_1, R_2, \dots, R_n$ având numele $Nume_1, Nume_2, \dots, Nume_n$.

II.8.2.3. Tabele arborescente

Unele compilatoare folosesc arbori binari pentru reprezentarea simbolurilor. Fiecare nod reprezintă o intrare în tabelă; el conține pe lângă nume și atribute, o legătură spre stânga și una spre dreapta către arborii ce conțin nume „mai mici” și respectiv „mai mari” decât numele respectiv, în ordinea alfabetică (figura 8.3).



*Exemplu de
implementare
a unei tabele
arborescente*

Figura 8.3

II. 8.2.4. Tabele dispersate

Metoda de căutare prin dispersie folosește o funcție de dispersie care aplicată la un identificator produce o valoare întreagă nenegativă numită indexul de dispersie al identificatorului. Prin funcția de dispersie, mulțimea identificatorilor este partiționată într-un anumit număr de clase, identificatorilor dintr-o clasă corespunzându-le același index de dispersie.

O funcție de dispersie bună trebuie să fie ușor de calculat și să creeze clase echilibrate din punct de vedere al numărului de identificatori ce-l conțin. Prin dispersie, timpul de căutare al unui identificator se reduce la timpul de calcul al indexului identificatorului și la timpul de căutare într-o singură clasă.

Funcțiile de dispersie folosite în proiectarea tabelor de simboluri prelucrează de obicei reprezentarea internă a șirului de caractere ce formează identificatorul. Iată două astfel de funcții:

a) index cuprins între 0 și 255

- se adună ultimii 4 biți ai primelor două caractere ale identificatorului
- la rezultat se adună lungimea identificatorului înmulțită cu 16
- se rețin ultimii 8 biți ai rezultatului.

b) index cuprins între 0 și 210 (și calculator avînd cuvîntul de memorie format din 4 octeți)

- se adună cuvintele ce conțin caracterele identificatorului
- se împarte rezultatul la 211 și se reține restul.

Dacă funcția de dispersie este bună, iar numărul indecșilor este destul de mare, este posibil ca, într-un program nu prea lung, să se disperseze pentru fiecare index cel mult un identificator. În general, însă, nu putem evita dispersarea a mai mult de un identificator pentru un index, fenomen numit coliziune. Principala tehnică de rezolvare a coliziunilor este înlănțuirea, care poate fi utilizată sub diverse forme; ne vom opri la cea numită înlănțuire separată. Această metodă implică înlănțuirea înregistrărilor ce colizionează, într-o zonă specială, separată de prima, conform figurii 8.4.

*Funcții de
dispersie*

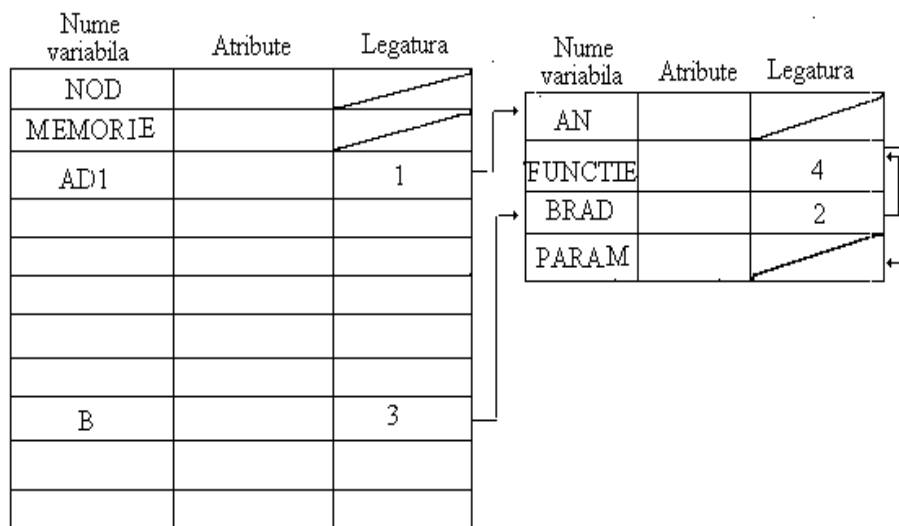


Tabela dispersată

Figura 8.4

II.8.3. Reprezentarea identificatorilor ținând seama de domeniul de valabilitate

În majoritatea limbajelor de programare, în cadrul unui program se poate folosi același identificator pentru a desemna variabile diferite, cu atribute diferite și zone de memorie diferite, alocate în faza de execuție. Tabela de simboluri trebuie să păstreze pentru fiecare variabilă intrări distincte. Deci, trebuie să existe un mecanism care la apariția unui identificator să selecteze intrarea corectă din mulțimea intrărilor ce conțin același nume simbolic. Dar, semnificația unei variabile rezultă din modul în care se stabilește domeniul de valabilitate al variabilelor din limbajul respectiv. Aceasta se poate realiza ținând seama de blocul în care apare identificatorul respectiv. Un identificator va fi reprezentat prin numele său și prin numărul blocului în care a fost declarat. Acest număr nu apare obligatoriu în mod explicit; el poate fi dedus plecând de la poziția identificatorului în tabela de simboluri.

Pentru a determina domeniul de valabilitate al unui identificator se definește noțiunea de bloc activ sau inactiv. Un bloc este activ atâta timp cât nu a fost întâlnit sfârșitul său. Când se analizează procedura 4 din figura 8.5, procedurile 1, 2 și 4 sunt active, iar 3 este inactivă.

```

PROCEDURE _ _
__ BEGIN
|   PROCEDURE _ _
|   __ BEGIN
|   |   PROCEDURE _ _
|   |   __ BEGIN
|   |   |   •
|   |   3|   •
|   |   |   •
|   |   ---- END
1| 2|   PROCEDURE _ _
|   |   __ BEGIN
|   |   |   •
|   |   4|   •
|   |   |   •
|   |   --- END
|   --- END
--- END

```

Structură de
tip bloc

Figura 8.5

Analiza lexicală folosește identificatorul unei variabile pentru a căuta descrierea sa în tabela de simboluri. O dată găsită, identificatorul poate fi înlocuit cu un indicator la această intrare ce va fi utilizat în generarea codului intermediar. După analiza zonei ce reprezintă domeniul de valabilitate al unei variabile, memorarea identificatorului nu mai este necesară; rămân necesare în continuare atributele. Referințele din codul intermediar vor fi spre această parte și nu neapărat spre numele simbolic, care poate lipsi.

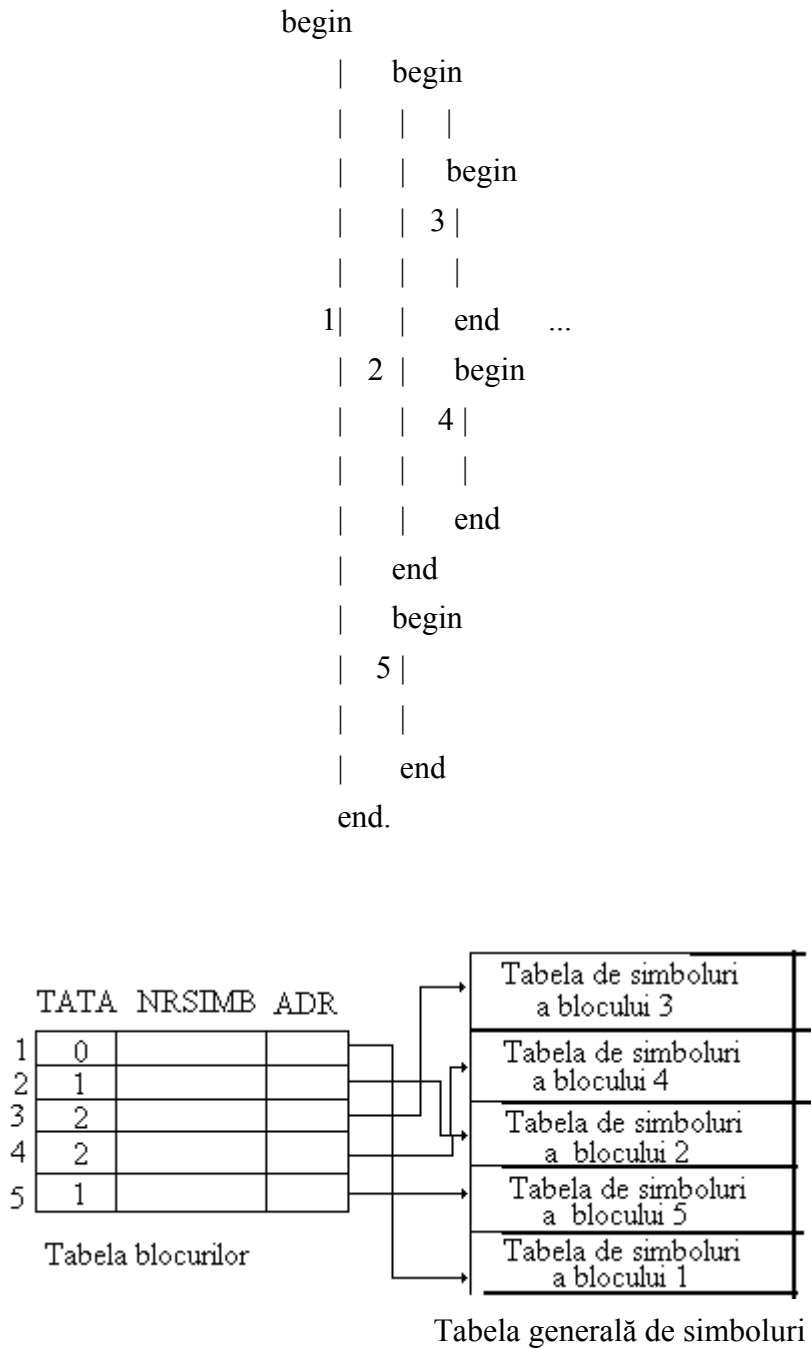
Pentru limbajele de tip “bloc” (de exemplu, *PASCAL*, *C*) trebuie ținut cont de incluziunea unul în altul a domeniilor de valabilitate reprezentate de blocuri, proceduri, cicluri. La întâlnirea unui identificator, el trebuie căutat în declarațiile celui mai apropiat bloc înconjurător, iar dacă nu este găsit este căutat în blocul înconjurător acestuia ș. a. m. d. Deci, tabela de simboluri trebuie să reflecte structura de bloc a limbajului. Este necesar ca:

- orice început de bloc să deschidă un domeniu de valabilitate, deci o tabelă de simboluri locale blocului;
- în orice moment trebuie să putem căuta nu numai în tabela de simboluri a

blocului curent, ci și în tabelele blocurilor înconjurătoare; deci este necesară o legătură între aceste tabele;

- la sfârșitul unui bloc, variabilele declarate în acest bloc devin inutile, deoarece atributele lor sunt completate și pe durata analizei nu vor mai fi folosite; este necesară stocarea lor în vederea fazelor ulterioare ale compilării.

O soluție de implementare a unui program de tip Pascal este dată în figura 8.6.



*Implementare
care ține
seama de
structura
blocurilor*

Figura 8.6

Tabela blocurilor păstrează (prin câmpul *Tata*) înlănțuirea blocurilor, numărul de simboluri declarate în fiecare bloc (prin câmpul *Nrsimb*) și adresa din tabela de simboluri generală la care începe tabela de simboluri dedicată fiecărui bloc. În continuare vom da un alt exemplu de implementare folosind tabele dispersate. Fiecare bloc primește un număr; acesta, împreună cu noțiunea de bloc activ, permit determinarea corespondenței între utilizarea unui identificator și declararea sa. Când un bloc este deja analizat el devine inactiv iar variabilele locale lui nu pot fi utilizate în alte blocuri decât dacă sunt redefinite. Blocurile inactive pot fi eliminate din tabela de legături, dar informația lor trebuie conservată pentru fazele următoare.

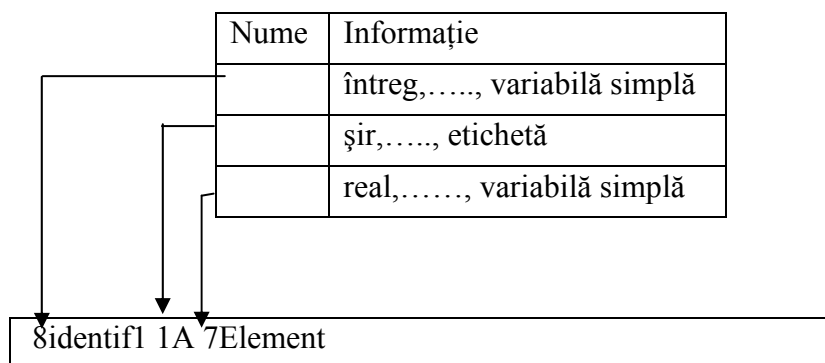
Teme Curs

Teste Autoevaluare

1. De ce se recomandă divizarea tabelii de simboluri?..... 1 punct
 2. Pentru ce este necesară tabela de simboluri în faza generării de cod?..... 1 punct
 3. Dați un exemplu de implementare a unei tabeli neordonate care nu folosește lungimea identificatorilor..... 2 puncte
 4. Care este dezavantajul utilizării unei tabeli arborescente? Dar avantajul?..... 2 puncte
 5. Care este principala caracteristică a tabelilor dispersate? Dar avantajul?..... 1 punct
 6. Ce informații se stochează în partea a doua a unei tabeli dispersate?..... 1 punct
 7. Prin ce se definește domeniul de valabilitate al unei variabile?..... 1 punct
- Oficiu..... 1 punct

Răspunsuri

1. Pentru a economisi memorie
2. Pentru a determina lungimea zonelor de memorie alocate
- 3.



4. Dezavantaj: poate genera arbori dezechilibrați. Avantaj: este foarte utilă în optimizarea codului
5. Realizează împărțirea identificatorilor în clase. Căutarea secvențială se face doar la nivelul unei clase
6. Cele care colizionează cu cele din prima
7. Prin blocul cel mai interior care o conține

Rezumat: S-au prezentat modalități de construire a tabelii de simboluri și de gestionare a informațiilor conținute.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se implementează metodele de creare și gestiune a tabelilor de simboluri ordonate.

Teme Laborator/Seminar

1. Implementați algoritmul de utilizare a unei tabeli de simboluri ordonată

alfabetic.

2. Implementați algoritmul de lucru cu o tabelă arborescentă.
3. Implementați algoritmul de înlănțuire separată.

Rezumat: *se implementează algoritmi de utilizare a tablei de simboluri*

Notății

Descriere Generală

Se prezintă sursele erorilor și principalele erori care se întâlnesc în fazele de analiză lexicală, sintactică, semantică și la executarea programelor.

Obiective

Cunoașterea surselor și a principalelor tipuri de erori care apar în faza de compilare și execuție a programelor

Cuprins

II.9.1 Sursele erorilor

II.9.2 Erori în analiza lexicală

II.9.3 Erori în analiza sintactică

II.9.4 Erori semantice

Conținut Curs

II.9.1 Sursele erorilor

Programele pe care le scriem sunt rareori corecte; ele prezintă diferite tipuri de erori pe care compilatorul trebuie să le detecteze și să le semnaleze clar. Un mesaj de eroare trebuie :

- să se exprime în termenii programului sursă și nu în termeni legați de reprezentarea internă
- să localizeze corect eroarea; este corect mesajul „ identificatorul *ID* n-a fost declarat în procedura *PROC* “ și nu „ lipsește declararea identificatorului *ID* “
- să nu fie redundant.

Strategia utilizată de majoritatea compilatoarelor constă în afișarea liniei eronate cu un marcator pe locul unde a fost detectată eroarea. Eroarea reală se găsește, în general, fie în locul marcat, fie cu câteva simboluri mai în față. În unele cazuri compilatorul nu poate să dea nici-o informație cu privire la eroarea întâlnită ; de exemplu, cazul când cuvântul cheie *END* este omis (în Pascal).

Erorile unui program pot fi detectate fie la compilare, fie la execuție, diferitele surse de erori fiind :

- erori generate de compilator; ele sunt detectate, în general, la execuție, dar din fericire sunt foarte rare ;
- erori datorate limitelor compilatorului (dimensiunea mare a programului compilat, numărul maxim de simboluri stocate în tabela de simboluri, etc) , limite ce nu apar în specificarea limbajului;
- erori lexicale; ele corespund inserării sau absenței unui caracter sau înlocuirii unui caracter prin altul. Aceste erori sunt ușor de detectat și recuperat .
- erori sintactice și semantice; compilatorul poate corecta unele din ele.

*Surse ale
erorilor*

Erorile sintactice sunt detectate în faza de analiză lexicală și sintactică. Exemplele de mai jos ne dau o imagine generală asupra acestor tipuri de erori :

- eroare de punctuație : utilizarea virgulei în loc de punct și virgulă

Function f (x : integer , y : integer) : real ;

- eroare de inserare :

if x : = y then writeln (x) else writeln (y) ;

sau *if x = y then writeln (x) ; else writeln (y) ;*

- erori dificil de semnalat

if x = 2+y-x) then writeln (x) else writeln (y) ;

În această instrucțiune simbolul „) ” este în plus sau lipsește „ (“. Aceste erori sunt dificil de corectat, trebuind să ghicim intenția programatorului;

- erori detectate mai târziu

if x + 2-y = y then writeln (x) else writeln (y) ;

În acest caz lipsește un spațiu între *if* și variabila *x*. Această eroare este detectată la întâlnirea caracterului „t” al lui *then*. Erorile de acest tip nu pot fi detectate imediat, fiind necesar un mijloc de a le repera și de a reveni la locul unde au fost detectate. Dacă prezența separatorilor este obligatorie, la întâlnirea lui „ if “, compilatorul va afișa mesajul „identificator nedeclarat”. Erorile semantice sunt detectate la :

*Exemple de
erori
sintactice*

- compilare , în timpul fazei de analiză sintactică :

- incompatibilitate de tip între operanzi și operatori, între parametrii actuali și cei formali, etc.

- identificator nedeclarat sau multiplu declarat

- la execuție

- împărțirea prin zero
- citirea unui fișier după închiderea sa

*Exemple de
erori
semantice*

- afectarea unei valori care nu aparține domeniului unei variabile: indice de tablouri , selectorul instrucțiunii case , etc .

Recuperarea erorilor . Odată detectată o eroare, compilatorul trebuie s-o semnaleze programatorului și să încerce s-o repare. Compilatoarele diferă după metoda de reparare a erorilor și după maniera de continuare a analizei după apariția unei erori. Există compilatoare care realizează :

- *redresarea* : toate activitățile compilatorului, altele decât analiza lexicală și sintactică, încetează după detectarea primei erori. Majoritatea compilatoarelor lucrează în această manieră .

- *recuperarea* : un compilator mai complex caută să repare eroarea, adică să transforme intrarea eronată într-una echivalentă, dar autorizată. Totuși, rezultatul nu este întotdeauna cel scontat; recuperarea unei erori sintactice poate duce la una semantică.

- *corectarea* : un compilator și mai complex caută să corecteze intrarea eronată încercând să ghicească intenția programatorului; un exemplu este compilatorul PL/C. Aceste corecții pot fi efectuate însă pentru un domeniu restrâns de erori.

*Eliminarea
erorilor*

II.9.2 Erori în analiza lexicală

Analizorul lexical este un automat finit; simbolurile de intrare îi permit trecerea de la o stare la alta. Dacă s-a ajuns în starea finală, a fost recunoscut un atom lexical. Dacă automatul se blochează pe stare nefinală, trebuie invocată o rutină de tratare a erorilor. Din păcate, acestea nu dispun de soluții miraculoase. Când un simbol de intrare nu permite nici-o trecere, unele rutine îl ignoră, iar altele îl înlocuiesc printr-unul ce permite trecerea.

O soluție mai elaborată constă în existența unui dialog între analizorul lexical și cel sintactic. De exemplu, analizorul sintactic poate cere celui lexical ca următorul atom să aparțină unei anumite clase de atomi. Astfel, analizorul lexical este orientat în căutarea dar și în repararea erorii. La rândul său, analizorul lexical poate raporta analizorului sintactic tipul eventualelor transformări presupuse de repararea erorii.

*Tratarea
erorilor
lexicale*

O categorie frecvent întâlnită de erori detectate de analizorul lexical se datorează scrierii greșite a cuvintelor cheie. Repararea este în acest caz posibilă datorită numărului relativ mic de posibilități de reparare. Tratatamentul este dificil pentru erori de tipul :

- câmp : = „acesta este un șir

când apostroful final este omis ;

- expresie : = AB

când este omis operatorul, AB fiind luat ca identificator.

II.9.3 Erori în analiza sintactică

Erorile detectate în timpul analizei lexicale nu sunt obligatoriu erori sintactice propriu-zise. De exemplu instrucțiunea

if x = y then writeln (x) ; else writeln (y) ;

conține un punct și virgulă inutil. Această eroare va fi detectată de analizorul lexical, dar ea este o eroare de inserare. Există mai multe metode de tratare a erorilor sintactice .

Recuperarea în mod panică este metoda cea mai folosită și cea mai ușor de implementat, fiind utilizată de toate tipurile de analizori, fără nici-o condiție. Când este întâlnit un simbol eronat, se înlătură simbolurile următoare până se întâlnește unul de sincronizare cum sunt punct și virgulă sau end. Se procedează la fel și cu simbolurile din vârful stivei.

*Tratarea
erorilor de
sintaxă*

Ștergerea de simboluri constă în ștergerea tuturor simbolurilor de intrare care nu dau o acțiune legală de executat. La fel ca precedenta, această metodă ignorează erorile ce pot apare datorită simbolurilor șterse.

Inserarea de simboluri. Când un simbol de intrare, combinat cu cel din vârful stivei, nu dă o acțiune legală, se inserează la intrare un simbol care dă o astfel de acțiune .

Tratarea erorilor în analiza LL(1)

Poate fi aplicată oricare din metodele anterioare, dar strategia cea mai importantă constă în completarea căsuțelor libere din tabloul de analiză cu pointeri către rutinele de tratare a erorilor. Aceste rutine au două funcții

- schimbă, inserează sau suprimă simboluri din câmpul de intrare
- modifică, în mod corespunzător, stiva .

II.9.4 Erori semantice

Sursa principală a erorilor semantice o constituie utilizarea incorectă a identificatorilor sau a expresiilor ce desemnează variabilele din program. Prima întâlnire a unui asemenea identificator sau expresie trebuie să determine afișarea

unui mesaj de eroare dar și înregistrarea sa în tabela de simboluri. Această înregistrare este necesară în vederea viitoarelor întâlniri ale acestui identificator: o operație identică din punct de vedere al atributelor nu trebuie să provoace emiterea aceluiași mesaj, iar aparițiile distincte trebuie înregistrate în tabelă. Problema recuperării din erori semantice este legată de suprimarea unor mesaje de eroare.

a) Mesajele datorate recuperării defectuoase a unui identificator sau expresii trebuie suprimate. Cazul cel mai frecvent este cel al variabilelor indexate de forma $A[ind_1, ind_2, \dots, ind_n]$ în cazul când A nu a fost declarat ca tablou sau declarația a fost greșită și, deci, ignorată. La întâlnirea lui „ $A[$ “ se dă un mesaj privind incompatibilitatea folosirii lui A în raport cu declarația. De asemenea la întâlnirea lui „ $]$ “ analizorul va verifica numărul de dimensiuni cu cel declarat; neconcordanța va determina emiterea unui nou mesaj. În cazul când identificatorul eronat este interpretat ca unul corect, se crează o intrare în tabela de simboluri având ca valori de atribut cele rezultate din contextul utilizării. În cazul prezentat anterior, intrarea va conține pentru attributele „tip identificator“ și „număr dimensiuni“ valorile „tablou“ și n . La întâlnirea unei situații de eroare datorate acestei „corectări“, faptul că vom căuta în înregistrarea corectată și nu în una normală, permite să evităm tipărirea mesajului.

b) Mesajele datorate utilizării incorecte de mai multe ori a aceluiași identificator sau expresii trebuie suprimate. Cazul cel mai frecvent este cel al omiterii declarațiilor. Apariția repetată a unui identificator nedeclarat poate determina emiterea de mai multe ori a aceluiași mesaj de eroare. Prima apariție a unui astfel de identificator trebuie să determine formarea unei intrări în tabela de simboluri, dedicată identificatorului. Atributele asociate intrării sunt cele rezultate din context. La întâlnirea identificatorului în aceeași situație, nu se va mai emite mesaj de eroare. Dacă identificatorul este folosit și în alte situații (de exemplu, este de alt tip), acestea trebuie memorate pentru a se evita mesaje identice. În acest scop, intrarea oricărui identificator va conține și un indicator spre o listă a tuturor modurilor incorecte în care a fost folosit identificatorul. Lista aceasta servește pentru a tipări doar mesajele de erori distincte.

*Tratarea
erorilor
semantice*

Teme Curs

Teste Autoevaluare

1. Care sunt caracteristicile mesajului de eroare..... 2 puncte
 2. Care sunt sursele erorilor?..... 2 puncte
 3. Când se detectează erorile sintactice..... 1 punct
 4. Când se detectează erorile semantice..... 1 punct
 5. Care sunt metodele de reparare a erorilor?..... 1 punct
 6. Care sunt metodele de tratare a erorilor sintactice si in ce consta?..... 2 puncte
- Oficiu..... 1 punct

Răspunsuri

1. - să se exprime în termenii programului sursă și nu în termeni legați de reprezentarea internă

- să localizeze corect eroarea
- să nu fie redundant.

2. - erori generate de compilator; ele sunt detectate, în general, la execuție, dar din fericire sunt foarte rare ;

- erori datorate limitelor compilatorului (dimensiunea mare a programului compilat, numărul maxim de simboluri stocate în tabela de simboluri, etc) , limite ce nu apar în specificarea limbajului;
- erori lexicale; ele corespund inserării sau absenței unui caracter sau înlocuirii unui caracter prin altul. Aceste erori sunt ușor de detectat și recuperat .
- erori sintactice și semantice; compilatorul poate corecta unele din ele.

3. În fazele de analiză lexicală și sintactică

4. La analiza sintactică și la execuție

5. Redresarea, recuperarea și corectarea

6. Recuperarea în mod panică: când este întâlnit un simbol eronat, se înlătură simbolurile următoare până se întâlnește unul de sincronizare cum sunt punct și virgulă sau end.

Stergerea de simboluri constă în ștergerea tuturor simbolurilor de intrare care nu dau o acțiune legală de executat.

Inserarea de simboluri. Când un simbol de intrare, combinat cu cel din vârful stivei, nu dă o acțiune legală, se inserează la intrare un simbol care dă o astfel de acțiune

Rezumat: S-au prezentat modalități de tratare a erorilor întâlnite în fazele de analiză.

Lucrari Practice (Laborator/Seminar)

Conținut Laborator/Seminar

Se implementează un algoritm care să corecteze erorile sintactice prin una din metodele: recuperare în mod panică, ștergere de simboluri, inserare de simboluri

Teme Laborator/Seminar

1. Scrieți un program care să elimine erorile din analiză lexicală

Rezumat: <i>se implementează un algoritm de corectare a erorilor sintactice</i>
--

Notății

BIBLIOGRAFIE

- [1] A. V. Aho, R. Sethi, J. D. Ullman: Compilers, Principles, techniques and tools, Addison-Wesley, 1986
- [2] A. Dincă, M. Andrei: Limbaje formale, Editura Universitaria, Craiova, 2002
- [3] D. Gries, Compiler construction for digital computers, Wiley and Sons, New York, 1971
- [4] A. Hajjam El Hassani, Les compilateurs el leur principes, Edition Scientifiques et Techniques, Ecole Superieure des Sciences Appliquees pour l'Ingenieurs, Mulhouse, 1993
- [5] I. Iancu : Teoria compilatoarelor, Editura « Vlad & Vlad», Craiova, 1997
- [6] I. Iancu, M. Andrei: Teoria compilatoarelor si semantica limbajelor de programare. Indrumar de laborator, Reprografia Universitatii din Craiova, 1998.
- [7] I. Iancu: Proiectarea compilatoarelor , Editura Universitaria, Craiova, 2002
- [8] C. Ionescu Texe, I. Zsako: Structuri arborescente si aplicatiile lor, Ed. Tehnica, Bucuresti, 1990
- [9] L. Livovschi, C. Popovici, H. Georgescu, N. Țăndăreanu : Bazele informaticii, Editura Didactică și Pedagogică, București, 1981
- [10] A. B. Pyster, Compiler design and construction. Tools and techniques, Van Nostrand Reinhold, New York, 1988
- [11] L. D. Șerbănați, Limbaje de programare și compilatoare, Editura Academiei, București, 1987
- [12] J. P. Tremblay, P. G. Sorenson, The theory and practice of compiler writing, McGraw-Hill, Inc., New York, 1985